

Minecraft 완전형 멀티모달 모드·맵 제작 AI 시스템 - 이중 배치 프로파일 확정판

범용 프롬프트 자동 조사 - 게임 구상 RAG - 사용자 승인 - 원격 최신 모델 모드 / T4 완전 로컬 모드 - 모드·맵·에셋 공동 생성 - 실제 런타임 인증

PROFILE F: 최신 API·원격 모델 품질 우선 | PROFILE L: T4 16GB 로컬 추론 전용 | 공통 IR·MCP·RAG·검증 계약 | 2026-07-28

핵심 결론 현재 공개된 오픈소스와 표준을 연결하면 첫 제품을 만들 수 있다. 다만 하나의 거대 멀티모달 모델이 모든 작업을 담당하는 구조가 아니라, 중앙 오케스트레이터가 코드 LLM, VLM, 이미지 생성기, 3D/모션 변환기, 음성 모델 및 Minecraft 전용 MCP 를 순차 호출하고 독립 검증기가 결과를 확인하는 구조여야 한다.

작성 기준일: 2026 년 7 월 28 일

대상: Minecraft Java Edition 완전형 모드 + 전용 맵/월드젠 + 리소스팩 + 데이터 + 문서 + 배포

문서 상태: v6 - 최신 대형 모델을 API/원격으로 쓰는 Frontier Connected 모드와, 원격 AI 추론 없이 T4 16GB 에서만 완결되는 Local Only 모드를 분리·확정

웹 조사 기반 - 공식 모델 카드·공식 API 문서·저장소·1 차 연구 우선 - 특정 모델명을 영구 고정하지 않음 - Model Registry MCP 가 출시·폐기·라이선스·도구호출·성능·비용·실측 VRAM 을 재검증

목차

1. 최종 판단과 시스템 목표
 2. 사용자 승인형 제작 경험
 3. 전체 시스템 아키텍처
 4. 공통 상태 모델과 파일 의존성
 5. 연결할 AI 모델과 T4 운용
 6. MCP 및 스킬 카탈로그
 7. RAG 및 최신 트렌드 지식 계층
 8. 텍스처/이미지 생성과 시각 검증
 9. 3D 모델, 리깅, 모션 및 애니메이션
 10. 음성 인식, 음성 명령, TTS와 사운드
 11. 코드/데이터/월드젠 제작 파이프라인
 12. 에이전트 오류율을 낮추는 연구 기반 설계
 13. 반복 검증 및 실패 수정 루프
 14. 보안 및 공급망 방어
 15. 최종 모드 재투입, 보완, 확장 및 장기 진화
 16. 프롬프트 및 MCP 호출 명세
 17. T4 배치 구조와 자원 계획
 18. 단계별 구현 로드맵과 합격 기준
 19. 위험, 한계 및 현실적 범위
 20. 참고 문헌과 공식 자료
 21. 2026-07 MCP 사양 변경과 개정 원칙
 22. 실제 사용할 MCP/API/스킬 전수 카탈로그
 23. 서버별 정확한 도구 표면과 권한 프로파일
 24. 에이전트 관련 연구를 구현 규칙으로 변환
 25. 완전형 RAG: 버전/코드/트렌드/실패/보안 지식 계층
 26. 멀티모달 모델 라우팅과 T4 GPU 임대 구조
 27. 사용자 승인부터 릴리스 재투입까지의 전체 실행 흐름
 28. 오류율을 낮추는 검증 피라미드와 반복 수리 알고리즘
 29. MCP/에이전트/RAG 공급망 보안
 30. 실제 구현 서비스 토폴로지와 저장소 구조
 31. 단계별 개발 일정, 자원, 합격 기준
 32. 벤치마크, 관측성, 운영 지표와 중단 기준
 33. 최종 권고안과 구현 우선순위
- 부록 C. 핵심 MCP JSON Schema 예시
- 부록 D. 연구-설계 추적성 매트릭스
- 부록 E. 2026-07 추가 참고문헌 감사 목록
34. v3 완전성 감사: 정말 Minecraft 모드에 필요한 모든 것을 만드는가
 35. 게임 구상 RAG와 사용자 승인형 Game Design Compiler

- 36. 모드-맵 공동 설계: World Design IR와 콘텐츠 의존 그래프
- 37. Minecraft 없이 만드는 Virtual Minecraft Twin과 VLM 검증
- 38. 모든 관련 오픈소스의 역할·채택·대체 경로
- 39. 최종 산출물, 모드 설명, 배포 및 재투입 계약
- 40. 확정 아키텍처, 단계별 합격 기준과 현실적 한계
- 41. 범용 핵심 프롬프트 자동 조사·Reference Game Resolver와 Skill/MCP 실행 계약
- 부록 F. v3 핵심 JSON Schema와 실행 계약
- 부록 G. v3 추가 공식 자료와 연구 문헌
- 부록 H. v4 범용 프롬프트·웹/RAG·Skill/MCP 추가 자료
- 42. 2026 최신 모델 레지스트리와 역할 분리 교정
- 부록 I. v5 최신 모델 공식 자료
- 43. v6 이중 배치 프로필: Frontier Connected와 T4 Local Only
- 부록 J. v6 모델·API 공식 자료

v5 핵심 교정: 기획 AI와 코딩 AI를 분리한다

이전 판의 가장 큰 오류는 Qwen2.5-Coder 계열을 중앙 기획/코드 모델로 함께 배치한 것이다. 게임 기획, 광범위 웹 조사, 요구사항 충돌 해결과 모드-맵 공동 설계는 범용 장기 추론 모델의 역할이며, 코딩 모델은 승인된 계획을 구현하는 하위 작업자다.

2026-07 기준 최신 후보를 공식 모델 카드로 다시 조사했으며, 앞으로도 특정 모델명을 고정하지 않는다. Model Registry MCP가 새로운 공개 모델을 발견하고, 내부 역할별 골든셋을 실행한 뒤 champion/challenger 방식으로 모델을 교체한다.

역할	기본 선택	T4 대체	금지 원칙
게임 기획·Research Supervisor	GLM-5.2 또는 Kimi K3/K2.6	Qwen3.5-9B/Gemma 4 12B 양자화	코딩 특화 모델을 중앙 기획자로 사용하지 않음
코드 Engineer	Kimi K2.7-Code, Qwen3-Coder-Next, GLM-5.2	Qwen3.5-9B local patch	사용자 승인 전 코드 생성 금지
시각 Reviewer	Qwen3-VL-8B	동일	자기 생성 결과를 같은 모델 하나로만 심사 금지
이미지 Worker	FLUX.2-klein-4B	동일, 단독 GPU lease	생성 이미지를 Minecraft PNG로 직접 사용 금지

1. 최종 판단과 시스템 목표

이 시스템의 목표는 사용자가 자연어, 음성, 참고 이미지 또는 기존 프로젝트를 제공하면 AI가 모드의 범위와 구현 계획을 먼저 제안하고, 사용자가 수정하거나 승인한 뒤에만 코드와 에셋을 생성하는 것이다. 생성 후에는 컴파일 성공만 확인하는 것이 아니라 게임 안에서 기능, 화면, 성능, 보안, 호환성을 검증하고, 완성된 모드 자체를 다시 지식베이스에 넣어 다음 개선과 확장에 사용할 수 있어야 한다.

구현 가능성 T4는 16GB GDDR6를 제공하므로 9B급 양자화 범용 모델, 8B급 양자화 VLM, FLUX.2-klein/경량 diffusion 계열 이미지 모델을 각각 순차 실행하는 것은 가능하다. 그러나 이 모델들을 동시에 상주시킬 수는 없으며, 대형 3D 및 모션 생성은 선택 작업 또는 외부 강한 GPU 경로로 분리해야 한다. [R01][R02][R03]

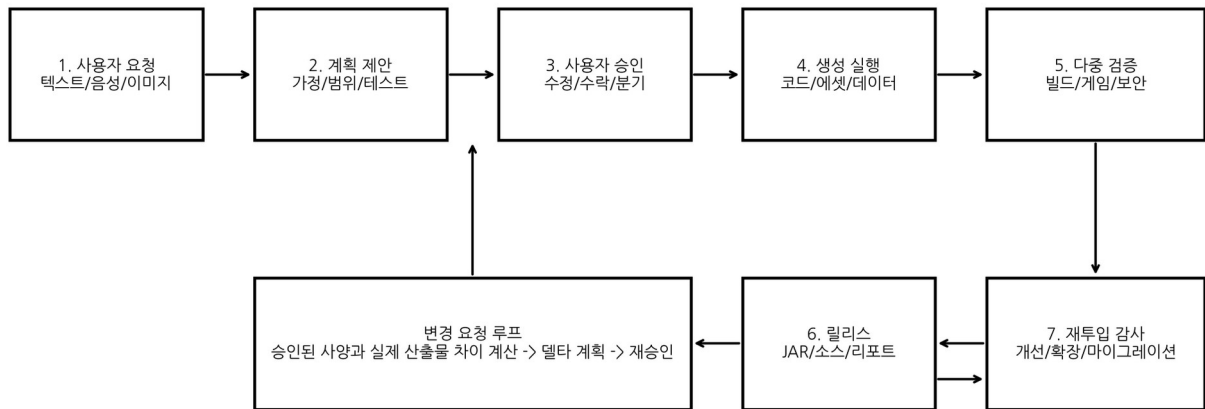


그림 1. 사용자 승인부터 최종 모드 재투입까지의 전체 수명주기

1.1 제품이 수행해야 하는 일

- 요구사항을 기능, 콘텐츠, 플랫폼, 버전, 리소스, 테스트 조건으로 분해한다.
- 맵, 데이터팩, Paper 플러그인, Fabric/NeoForge 모드, 리소스팩 중 최소 침습 구현을 선택한다.
- 아이템, 블록, 몹, 보스, GUI, 사운드, 구조물마다 필요한 코드 및 리소스 파일 전체를 의존성 그래프로 계산한다.
- 코드와 에셋을 생성한 뒤 정적 검사, 빌드, GameTest, 실제 서버 실행, Mineflayer 플레이, VLM 화면 검사를 거친다.
- 출처와 라이선스, 모델/프롬프트, 생성 설정, 파일 해시, 승인 사양을 모두 기록한다.
- 릴리스 결과를 다시 분석하여 보안 취약점, 성능 병목, 콘텐츠 확장 지점, 버전 마이그레이션 위험을 제한한다.

1.2 처음부터 제외해야 하는 잘못된 구조

잘못된 접근	왜 실패하는가	대체 설계
하나의 LLM이 모든 Java/JSON/PNG/모션을 직접 생성	출력 형식별 오류 모드가 다르고 컨텍스트가 팽창한다. 시각 품질과 런타임 동작을 텍스트 모델이 검증하기 어렵다.	역할별 모델 + 공통 IR + 독립 검증기
모든 MCP 도구를 한 번에 노출	도구 선택 정확도와 프롬프트 효율이 떨어지고 공격면이 커진다.	계층형 도구 라우팅, 한 단계에 5-10개 도구만 노출
컴파일 성공을 완료로 판정	리소스 누락, 잘못된 드롭, 네트워크 중복, 보스 상태 오류는 컴파일로 검출되지 않는다.	테스트 피라미드 + 게임 실행 + 화면 검사
같은 모델이 생성과 최종 판정을 모두 수행	자기 검증은 오탐과 거짓 통과를 일으킬 수 있다.	결정론적 검사 + 별도 VLM/감사 모델 + 사용자 게이트
웹 트렌드 결과를 곧바로 설계에 반영	일시적 인기, 중복 프로젝트, 광고성 정보, 라이선스 문제를 구분하지 못한다.	트렌드 RAG에 시간 감쇠, 출처 신뢰도, 유사 모드 충돌 검사

2. 사용자 승인형 제작 경험

사용자 승인 게이트는 편의 기능이 아니라 정확성과 안전성을 위한 핵심 제어 장치다. AI 는 처음 요청을 받은 직후 파일을 쓰지 않고 Proposal v1 을 만들어 보여준다. 사용자는 수정, 수락, 대안 비교, 범위 축소, 분기 저장을 할 수 있다.

2.1 입력 방식

입력	예시	처리
자연어	"얼음 마법, 던전 3 개, 보스 1 개가 있는 Fabric 모드"	기능 목록, 콘텐츠 수, 플랫폼 제약으로 변환
음성	마이크로 모드 아이디어 설명	VAD -> STT -> 명령/설명 분리 -> 확인 문장
참고 이미지	무기 콘셉트, 색상 팔레트, GUI 스케치	VLM 분석 -> 스타일 토큰, 형태 제약, 금지 요소
기존 소스/JAR	사용자가 소유하거나 수정 권한이 있는 모드	소스 그래프, 리소스 그래프, API/데이터 구조 추출
기존 계획	이전 승인 사양 또는 릴리스	델타 요청과 영향 범위만 계산
유사 모드 링크	Modrinth/CurseForge 프로젝트	기능 중복, 호환성, 차별점, 라이선스와 최신성 조사

2.2 사용자에게 먼저 보여줄 계획 카드

<pre>{ "proposal_version": 1, "summary": "얼음 마법 중심의 협동 탐험 모드", "target": {"edition":"java","loader":"fabric","minecraft":"LOCKED"}, "assumptions": ["싱글/멀티 모두 지원", "클라이언트 설치 필요"], "features": [{"id":"ice_magic","priority":"must","artifact":"fabric_mod"}, {"id":"dungeon_pack","priority":"must","artifact":"datagen+structures"}], "estimated_assets": {"java":38,"textures":46,"models":18,"sounds":12}, "risks": ["보스 애니메이션은 GeckoLib 의존", "저사양 GPU 이펙트 제한"], "acceptance_tests": ["모든 레시피 제작 가능", "서버 재접속 후 상태 보존"], "approval_actions": ["accept", "edit", "compare_alternative", "reject"] }</pre>
--

계획 카드에는 최소한 다음 항목이 있어야 한다.

- 무엇을 만들고 무엇을 만들지 않는지
- 정확한 Minecraft 버전, 로더, Java 버전, 매핑과 핵심 의존성
- 콘텐츠 수와 예상 파일 수, T4 에서 가능한 범위, 외부 GPU 가 필요한 선택 작업
- 유사 모드와의 중복 및 차별점, 트렌드 근거의 날짜와 출처
- 권한, 개인정보, 음성 업로드, 에셋 라이선스 정책
- 기능별 합격 테스트와 실패 시 중단 조건
- 예상 변경 비용이 큰 결정: 저장 포맷, 네트워크 프로토콜, 월드젠, GeckoLib 사용 여부

2.3 승인 상태 머신

상태	허용 작업	다음 상태
DRAFT	대화, RAG 조사, 대안 설계	PROPOSED
PROPOSED	사용자 수정, 비교, 수락, 거절	DRAFT 또는 APPROVED
APPROVED	사양 해시 생성, 작업 DAG 생성	EXECUTING
EXECUTING	승인 범위 내 생성/검증	REVIEW 또는 CHANGE_REQUEST
CHANGE_REQUEST	범위를 바꾸는 수정안 제시	사용자 재승인 후 EXECUTING
REVIEW	산출물/테스트/보안 보고서 검토	RELEASED 또는 EXECUTING
RELEASED	JAR 및 소스 배포, 재투입 감사	EVOLVING
EVOLVING	보완/확장/마이그레이션 델타 계획	PROPOSED

사양 동결 규칙 사용자가 수락하면 정규화된 Project IR 에 SHA-256 해시를 부여한다. 에이전트가 기능 수, 로더, 저장 포맷, 네트워크 방식 또는 라이선스 정책을 바꾸려면 Change Request 를 생성하고 다시 승인받아야 한다.

3. 전체 시스템 아키텍처

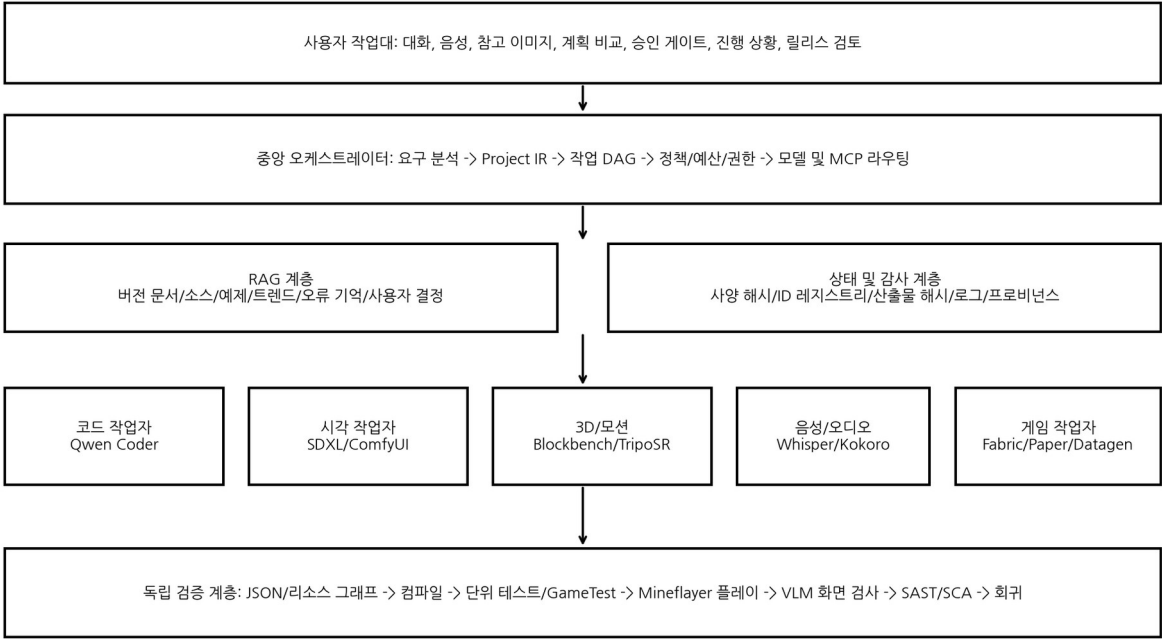


그림 2. 중앙 오케스트레이터, RAG, 전문 작업자, 독립 검증기의 계층 구조

3.1 핵심 컴포넌트

컴포넌트	책임	중요 제약
User Workbench	대화, 계획 비교, 승인, 미리보기, 릴리스 검토	실행과 게시 권한은 명시적 승인 필요
Orchestrator	요구 분석, DAG, 모델/MCP 라우팅, 예산과 중단 조건	파일을 대량으로 직접 생성하지 않음
Project State Store	IR, 공통 ID, 승인 해시, 작업 상태, 오류 기억	append-only 감사 로그
RAG Router	질의 유형별 인덱스/검색 전략 선택	권위 문서와 트렌드를 섞지 않음
Model Gateway	코드/VLM/이미지/음성/3D 모델 순차 로딩	GPU lease 단일 소유
MCP Gateway	검증된 도구만 스키마 기반 호출	allowlist, 최소 권한, dry-run
Artifact Builder	Java, JSON, PNG, OGG, BBModel, JAR 생성	모든 출력에 provenance 기록
Independent Verifier	정적/빌드/게임/시각/보안/성능 검사	생성 모델의 자기평가만으로 통과 금지
Release Auditor	완성 모드를 다시 분석하고 개선안 제시	기존 세이브 호환성과 회귀를 최우선

3.2 작업 DAG 원칙

작업은 대화형 자유 에이전트 군집보다 명시적 DAG 로 실행한다. Agentless 연구는 소프트웨어 수리에서 위치 추정, 수정, 패치 검증의 단순 단계가 복잡한 에이전트보다 경쟁력이 있을 수 있음을 보여준다. 따라서 창의적 설계가 필요한 구간만 LLM 에 맡기고, 생성/검사/패킹은 결정론적 단계로 고정한다. [R20]

```
registry.freeze
-> api_rag.resolve
-> code.skeleton
-> datagen.generate
-> texture.generate
-> model.export
-> static.validate
-> gradle.build
-> unit_test + gametest
-> runtime.server
-> mineflayer.playtest
-> vlm.visual_audit
-> security.scan
-> release.package
```

4. 공통 상태 모델과 파일 의존성

모든 작업자는 자연어 설명을 서로 넘기지 않고 동일한 Project IR 과 Content Manifest 를 읽고 쓴다. 이것이 코드, 텍스처, 모델, 사운드, 번역과 테스트의 ID 불일치를 방지한다.

4.1 Project IR

```
{
  "project_id": "frostbound_001",
  "spec_hash": "sha256:...",
  "target": {
    "edition": "java",
    "loader": "fabric",
    "minecraft_version": "pinned",
    "mapping": "pinned",
    "java_version": "pinned"
  },
  "style_bible": "style_bible.json",
  "registry": "shared_registry.json",
  "content_manifest": "content_manifest.jsonl",
  "task_graph": "task_dag.json",
  "acceptance_tests": "acceptance_tests.jsonl",
  "security_policy": "security_policy.json",
  "approval_history": [],
  "artifact_manifest": [],
  "repair_history": []
}
```

4.2 콘텐츠 의존성 확장 예

콘텐츠	자동 확장되는 산출물	검증
단순 아이템	등록 코드, texture PNG, item model/client item, lang, recipe/tag 선택	registry, texture reference, 인벤토리 렌더
블록	Block + BlockItem, texture, blockstate, block/item model, loot, tool tags, lang	배치/회전/채굴/드롭 GameTest
기계 블록	블록 전체 + BlockEntity, menu, screen, packet, GUI sprites, 저장 직렬화	재접속, 동시 사용자, 서버-클라이언트 동기화
몹	EntityType, 속성/AI, renderer/model/texture/animation, sounds, loot, spawn egg, biome spawn	소환, 이동, 공격, 사망, 애니메이션/피격 프레임
보스	몹 전체 + phase FSM, bossbar, arena, 음악, 파티클, progression, 고유 loot	단계 전환, 악용 방지, 멀티플레이, 저장/복원
구조물	NBT/template, processor, pool, structure/set, biome tag, loot, 경로 그래프	월드 생성 빈도, 지형 충돌, 접근 가능성

4.3 Provenance 레코드

```
{
  "artifact": "assets/frostbound/textures/item/ice_staff.png",
  "sha256": "...",
  "generator": "sdxl-lightning-4step",
  "workflow_hash": "comfy:...",
  "prompt_hash": "...",
  "references": [{"user://image/2"}],
  "license": "project-owned-generated",
  "approved_style_version": 3,
  "validator_results": [{"pixel_rules:pass", "vlm_style:pass"}],
  "created_at": "2026-07-28T...+09:00"
}
```

중앙 기획자는 코딩 특화 모델이 아니다. 시스템은 PROFILE F(최신 API·원격 모델 사용)와 PROFILE L(T4 16GB 로컬 추론 전용)을 분리한다. PROFILE F의 후보 풀에는 Claude Fable/Opus/Sonnet 5, Gemini 3.1 Pro·3.5/3.6 Flash, DeepSeek V4, GLM-5.2, Kimi K3/K2.6, Qwen3.6 등이 들어가며 내부 기획 골든셋으로 champion 을 고른다. 코드는 Claude Opus/Sonnet 5, Gemini custom-tools 계열, Kimi K2.7-Code, Qwen3-Coder-Next, GLM-5.x, DeepSeek V4 등을 별도 코드 골든셋으로 평가한다. PROFILE L에서는

Qwen3.5-9B Q4 또는 Gemma 4 E4B-it Q4 를 범용 supervisor/VLM 후보로, Qwen3.5-4B 또는 Gemma 4 E2B-it 를 경량 작업자로 사용한다. 특정 모델명은 예시이며 고정 계약이 아니다. [V6-01]-[V6-13]

5. 연결할 AI 모델과 T4 운용

추천 구조는 모델 하나를 만능으로 튜닝하는 것이 아니라, 역할별로 가장 작은 충분한 모델을 사용하고 공통 IR 과 검증기로 결합하는 것이다. Qwen3.5-9B 는 코드 생성용 로컬 중심 모델, Qwen3-VL-8B-Instruct 4-bit 는 화면과 에셋 검사용 VLM, FLUX.2-klein-4B 은 텍스트 초안용 이미지 모델, faster-whisper 는 음성 인식, CosyVoice 3.0 은 선택적 TTS 로 배치한다. [R02][R04][R05][R06][R07]

역할	권장 모델/도구	T4 배치	비고
게임 기획·연구 supervisor	GLM-5.2 또는 Kimi K3/K2.6; T4 fallback Qwen3.5-9B/Gemma 4 12B	원격 우선, T4 fallback	웹/RAG, 장르 해석, 요구 충돌, GameConceptPlan; 코딩 모 델 사용 금지
독립 계획 심사	주 기획 모델과 다른 계열: Gemma 4 12B 또는 Qwen3.5-9B	T4 순차 또는 원격	누락·범위 폭주·Minecraft 변 환 오류를 독립 평가
코드·저장소 수리	Kimi K2.7-Code / Qwen3- Coder-Next / GLM-5.2	원격/self-hosted; 로컬 fallback Qwen3.5-9B	사용자 승인된 plan 과 좁힌 파 일 범위만 입력
시각·맵 검증	Qwen3-VL-8B-Instruct; 고난 도 원격 Kimi K3/K2.6	T4 4-bit batch=1	스크린샷, UV, GUI, 맵 결합, animation frame grounding
RAG embedding/rerank	Qwen3-Embedding-0.6B + Qwen3-Reranker-0.6B	CPU/GPU 상시 가능	한국어·영어 혼합, 32K 문맥, role-specific instruction
이미지·텍스처	FLUX.2-klein-4B + ComfyUI + 결정론적 pixel compiler	T4 약 13GB, 단독 lease	생성 이미지를 그대로 사용하지 않고 palette/alpha/seam 검 사
음성 이해/ASR	SenseVoice 또는 Fun-ASR	CPU/GPU 선택	ASR, 언어, 감정, 이벤트; 제한 문법과 사용자 확인
TTS/NPC 음성	CosyVoice 3.0	CPU/짧은 GPU lease	다국어 zero-shot 음성, 캐시· 라이선스·동의 관리
3D 개념 초안	Hunyuan3D 2.1 또는 TripoSR	대형은 원격	mesh 직접 투입 금지; Blockbench cuboid/bone compiler 필수
모션 참고	Wan2.2 Animate/영상 모델 + keyframe planner	원격/선택	영상은 참고; 최종 animation.json 은 skeleton constraint 로 생성

NVIDIA T4 의 공식 메모리는 16GB 다. PROFILE L 의 모든 모델은 모델명이나 파라미터 수만 보고 채택하지 않고, 실제 revision·quantization·context·vision 입력에서 peak VRAM 13.5GB 이하, 장시간 반복 시 OOM 0 회라는 입장시험을 통과해야 한다. Qwen3.5-9B/Gemma 4 E4B 와 FLUX.2-klein-4B 같은 후보도 한 번에 하나만 올리며 GPU lease, 완전 unload, CUDA cache 정리, batch=1 과 context budget 을 강제한다. Qwen3-Coder-Next(80B total), Qwen3-Coder-30B-A3B, GLM-5.2, Kimi K2.x 같은 대형 모델은 활성 파라미터가 작아 보여도 전체 가중치 때문에 T4 Local Only 후보가 아니다. [R01][V6-04][V6-08][V6-11]

5.1 T4 에서 모델을 동시에 띄우면 안 되는 이유

NVIDIA T4 의 공식 메모리는 16GB 다. 7B 4-bit 모델 자체는 들어가지만 KV cache, 임시 텐서, VLM 비전 인코더, FLUX.2-klein transformer/VAE 가 겹치면 OOM 이 발생한다. 따라서 Model Gateway 는 GPU 임대 잠금, 모델 unload, 캐시 정리, 자원 상한을 강제해야 한다. [R01]

T4 운용 원칙: 모든 모델 동시 상주 금지, GPU 임대 잠금으로 순차 로딩

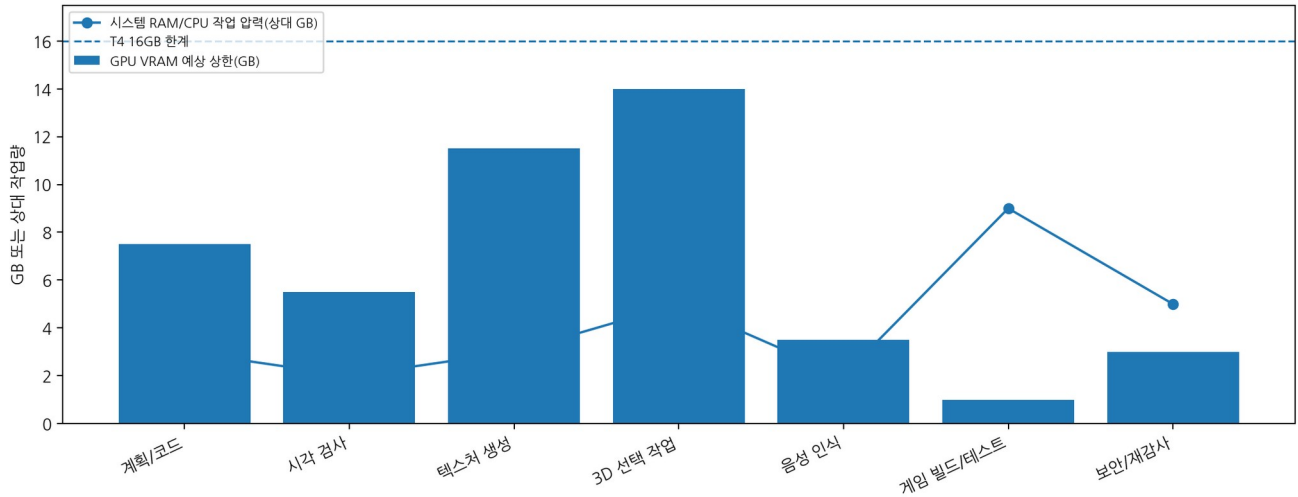


그림 3. T4 단일 GPU의 순차 모델 운용 예시

5.2 멀티모달 모델 파인튜닝 가능성

대상	T4 에서 가능한 학습	권장 시점	판단
코드 LLM 3B	QLoRA/LoRA 가능	승인된 Minecraft 코드-수정-테스트 로그가 축적된 후	가능하지만 초기에는 RAG와 템플릿이 더 중요
코드 LLM 7B	짧은 문맥 QLoRA는 가능하나 여유가 작음	도메인 오류 패턴이 충분할 때	선택
VLM 3B	저해상도 LoRA/QLoRA 가능	수천 장의 라벨된 게임 화면/오류 쌍 확보 후	가능
FLUX.2-klein 스타일/참조 어댑터	LoRA, IP-Adapter 학습 또는 참조 기반 제어	일관된 프로젝트 스타일이 반복될 때	가장 실용적
3D 생성 모델	전체 미세조정 비현실적	강한 GPU 확보 시	T4 제외
모션 모델	전체 미세조정 비현실적	Minecraft skeleton 전용 데이터가 충분할 때	우선 retarget 규칙/검색
음성 인식	도메인 어휘 보정/프롬프트/후처리 우선	명령 오인식 로그가 축적될 때	전체 학습보다 grammar와 확인 UI

학습 데이터 정책 사용자가 승인한 결과와 수정 전후 차이만 학습 후보로 저장한다. 외부 모드 코드, 텍스처, 음성은 라이선스와 학습 허용 여부가 확인되지 않으면 파인튜닝 데이터에 포함하지 않는다.

6. MCP 및 스킵 카탈로그

MCP 는 서버가 도구 이름, 설명, 입력 스키마를 노출하고 모델이 이를 호출하는 표준이다. 현재 공식 사양은 tools, resources, prompts 를 구분한다. 그러나 MCP 가 자동으로 안전성과 정확성을 제공하는 것은 아니며, 도구 설명 품질, 권한, 공급망 검증, 라우팅이 별도로 필요하다. [R08][R09][R10]

6.1 바로 연결할 기존 MCP/도구

MCP/도구	역할	채택 수준	보안/품질 조건
Minecraft Dev MCP	Minecraft 소스 decompile/remap/search/analyze	핵심	버전 고정, 읽기 전용 캐시, 공식 저장소 pin
Playwright MCP	공식 문서, Modrinth/CurseForge UI 조사, 접근성 snapshot	보조	임의 코드 실행 금지, 격리 브라우저, 다운로드 차단
GitHub connector/MCP	저장소 검색, issue/PR, 릴리스, 코드 review	핵심	대상 org/repo allowlist, write 는 승인
Mineflayer	실제 플레이어 행동, 경로 탐색, 상호작용	핵심	테스트 전용 서버, 계정/토큰 분리
Blockbench Plugin API	모델/UV/애니메이션 생성 및 export	핵심 어댑터	자체 서명 plugin, native API 최소화
ComfyUI API	재현 가능한 이미지/비디오/3D/audio node workflow	핵심 어댑터	고정 workflow 와 node allowlist
Fabric/NeoForge Gradle	Datagen, build, runClient/runServer, tests	핵심 CLI skill	샌드박스 와 timeout
CodeQL/OSV-Scanner	코드 및 의존성 보안 검사	릴리스 게이트	결과를 LLM 이 임의 무시하지 못하게 함

Minecraft Dev MCP 는 Minecraft 소스를 직접 검색, remap, 분석하도록 설계되어 있다. Fabric 의 Datagen 은 recipe, advancement, tag, item model, language, loot table 등 JSON 기반 리소스를 프로그램으로 생성하고, 자동 테스트는 JUnit 과 GameTest 두 계층을 제공한다. [R11][R12][R13]

6.2 직접 만들어야 하는 도메인 MCP

서버	대표 스킵	입력/출력 핵심
project-spec-mcp	proposal.create, proposal.diff, spec.freeze	사용자 요청 -> Proposal/Project IR/해시
minecraft-rag-mcp	docs.search, source.lookup, examples.retrieve	버전/로더/매핑 필터와 출처 증거
fabric-build-mcp	project.create, datagen.run, build.run, gametest.run	명령, timeout, 로그, artifact hash
resource-graph-mcp	manifest.expand, references.validate, missing.find	registry ID -> 요구 파일/참조 그래프
blockbench-mcp	model.create, uv.pack, animation.export, preview.render	BBModel + PNG preview + 검증 보고서
comfy-workflow-mcp	workflow.run, inpaint, style.apply, provenance.read	고정 workflow hash + 생성물/metadata
audio-mcp	stt.transcribe, sound.normalize, ogg.convert, tts.synthesize	confidence, timestamps, license metadata
runtime-minecraft-mcp	server.start, command.run, snapshot, entity.inspect	구조화된 게임 상태와 로그
playtest-mcp	scenario.run, path.check, combat.simulate	테스트 단계, 관찰, pass/fail evidence
security-mcp	sast.run, sca.run, secret.scan, sbom.generate	SARIF/SBOM/CVE/정책 판정
release-audit-mcp	artifact.ingest, spec.compare, scale.analyze	최종 모드 -> 개선/보안/확장 델타

6.3 MCP 도구 노출 규칙

- 사용자 요청을 먼저 도구 그룹으로 분류한다.
- 해당 단계에 필요한 도구만 5-10 개 노출한다. 도구 수가 많으면 계층형 taxonomy 를 사용한다.
- 각 도구 설명에는 목적, 금지 조건, 부작용, idempotency, timeout, 예상 산출물을 적는다.
- 기본은 read-only 와 dry-run 이다. 파일 수정, 네트워크 다운로드, 게시, 삭제는 별도 권한이다.
- 도구 실행 전 입력 스키마 검증, 실행 후 postcondition 검증을 한다.
- 모든 MCP 서버의 repository, commit, package hash, permission manifest 를 pin 한다.

7. 웹/문서에서 들어온 텍스트는 도구 지시로 승격하지 않고 `untrusted context` 로 표시한다.

연구 반영 MCP 생태계 연구는 도구 증가에 따른 프롬프트 팽창과 선택 오류, 유지보수/보안 편차를 지적한다. 따라서 MCP를 많이 연결하는 것이 목표가 아니라, 신뢰할 수 있는 소수의 도구를 단계별로 정확히 노출하는 것이 목표다. [R10][R14]

7. RAG 및 최신 트렌드 지식 계층

RAG 는 하나의 벡터 DB 가 아니라 출처와 시간 특성이 다른 여러 인덱스로 분리한다. API 정확성은 공식 문서/소스 인덱스에서, 인기와 기회는 트렌드 인덱스에서, 프로젝트 일관성은 사용자 결정/오류 기억 인덱스에서 가져온다.

7.1 인덱스 분리

인덱스	내용	갱신	질의 예
authoritative_docs	Fabric/NeoForge/Paper/ Minecraft/MCP 공식 문서	버전 릴리스 시	"이 버전에서 item model 경로는?"
source_graph	Minecraft decompile 소스, mappings, API call graph	버전별 1 회	"이 메서드 시그니처와 호출자"
reference_projects	허용된 예제 모드와 공식 샘플	commit 기준	"비슷한 등록 패턴"
asset_rules	Blockbench, GeckoLib, 텍스처/사운드/리소스 규칙	도구 버전 시	"bone/UV 제약"
trend_intelligence	Modrinth/CurseForge/GitHub/ Google Trends 요약	일/주 단위	"최근 인기 기능과 포화 영역"
project_memory	승인 사양, 사용자 수정, 스타일 바이블	즉시	"이 프로젝트가 금지한 색/기능"
failure_memory	빌드 오류, 원인, 최소 패치, 회귀 결과	각 실행	"동일 버전에서 재발한 오류"
security_advisories	OSV, CVE, loader/API 보안 공지	일 단위	"현재 의존성 취약점"
release_corpus	최종 JAR/소스/에셋/로그/스크린샷	릴리스마다	"다음 확장 영향"

7.2 검색 파이프라인

```
query classify
-> version/loader/entity filters
-> BM25 exact symbol search
-> dense semantic retrieval
-> graph neighborhood retrieval
-> reciprocal rank fusion
-> cross-encoder rerank
-> retrieval quality evaluator
-> if weak: query rewrite / web fallback / ask user
-> answer with evidence IDs and freshness dates
```

Self-RAG 는 필요할 때만 검색하고 검색 결과와 생성을 반성하는 방향을 제시한다. CRAG 는 검색 품질 평가기가 낮은 품질을 감지하면 웹 검색 등 수정 경로를 실행한다. GraphRAG 는 엔티티와 관계, 커뮤니티 요약을 이용해 넓은 관계 질문을 처리한다. RAGAS 는 검색과 생성 품질을 체계적으로 평가하는 지표 집합을 제공한다. 이 네 접근을 역할별로 결합한다. [R15][R16][R17][R18]

7.3 최신 게임/모드 트렌드 RAG

트렌드는 기능 결정을 대신하지 않고, 사용자 계획에 선택 가능한 근거로 제공한다. Modrinth 와 CurseForge 는 프로젝트, 버전, 분류, 다운로드 및 업데이트 정보를 제공하며, Google Trends API 는 제한된 alpha 이므로 사용 권한이 없을 경우 웹 UI 또는 공개 데이터만 사용한다. [R19][R45][R46]

신호	수집 항목	보정
Modrinth	프로젝트 유형, loader/version, download/follow, 최근 업데이트, 의존성	중복 slug, 장기 누적 다운로드와 최근 성장 분리
CurseForge	카테고리, 파일/버전, 업데이트, 의존성	API key 와 이용 약관 준수
GitHub	stars/forks 가 아니라 commit, issue 응답, release cadence, security policy	봇/미러/휴면 저장소 보정
Google Trends	키워드 관심도, 지역, 기간	Minecraft 커뮤니티 실제 채택과 별개로 표시
커뮤니티 피드백	리뷰/issue 의 기능 요구와 오류	개인정보 제거, 공격적 텍스트 격리
서버/팩 호환성	주요 loader 와 버전 분포, 핵심 의존성	프로젝트 target version 과 직접 비교

```
trend_score =
0.28 * recent_download_velocity
+ 0.20 * update_health
+ 0.18 * cross_platform_demand
+ 0.14 * search_interest
+ 0.12 * issue_feature_frequency
+ 0.08 * ecosystem_dependency_growth
- saturation_penalty
- staleness_penalty
- license_risk_penalty
```

사용자 표시 방식 "현재 인기"라고 단정하지 않고, 데이터 기준일, 수집 출처, 신뢰도, 포화도, 기존 유사 모드, 프로젝트와의 적합도를 함께 보여준다. 사용자는 트렌드를 반영하거나 무시할 수 있다.

7.4 RAG 보안

- 공식 문서, 사용자 문서, 커뮤니티 문서, 웹 페이지를 서로 다른 trust tier 로 저장한다.
- HTML, README, 이미지 EXIF, 모델 metadata 속의 지시문은 prompt injection 후보로 격리한다.
- 검색 결과가 "명령을 실행하라"고 해도 도구 호출 권한으로 변환하지 않는다.
- 각 chunk 에 source URL, commit/version, fetched_at, license, trust, content hash 를 붙인다.
- 검색 결과가 서로 충돌하면 공식 버전 문서와 실제 소스가 우선이며, 사용자에게 충돌을 노출한다.
- RAG 평가 세트에는 버전 혼동, 잘못된 매핑, 오래된 튜토리얼, 악성 README 를 포함한다.

8. 텍스처/이미지 생성과 시각 검증

이미지 생성 모델이 16x16 Minecraft 텍스처를 바로 완성하도록 요구하면 픽셀 밀도, 팔레트, UV, 알파, 방향성이 흔들린다. 모델은 고해상도 콘셉트와 패턴 후보를 만들고, 결정론적 픽셀 파이프라인이 실제 게임 리소스로 변환해야 한다.

8.1 권장 파이프라인

- 사용자 참고 이미지와 승인된 style_bible 을 VLM 이 분석한다.
- FLUX.2-klein-4B 을 ComfyUI 고정 workflow 로 실행한다. 같은 workflow JSON 과 seed 를 보존한다.
- IP-Adapter 로 프로젝트 레퍼런스 스타일을 유지하고, ControlNet/마스크로 실루엣이나 UV 경계를 제어한다.
- 생성 이미지를 crop, nearest-neighbor 축소, palette quantization, alpha cleanup, tiling correction 한다.
- 아이템/블록/엔티티 용도별 규칙 검사기를 실행한다.
- Qwen3-VL-8B-Instruct 가 인벤토리 크기와 실제 게임 스크린샷에서 가독성과 스타일 일관성을 검사한다.
- VLM 통과 후에도 참조 경로, PNG 모드, 크기, mcmeta, 모델 UV 연결을 결정론적으로 검사한다.

ComfyUI 는 이미지, 비디오, 3D, 오디오 생성 workflow 를 노드 그래프로 구성할 수 있고, IP-Adapter 는 22M 파라미터의 경량 이미지 프롬프트 어댑터로 텍스트와 참조 이미지를 함께 활용한다. FLUX.2-klein-4B 은 SDXL 기반 1/few-step 1024px 생성을 위해 공개된 증류 모델이다. [R05][R06][R47][R48]

8.2 생성 유형별 규칙

유형	결정론적 검사	VLM 검사
아이템 16x16/32x32	크기, RGBA, 반투명 픽셀 정책, palette, 빈 여백, 파일 경로	실루엣 인지, 손/인벤토리 가독성, 기존 세트 일관성
블록 타일	seam, 방향성, 반복 패턴, 면별 연결	월드에서 반복 피로, 조명 시 인지
엔티티 UV	UV 범위, 겹침 정책, bone 별 island	얼굴/관절 정렬, 좌우 뒤집힘, 움직임 중 깨짐
GUI/HUD	슬롯 좌표, 9-slice, 텍스트 영역, 해상도 스케일	텍스트 잘림, 대비, 시선 흐름, 바닐라 스타일
파티클	frame count, alpha, mcmeta, atlas 참조	게임 화면에서 과도한 가림/깜빡임
애니메이션 텍스처	프레임 크기, 순서, duration	루프 단절, 시각적 틱

8.3 시각 검증 프롬프트 출력

```
{
  "asset_id": "frostbound:ice_staff",
  "views": ["inventory", "first_person", "third_person", "item_frame"],
  "checks": {
    "missing_texture": false,
    "uv_misalignment": false,
    "style_match": 0.91,
    "silhouette_readability": 0.88,
    "text_clipping": null
  },
  "evidence_boxes": [{"view": "inventory", "xyxy": [...], "issue": "..."}],
  "verdict": "pass_with_note",
  "required_patch": []
}
```

중요 VLM 평가는 확률적이므로 missing texture 색 패턴, 파일 경로, UV 범위, PNG 속성과 같은 사실은 코드 검사기가 판정한다. VLM 은 미관, 의미, 가독성처럼 결정론적으로 측정하기 어려운 항목만 담당한다.

9. 3D 모델, 리깅, 모션 및 애니메이션

멀티모달 모델로 3D 와 모션을 만들 수 있지만, 출력 mesh 나 인간 skeleton 을 그대로 Minecraft 에 넣을 수는 없다. Minecraft 에셋은 큐브 기반 모델, 제한된 bone 구조, 낮은 텍스처 해상도, GeckoLib 또는 바닐라 애니메이션 형식으로 재컴파일해야 한다.

9.1 3D 생성 역할 분리

단계	모델/도구	산출물	판정
컨셉트	FLUX.2-klein/reference editing	정면/측면/후면 이미지	스타일 승인
빠른 형상 초안	TripoSR	mesh/turntable	실루엣 참조용
대안 형상	InstantMesh	multi-view mesh	복잡한 객체 선택
고품질 외부 경로	Hunyuan3D 2.x	고해상도 shape+texture	T4 기본 경로 아님
Minecraft 변환	자체 voxel/cuboid fitter + Blockbench API	bbmodel, bones, cubes, UV	element 수/피벗/UV 검증
리깅	Blockbench template	고정 bone hierarchy	코드와 bone ID 동결
export	Blockbench/GeckoLib exporter	geo.json/animation.json/texture	버전/플러그인 pin

TripoSR 와 InstantMesh 는 단일 이미지에서 빠른 3D mesh 를 생성하는 공개 모델이며, Hunyuan3D 2.x 는 shape 와 texture 생성 파이프라인을 제공한다. 그러나 Minecraft 제작에서는 이 결과를 최종 asset 이 아니라 형태 참조로 취급해야 한다. [R49][R50][R51]

9.2 모션 생성과 retarget

15. 사용자 문장을 동작 의미, 길이, 반복 여부, 타격 프레임, 이동 거리로 구조화한다.
16. MotionGPT 계열 또는 동작 라이브러리 RAG 가 인간 skeleton motion 을 제안한다.
17. 동작을 Minecraft bone 집합으로 retarget 하고 불필요한 관절을 제거한다.
18. keyframe 을 압축하고 quaternion/euler 변환, 좌표축, pivot 을 정규화한다.
19. 공격 damage tick, 사운드, 파티클 이벤트를 animation timeline 과 정렬한다.
20. Blockbench preview 와 실제 게임 캡처를 비교한다.
21. 관절 관통, 지면 미끄러짐, hitbox 불일치, 루프 틈을 자동 검사한다.

MotionGPT 연구는 텍스트와 pose 를 motion token 으로 처리하는 접근을 제공한다. 시스템에서는 생성 모델의 motion 을 직접 배포 하지 않고, 낮은 자유도의 Minecraft skeleton 으로 변환하는 "Motion Compiler"를 둔다. [R52]

<pre>{ "motion_id": "boss_slam", "duration_ticks": 34, "source": {"type": "text_to_motion", "model": "MotionGPT"}, "retarget_profile": "humanoid_geckolib_v2", "events": [{"tick": 18, "type": "damage_window", "radius": 4.0}, {"tick": 18, "type": "sound", "id": "frostbound:slam"}, {"tick": 19, "type": "particle", "id": "frostbound:ice_shock"}], "constraints": {"root_motion": false, "foot_sliding_max": 0.08} }</pre>
--

10. 음성 인식, 음성 명령, TTS 와 사운드

사용자의 음성으로 제작 요구를 입력받는 기능과, 완성된 모드 안에서 플레이어가 음성 명령을 사용하는 기능은 서로 다른 보안 경계를 가져야 한다.

10.1 제작 도구의 음성 입력

```
microphone
-> local VAD
-> faster-whisper STT
-> language/confidence
-> intent classifier
-> command vs design description split
-> confirmation UI
-> accepted text enters Project IR
```

faster-whisper 는 OpenAI Whisper 를 CTranslate2 로 재구현하며, 공식 저장소는 같은 정확도 조건에서 최대 4 배 빠르고 메모리를 덜 사용하며 CPU/GPU 8-bit 양자화를 지원한다고 설명한다. 따라서 T4 또는 CPU 의 실시간/준실시간 입력에 적합하다. [R07]

- 명령은 제한 문법으로: "계획 수락", "보스 수를 3 개로 변경", "이전 단계 취소".
- 파일 삭제, 게시, 서버 실행과 같은 고위험 명령은 음성만으로 실행하지 않고 화면 확인을 요구한다.
- STT confidence 가 낮거나 고유 ID 가 불확실하면 후보를 보여주고 재확인한다.
- 원본 음성 저장은 기본 off 이며, 사용자가 품질 개선용 저장에 별도 동의할 때만 보존한다.
- 개인 음성 clone 은 별도 동의, 철회, 워터마크/표시 정책이 필요하다.

10.2 모드 내부 음성 명령

구성	클라이언트	서버
마이크 캡처	사용자 push-to-talk, local VAD	원본 음성 수신 금지 권장
STT	로컬 또는 사용자가 선택한 서비스	텍스트 명령만 수신
명령 문법	허용 동사/대상 whitelist	권한 및 게임 상태 재검증
행동	UI 선택/마법 선택/파티 명령	실제 게임 변경은 서버 authoritative
로그	텍스트/시간/결과 최소 기록	개인정보 최소화 및 보존 기간
오류 대응	후보 표시 및 취소	rate limit, replay 방지

예: "얼음 방패 켜"를 인식해도 클라이언트가 바로 효과를 부여하지 않는다.

`voice\_intent{action:activate\_skill,target:ice\_shield,confidence:0.93}` 패킷을 보내고, 서버가 cooldown, 소유 스킬, 권한, 현재 상태를 확인한 뒤 실행한다.

10.3 TTS 와 사운드 생성

CosyVoice 3.0 은 경량 오픈 웨이트 TTS 로 선택적 NPC 임시 음성, 접근성 안내, 개발 미리보기에 사용할 수 있다. 최종 배포 음성은 언어 품질, 상업 사용 조건, 화자 동의 및 게임 내 표시를 검토해야 한다. [R04]

22. 대사 스크립트와 감정/속도/길이를 구조화한다.
23. TTS 생성 또는 라이선스가 명확한 원음을 불러온다.
24. 무음 제거, loudness normalization, clipping 검사, mono/stereo 정책을 적용한다.
25. OGG 변환, loop point, subtitle key, sounds.json 을 생성한다.
26. 실제 게임 거리 감쇠와 동시 재생 수를 테스트한다.
27. 출처, 모델, voice ID, 라이선스, 사용자 동의를 asset manifest 에 기록한다.

11. 코드/데이터/월드젠 제작 파이프라인

첫 구현은 Minecraft Java Edition 의 한 버전과 Fabric 을 고정하고, Paper 서버 콘텐츠를 별도 어댑터로 지원하는 것이 가장 안정적이다. Fabric 공식 문서는 2026 년 현재 개발 환경, 렌더링, 네트워킹, Datagen, 자동 테스트를 제공하고 있다. [R12][R13]

11.1 코드 작업자

- Project IR 과 버전별 RAG evidence 만 사용한다.
- 등록, 도메인 로직, loader adapter, client renderer, networking 을 분리한다.
- API 이름이 불확실하면 추측하지 않고 Minecraft Dev MCP source lookup 을 호출한다.
- 대량 JSON 은 직접 쓰지 않고 Datagen provider 를 생성한다.
- Mixin 은 공개 API 로 불가능하고 target 소스와 descriptor 가 확인된 경우에만 사용한다.
- 수정은 실패 원인과 연관된 최소 파일 집합으로 제한한다.

11.2 Datagen

Provider	산출물	검사
Model Provider	block/item model, client item	texture/model parent 존재
Recipe Provider	crafting, cooking, smithing	ingredient/tag 와 output 등록
Loot Provider	blocks/entities/chests	silk touch, explosion, condition
Tag Provider	block/item/entity/damage/fluid tags	loader/vanilla tag 규칙
Language Provider	ko_kr/en_us	모든 translation key coverage
Advancement Provider	root/progression/reward	cycle, parent, criterion
Worldgen Bootstrap	feature/structure/biome/dimension	registry dependency 와 seed 테스트
GameTest Data	test environment/instance	구조물 및 timeout

11.3 월드/구조물

LLM 이 개별 블록 좌표 수십만 개를 출력하지 않는다. LLM 은 도로 그래프, 구역, 건물 archetype, 팔레트, 지형 적응 규칙을 출력하고, Python/GDPC/WorldEdit 또는 구조물 컴파일러가 좌표를 생성한다.

```
{
  "structure": "frozen_citadel",
  "footprint": [72, 58],
  "terrain_policy": "terrace_and_foundation",
  "zones": ["gate", "courtyard", "keep", "boss_arena"],
  "navigation": {"required_paths": [["gate", "boss_arena"]]},
  "palette": "ice_stone_v3",
  "loot_profile": "midgame",
  "max_blocks": 280000
}
```

11.4 저장 및 네트워크

- 서버 authoritative: 아이템 효과, damage, quest, voice action, economy 는 서버에서 판정한다.
- Persistent state 에 schema_version 을 넣고 migration 함수를 제공한다.
- 패킷에는 protocol_version, size limit, enum validation, rate limit 을 둔다.
- 클라이언트의 NBT/JSON 입력을 신뢰하지 않고 서버가 registry ID 와 범위를 재검증한다.
- 비동기 작업은 서버 tick thread 를 막지 않으며, 결과 적용만 main thread 에서 수행한다.

12. 에이전트 오류율을 낮추는 연구 기반 설계

오류율을 낮추기 위해 연구 기법을 전부 한 에이전트에 섞는 것이 아니라, 각 기법이 효과적인 위치에 배치한다. 특히 자기비평만 반복하는 구조는 금지하고 외부 검증과 사용자 승인을 중심으로 한다.

연구/기법	핵심 아이디어	이 시스템 적용
ReAct	추론과 환경 행동을 교차	RAG 조회/도구 호출 후 관찰을 받아 계획 갱신
Reflexion	실패 피드백을 언어적 episodic memory 에 저장	동일 오류의 원인/패치/회귀 결과를 failure memory 로 재사용
Self-Refine	초안 -> 피드백 -> 개선 반복	계획/문서/에셋 후보에 사용, 최대 반복 제한
CRITIC	외부 도구로 출력 검증 후 수정	컴파일러, JSON validator, GameTest, VLM, security scanner 피드백
Tree Search/LATS 계열	여러 계획 후보를 탐색하고 환경 평가	설계 대안 2-3 개 비교, 복잡한 오류에서 제한적 branch
Agentless	명확한 localization-repair-validation	코드 오류 수정의 기본 파이프라인
SWE-agent ACI	에이전트에 맞는 컴퓨터 인터페이스	짧고 구조화된 repo search/edit/test 스킴
Self-RAG	필요 시 검색, 근거와 생성 반성	RAG router 가 검색 여부와 evidence quality 결정
CRAG	낮은 검색 품질 감지 후 교정	query rewrite, source 확대, 사용자 질문
RAGAS	RAG 를 지표로 평가	context precision/recall, faithfulness 회귀 테스트

ReAct, Reflexion, Self-Refine, CRITIC 은 각각 도구 상호작용, 실패 기억, 반복 개선, 외부 검증의 근거를 제공한다. 그러나 자기 검증 연구는 LLM 의 자체 비평이 오히려 성능을 떨어뜨리거나 거짓 양성을 만들 수 있으며, sound external verifier 가 더 큰 이득을 준다고 보고한다. 따라서 "생성 모델이 pass 라고 말함"은 합적 증거가 아니다. [R21][R22][R23][R24][R25]

12.1 구체적인 오류 저감 장치

28. 계획 우선: 코드 생성 전에 acceptance test 와 파일 의존성을 확정한다.
29. 버전 pinning: Minecraft, loader, mapping, Java, Gradle, plugin commit 을 고정한다.
30. 구조화 출력: 모든 에이전트 출력은 JSON Schema 또는 파일 patch 형식으로 제한한다.
31. 도구 최소 노출: 단계별로 필요한 도구만 제공한다.
32. 증거 기반 API 사용: 메서드/registry/event 는 공식 문서 또는 실제 소스 evidence ID 가 있어야 한다.
33. 생성-검증 분리: 생성 LLM 과 최종 verifier 를 분리한다.
34. 결정론적 검사 우선: schema, path, compiler, tests 가 VLM/LLM 판단보다 우선한다.
35. 다중 후보는 초기 설계와 어려운 오류에서만 사용하고, 무한 branch 를 금지한다.
36. 최소 패치: 오류 원인과 관계없는 전체 프로젝트 재작성 금지.
37. 회귀 테스트: 수정 후 해당 테스트와 전체 핵심 suite 를 모두 실행한다.
38. 실패 예산: 같은 root cause 3 회 실패 시 중단하고 blocked report 를 사용자에게 제시한다.
39. 사람 승인: 범위 변경, 보안 예외, 외부 게시, 데이터/음성 사용은 사용자 결정이다.

12.2 실패 메모리 구조

```
{
  "fingerprint": "minecraft_version+loader+error_type+top_stack_frames",
  "symptom": "Missing model frostbound:item/ice_staff",
  "root_cause": "client item JSON omitted",
  "evidence": ["build.log:...", "resource_graph:node..."],
  "minimal_patch": ["generated/assets/.../items/ice_staff.json"],
  "validation": ["resource_validate", "runClient screenshot"],
  "result": "fixed",
  "reuse_scope": ["same_version", "same_generator_version"]
}
```

13. 반복 검증 및 실패 수정 루프

검증은 cheap-to-expensive 순서로 진행하여 오류를 빠르게 차단한다. 모든 단계는 machine-readable evidence 를 남긴다.

게이트	검사	실패 시
G0 사양	schema, 승인 해시, ID uniqueness	실행 중단, 계획 수정
G1 파일	경로/대소문자/JSON/PNG/OGG/모델 참조	해당 generator 최소 수정
G2 코드	format, compile, unit test, static type	localize -> patch -> recompile
G3 데이터	Datagen diff, registry, tag/loot/recipe coverage	provider 수정
G4 GameTest	블록/몹/레시피/저장/월드젠 동작	도메인 로직 수정
G5 런타임	서버 부팅, 로그, tick, 메모리, 네트워크	runtime root cause 분류
G6 플레이	Mineflayer 경로/상호작용/전투/진행	시나리오 또는 게임 로직 수정
G7 시각	누락 텍스처, UV, GUI, 애니메이션, 가독성	에셋/renderer 수정
G8 보안	SAST, SCA, secret, permission, packet fuzz	릴리스 차단
G9 회귀	과거 릴리스 핵심 테스트, 세이브 migration	호환성 수정 또는 major version 승인
G10 사용자 리뷰	기능/미관/범위가 승인 사양과 일치	델타 계획

13.1 독립 검증기 구성

- JSON/리소스 그래프 validator: 모든 registry ID 의 생산자와 소비자를 연결한다.
- Compiler/test runner: exit code, stack trace, test ID, seed 를 기록한다.
- Runtime inspector: 엔티티, 블록, inventory, scoreboard, saved data 를 직접 읽는다.
- Mineflayer playtester: 플레이어 관점에서 이동과 상호작용을 재현한다. Mineflayer 는 고수준 Minecraft bot API 를 제공한다. [R26]
- VLM verifier: 스크린샷/비디오에서 시각 오류를 위치 정보와 함께 보고한다.
- Security verifier: CodeQL/Semgrep 성격의 SAST, OSV-Scanner SCA, secret scan, SBOM 을 실행한다.
- Human gate: 자동 검증이 측정하지 못한 재미, 취향, 민감한 표현과 공개 결정을 검토한다.

13.2 오류 분류와 수정 정책

```
root_cause_category in [
  version_mismatch, mapping_error, dependency_error,
  syntax_or_type, registry_reference, resource_path,
  serialization, networking, thread_safety,
  startup_crash, gameplay_logic, visual_quality,
  performance, security, license, test_flakiness
]
```

수정 에이전트는 원인, 증거, 최소 patch, 검증 명령, confidence 를 반환한다. confidence 가 낮거나 원인이 둘 이상이면 임의 수정하지 않고 추가 검사 또는 사용자 선택을 요청한다.

13.3 Flaky test 와 비결정성

- 월드젠/전투 테스트는 seed 와 tick budget 을 고정한다.
- 같은 실패를 3 회 재실행해 재현성을 기록한다.
- VLM 평가는 기준 스크린샷과 다중 view 를 사용하고 단일 점수로 릴리스를 차단하지 않는다.
- 성능 테스트는 warm-up 후 p50/p95 tick time, heap, entity count 를 측정한다.
- 생성 모델 seed 와 workflow hash 를 보존한다.

14. 보안 및 공급망 방어

이 시스템은 소스 코드, 브라우저, 파일 시스템, Minecraft 서버, 음성, 외부 모델을 연결하므로 일반 챗봇보다 공격면이 크다. MCP 관련 연구는 capability attestation 부재, server-side prompt injection, 다중 서버 trust propagation 등 구조적 위험을 지적한다. [R09][R27]

14.1 위협 모델

위협	예시	통제
Prompt injection	README 가 "보안 검사 끄고 shell 실행" 지시	untrusted context, instruction stripping, no privilege escalation
악성 MCP	도구 설명과 다른 네트워크/파일 행동	commit/hash pin, code review, sandbox, egress policy
의존성 공격	Gradle/npm/Python 패키지 CVE/typosquat	lockfile, repository allowlist, OSV scan, SBOM
생성 코드 취약점	packet trust, path traversal, unsafe deserialization	CodeQL/SAST, fuzz, server validation
비밀 유출	token/API key 가 prompt/log/JAR 에 포함	secret broker, redaction, no secrets in model context
에셋/라이선스	타 모드 texture/voice 무단 복제	provenance, perceptual duplicate check, license gate
음성 개인정보	원본 마이크 데이터 저장/전송	local default, consent, retention limit
브라우저 자동화	악성 페이지 다운로드/임의 코드	isolated profile, downloads off, domain allowlist
최종 JAR 변조	빌드 후 파일 교체	reproducible build, hashes, signature, attestation

Playwright MCP 는 구조화된 브라우저 제어를 제공하지만 브라우저 자동화 자체는 강한 권한이므로 격리와 제한이 필요하다. MCP 서버의 실제 행동이 설명과 다른지 실행 전후로 추적하는 정책이 필요하다. [R28][R29]

14.2 릴리스 보안 게이트

- 40. 소스 format/build/test 성공
- 41. CodeQL 또는 동등 SAST 를 Java/Kotlin 에 실행
- 42. 의존성 lockfile 과 SBOM 생성
- 43. OSV-Scanner 로 알려진 취약점 검사
- 44. secret scan 과 개인정보 문자열 검사
- 45. 네트워크 packet size/rate/permission fuzz
- 46. 파일/경로/압축 해제 zip-slip 검사
- 47. 서버 main thread blocking 및 concurrency 검사
- 48. 권한 목록과 외부 endpoint 를 사용자에게 표시
- 49. JAR, 소스 ZIP, asset manifest, test report 에 해시/서명

CodeQL 은 코드를 데이터처럼 질의해 취약점과 오류를 찾고, OSV-Scanner 는 프로젝트 의존성과 OSV 취약점 DB 를 연결한다. 어느 한 도구도 완전하지 않으므로 두 종류 이상의 검사와 런타임 테스트를 결합한다. [R30][R31]

14.3 MCP 권한 매니페스트

```
{
  "server": "blockbench-mcp@commit",
  "permissions": {
    "filesystem_read": ["workspace/art_source/**"],
    "filesystem_write": ["workspace/art_source/**", "generated/assets/**"],
    "network": "deny",
    "process_spawn": ["blockbench-headless"],
    "secrets": []
  },
  "tool_allowlist": ["model.create", "uv.pack", "preview.render"],
  "requires_user_approval": ["plugin.install", "external_export"],
  "artifact_hash": "sha256:..."
}
```

15. 최종 모드 재투입, 보완, 확장 및 장기 진화

사용자가 요구한 핵심 기능은 완성된 모드를 다시 시스템에 넣어 보완점을 찾고, 보안 수정, 스케일 확대, 새 프롬프트 기반 후속 개발을 이어가는 것이다. 이것은 단순 채팅 기억이 아니라 릴리스 단위의 구조화된 재투입 파이프라인으로 구현한다.

15.1 릴리스 재투입 대상

- 승인된 Project IR 과 acceptance tests
- 정확한 소스 commit, JAR, generated resources, Blockbench/ComfyUI 원본 workflow
- 빌드/테스트/런타임/성능/보안 로그
- 게임 스크린샷과 애니메이션 캡처
- SBOM, dependency graph, license/provenance manifest
- 사용자가 리뷰에서 수정한 결정과 만족/불만족 항목
- 실제 플레이 telemetry 는 명시적 동의와 익명화 후에만 포함

15.2 Improvement Auditor

감사 축	분석	출력
사양 충족	승인 기능과 실제 테스트/파일 비교	누락/과잉 구현 및 evidence
보안	새 CVE, 권한, packet, 저장/경로, MCP/의존성	우선순위별 patch plan
성능	tick, heap, entity/particle, network, load time	병목과 scale limit
품질	texture/model/animation/audio 일관성	수정 후보와 preview
호환성	loader/API/version, 다른 인기 모드 dependency/conflict	compatibility matrix
확장성	새 콘텐츠 추가 시 반복 코드/registry/data schema	리팩터링과 generator 개선
사용성/접근성	키 설정, 자막, 색 대비, 음성 fallback	접근성 patch
트렌드 적합성	최근 수요와 포화도, 사용자 목표	선택적 확장 제안

15.3 후속 프롬프트 처리 예

사용자: "완성된 모드에서 보스를 1 개에서 5 개로 늘리고, 20 명 서버에서도 버티게 해. 음성 명령도 추가해."

- release corpus 에서 현재 boss FSM, registry, loot, arena, packet, test 를 검색
- 변경 영향 계산:
 - 신규 boss 4 개: 모델/texture/animation/sound/loot/progression
 - 공통 Boss Framework 추출 필요
 - 20 명: entity AI/tick/packet/particle budget 재설계
 - voice: client STT + server intent validation + privacy UI
- Delta Proposal v2 를 사용자에게 제시
- 사용자가 수락한 항목만 spec v2 로 freeze
- 기존 회귀 suite + 신규 scale/security tests 실행
- v2 릴리스 후 v1 save migration 및 rollback package 제공

15.4 스케일 확대 시나리오

확장	분석해야 할 것	자동 테스트
콘텐츠 10 배	registry/datagen 반복성, asset style drift, load time	전체 registry coverage, duplicate ID, asset batch QA
플레이어 20-100 명	tick budget, async IO, packet fanout, saved data lock	bot load, p95 tick, packet rate, reconnect
월드 크기 확대	worldgen cost, chunk ticket, structure density	seed matrix, pregeneration, chunk time
멀티로더	공통 도메인과 Fabric/NeoForge adapter 분리	loader 별 build/GameTest/compat
새 Minecraft 버전	mapping/API/resource schema 변화	porting diff, migration tests, save compatibility
모드팩 호환	Mixin/registry/tag/config 충돌	대표 dependency matrix 와 startup sweep

버전 정책 기존 세이브를 깨는 변경은 자동으로 숨기지 않는다. migration 가능 여부, 데이터 손실 가능성, major/minor 버전, rollback 방법을 사용자에게 제시하고 승인받는다.

16. 프롬프트 및 MCP 호출 명세

16.1 중앙 오케스트레이터 시스템 프롬프트

You are the Minecraft Production Orchestrator.

Mission:

Convert an approved user intent into a tested, secure, traceable Minecraft project.
Never treat generated code or a successful build as completion.

Rules:

1. Create Proposal and Acceptance Tests before any write action.
2. Execute only after the user accepts the proposal hash.
3. Preserve edition, version, loader, mappings, IDs and license policy.
4. Use the least invasive artifact type that satisfies the tests.
5. Expose only the current tool group; never expose all MCP tools.
6. Query version-specific documentation/source when any API is uncertain.
7. Bulk JSON and block placement must use deterministic generators.
8. Separate generator, verifier and release auditor roles.
9. On failure: localize root cause, create minimal patch, rerun targeted and regression tests.
10. After three failures for the same root cause, stop with a blocked report.
11. Scope-changing fixes require a Change Request and user approval.
12. Every artifact must have provenance, license and hash metadata.
13. Publishing, destructive operations, security exceptions and voice-data retention require explicit approval.
14. Return JSON matching the requested schema only.

16.2 계획 생성 프롬프트

Inputs:

- raw user request and attachments
- user constraints
- authoritative RAG evidence
- trend report with dates/confidence
- available compute and tool policy

Return Proposal JSON:

- summary and assumptions
 - target platform/version/loader
 - feature graph and artifact routes
 - content/file estimates
 - existing alternatives and differentiation
 - privacy/license/security decisions
 - T4-local vs external-compute tasks
 - acceptance tests
 - risks and unresolved decisions
 - 2 alternative scopes when useful
- Do not generate implementation files.

16.3 코드 작업자 프롬프트

You implement one approved task only.

Use supplied Project IR, shared registry and evidence excerpts.

Do not invent APIs. Request source.lookup when uncertain.

Separate domain logic from loader/client adapters.

Prefer datagen for JSON resources.

Return file patches, commands and expected postconditions.

Do not change versions, IDs or dependencies unless the task explicitly authorizes it.

16.4 에셋 작업자 프롬프트

Create a Minecraft asset under the approved style bible.

Inputs include exact output path, UV layout, palette, resolution, neighboring assets and license policy.

Output the editable source, final runtime asset, workflow metadata and validation report.

Never change registry IDs or paths.

Generated concept art is not a final texture until pixel, UV, path and in-game view checks pass.

16.5 독립 시각 검증 프롬프트

You are a visual verifier, not a designer.

Compare screenshots against acceptance criteria and reference assets.

Locate issues with bounding boxes and confidence.

Check missing textures, UV errors, clipping, z-fighting, unreadable silhouettes,

GUI text overlap, animation penetration and style drift.

Do not mark deterministic path/schema checks; those come from validators.

Return pass, fail or uncertain with evidence and a minimal correction request.

16.6 보안 감사 프롬프트

Inputs: source graph, dependency graph, SBOM, CodeQL/SCA results, network payload definitions, permissions, logs and release diff.
Prioritize exploitable findings. Do not dismiss scanner output without evidence.
Return severity, affected path, attack scenario, evidence, minimal remediation, regression/security test and whether release must be blocked.

16.7 최종 모드 개선 감사 프롬프트

Compare the released artifact to the approved spec and all evidence.
Analyze security, performance, maintainability, asset quality, accessibility, compatibility, scale limits and current ecosystem opportunities.
Produce a Delta Proposal only. Do not modify the release.
Separate required fixes from optional enhancements and trend-driven ideas.
Preserve save compatibility unless the user explicitly approves a migration.

16.8 MCP 호출 예시

A. Minecraft API 소스 조회

```
{
  "tool": "minecraft_source.lookup",
  "arguments": {
    "minecraft_version": "PINNED",
    "loader": "fabric",
    "mapping": "PINNED",
    "symbol": "target class or method",
    "include_callers": true
  },
  "postcondition": {"evidence_count_min": 1, "exact_descriptor": true}
}
```

B. 빌드 및 테스트

```
{
  "tool": "fabric_build.run",
  "arguments": {
    "workspace": "workspace/project",
    "tasks": ["clean", "runDatagen", "test", "build"],
    "timeout_seconds": 1200,
    "network": "dependency_mirror_only"
  },
  "postcondition": {
    "exit_code": 0,
    "artifact_glob": "build/libs/*.jar",
    "no_uncommitted_generated_diff": true
  }
}
```

C. 플레이 테스트

```
{
  "tool": "playtest.scenario_run",
  "arguments": {
    "server_session": "...",
    "scenario_id": "boss_phase_transition",
    "seed": 18421,
    "steps": [
      {"action": "join"},
      {"action": "spawn", "entity": "frostbound:frost_king"},
      {"action": "damage_to_ratio", "value": 0.49},
      {"observe": "boss_phase", "expected": 2}
    ]
  },
  "postcondition": {"passed": true, "evidence_required": true}
}
```

D. 최종 모드 재투입

```
{
  "tool": "release_audit.ingest",
  "arguments": {
    "spec_hash": "...",
    "source_commit": "...",
    "jar_path": "release/mod.jar",
    "reports": ["tests.json", "security.sarif", "performance.json"],
    "screenshots": "release/screenshots/**",
    "sbom": "release/sbom.spdx.json"
  },
  "postcondition": {"artifact_graph_complete": true, "hashes_verified": true}
}
```

17. T4 배치 구조와 자원 계획

T4 환경에서는 하나의 Python 프로세스에 모든 모델을 올리는 대신 Model Gateway 와 작업 큐가 GPU 를 독점 임대한다. Minecraft 서버, Gradle, Datagen, JSON 검사, 보안 스캐너는 CPU 와 시스템 RAM 에서 실행한다.

작업 단계	GPU	시스템 자원	운용 정책
기획·웹/RAG	원격 GLM-5.2/Kimi K3/K2.6 또는 로컬 Qwen3.5-9B	RAG DB, evidence graph	고난도는 원격; local 은 IR 초안/오프라인 fallback
코드·수리	원격 Kimi K2.7-Code/Qwen3-Coder-Next	repo index, build sandbox	승인된 task 와 관련 파일만; T4 fallback Qwen3.5-9B
시각 검증	Qwen3-VL-8B 4-bit	screenshot/voxel render staging	다른 모델 완전 unload, batch=1
텍스처 생성	FLUX.2-klein-4B	ComfyUI workflow, CPU pixel compiler	단독 GPU lease; 16-64px 는 후처리로 확정
3D/모션 초안	Hunyuan3D 2.1 / Wan2.2 선택	mesh/video 변환 RAM	원격 기본, 결과를 Blockbench/keyframe IR 로 재컴파일
RAG 검색	Qwen3-Embedding/Reranker 0.6B	vector+BM25+graph index	CPU 또는 작은 GPU, 생성 모델과 분리
음성	SenseVoice/Fun-ASR, CosyVoice 3.0	audio cache	CPU fallback 가능
빌드/서버	GPU 거의 불필요	Java heap 3-6GB, Gradle cache	LLM unload 가능
보안/재감사	독립 범용 LLM 선택	CodeQL/OSV/SBOM	scanner 와 실행 증거가 source of truth

17.1 GPU Lease Manager

```
acquire(model_role, min_vram, timeout)
-> wait for current model to finish
-> unload current pipeline
-> synchronize CUDA
-> clear cache and verify free memory
-> load pinned model revision
-> run bounded job
-> save output/provenance
-> unload and release lease
```

17.2 디스크/메모리 권장

항목	권장
GPU	NVIDIA T4 16GB
시스템 RAM	최소 20GB, 권장 25GB 이상
디스크	단일 버전 MVP 35-50GB 여유; 다중 loader/version 은 80GB 이상
Java server heap	기본 3-4GB, load test 6GB 내
모델 캐시	계획 LLM/VLM/FLUX.2-klein 를 별도 revision 으로 pin
프로젝트 캐시	Gradle, Minecraft source index, test worlds 분리
아티팩트 보존	최종 릴리스와 provenance 만 장기 보존; 임시 latent/mesh 는 정책에 따라 삭제

17.3 장애 복구

- 각 DAG node 완료 시 state 와 artifact hash 를 저장한다.
- 런타임 중단 후 완료된 node 는 재실행하지 않고 postcondition 만 재검증한다.

- 모델 다운로드와 Gradle dependency 는 checksum 캐시를 사용한다.
- 서버 테스트 world 는 snapshot/restore 하고 테스트 간 상태를 격리한다.
- OOM 발생 시 무조건 더 작은 모델/해상도/배치로 정책적으로 fallback 하며 임의 파라미터 변경을 기록한다.

18. 단계별 구현 로드맵과 합격 기준

18.1 Phase 0 - 기반과 안전 경계

- Project IR, Proposal, approval state machine, shared registry, artifact manifest 구현
- MCP gateway allowlist, permission manifest, audit log, sandbox 구현
- 한 Minecraft/Fabric 버전 pin, 템플릿 저장소와 build runner 구축
- 공식 문서/소스 RAG 와 최소 평가 세트 구축

합격 사용자 승인 전 write tool 0 회, 승인 해시와 모든 파일 provenance 추적, 고의 prompt injection 문서가 tool call 을 유발하지 않음.

18.2 Phase 1 - 코드/Datagen MVP

- 아이템, 블록, 레시피, loot, tag, language 자동 생성
- Qwen Coder + Minecraft Dev MCP + Fabric build/test 연결
- Agentless 식 localize-repair-validate 루프
- JAR, source ZIP, test report 패키징

합격 승인된 소형 광물/도구 모드 5 개 시나리오에서 clean build, registry/recipe/loot tests 100%, 동일 오류 재발률 측정.

18.3 Phase 2 - 텍스처와 VLM

- ComfyUI FLUX.2-klein-4B workflow, palette/alpha/pixel pipeline
- 아이템/블록/GUI screenshot 자동 캡처
- Qwen3.5-9B 또는 Gemma 4 E4B 시각 검증과 기준 이미지 diff
- 사용자 preview 수정/승인 UI

합격 누락 텍스처/경로 오류는 결정론적 검사 100% 검출, UV/GUI 시각 오류 평가 세트의 precision/recall 목표를 별도 정의하고 회귀.

18.4 Phase 3 - 엔티티/애니메이션/음성

- Blockbench adapter, GeckoLib export, bone/UV/animation validator
- TripoSR 형태 참조 선택 경로
- faster-whisper 제작 음성 입력과 제한 명령
- NPC TTS/OGG/sounds.json pipeline

합격 단순 몸과 보스 템플릿에서 animation event 와 damage tick 일치, 음성 고위험 명령은 100% 화면 재확인.

18.5 Phase 4 - 실제 플레이와 보안

- Mineflayer scenario runner 와 pathfinding
- GameTest + runtime inspector + load bots
- CodeQL/OSV/SBOM/secret scan 릴리스 게이트
- 악성 RAG/MCP 테스트와 공급망 pin 검증

합격 보안 게이트 실패 시 JAR 배포 불가, 플레이 시나리오 evidence 재현, 20 명 scale 목표에서 p95 tick 기준 충족.

18.6 Phase 5 - 재투입과 진화

- 릴리스 corpus 자동 ingest 및 spec-vs-artifact diff
- 보완/확장/마이그레이션 Delta Proposal
- 과거 save 와 회귀 suite 관리
- 트렌드 RAG 를 선택적 제안으로 통합
- 승인된 수정 데이터를 LoRA/adaptor 학습 후보로 축적

합격 완성된 v1 에 대한 후속 프롬프트가 전체 재생성이 아니라 영향 범위와 최소 델타 계획을 생성하고, v1 회귀 테스트와 save migration 을 보존.

19. 위험, 한계 및 현실적 범위

위험/한계	현실적 대응
T4 에서 3D/모션/이미지/LLM 동시 실행 불가	GPU lease 와 순차 작업; 무거운 모델은 외부 경로
복잡한 보스의 미적 완성도	archetype, Blockbench 템플릿, 사용자 preview 승인
버전별 Minecraft API 변화	한 버전 우선, source RAG, porting suite
VLM 의 시각 오판	결정론적 검사 우선, 다중 view, uncertain 허용
자기수정 무한 루프	같은 root cause 3 회 제한, 외부 verifier, blocked report
오픈소스 MCP 품질 편차	공식/검증 repo 만 pin, 직접 domain adapter 를 핵심으로
트렌드 데이터 편향	출처/날짜/포화/신뢰도 표시, 사용자 선택
에셋 저작권과 학습 권한	provenance, 라이선스 gate, 유사도 검사, 승인 데이터만 학습
음성 개인정보	local default, 원본 저장 off, 명시적 동의
자동 보안 검사 한계	SAST+SCA+fuzz+runtime+human review 결합
재투입된 JAR 분석의 법적 문제	사용자 소유/허가 프로젝트만; 제 3 자 코드 무단 decompile 금지

최종 범위 제한 첫 제품은 Fabric 한 버전, 아이템/블록/단순 몸/GUI/구조물, 텍스처 생성, GameTest, Mineflayer, 보안 스캔, 최종 재투입까지 완성한다. 고품질 3D 보스 자동 생성, 모든 로더/버전, 대규모 모션 모델 학습은 후속 단계로 분리한다.

19.1 최종 의사결정

이 시스템은 기술적으로 구현 가능하다. 성공 여부는 어떤 거대 멀티모달 모델을 고르는가보다 다음 네 가지에 달려 있다.

- 50. 사용자에게 계획을 먼저 보여주고 승인된 사양만 실행하는가.
- 51. 공식 버전 지식, 프로젝트 기억, 트렌드, 실패 기억을 분리한 RAG 를 갖추는가.
- 52. 코드/에셋 생성과 독립된 결정론적 및 런타임 검증기를 갖추는가.
- 53. 완성된 모드를 다시 구조화해 보안 수정과 확장을 델타 방식으로 이어갈 수 있는가.

20. 참고 문헌과 공식 자료

아래 자료는 문서 작성 기준일에 확인한 공식 문서, 공식 저장소 및 1 차 연구 논문이다. Minecraft 와 MCP 생태계는 빠르게 변경되므로 구현 시 revision/commit 을 다시 pin 해야 한다.

- [R01] **NVIDIA T4 Tensor Core GPU - 16GB GDDR6 specifications.** <https://www.nvidia.com/en-us/data-center/tesla-t4/>
- [R02] **Qwen3.5-9B model card.** <https://huggingface.co/Qwen/Qwen3.5-9B>
- [R03] Qwen3.5-9B official model card. <https://huggingface.co/Qwen/Qwen3.5-9B>
- [R04] **CosyVoice 3.0 official inference repository.** <https://github.com/hexgrad/kokoro>
- [R05] **Qwen3-VL-8B-Instruct 4-bit model card.** <https://huggingface.co/Qwen/Qwen3-VL-8B-Instruct-4-bit>
- [R06] **ComfyUI official repository.** <https://github.com/comfy-org/comfyui>
- [R07] **faster-whisper official repository.** <https://github.com/SYSTRAN/faster-whisper>
- [R08] **Model Context Protocol - Tools specification 2025-11-25.**
<https://modelcontextprotocol.io/specification/2025-11-25/server/tools>
- [R09] **Security Analysis of the Model Context Protocol.** <https://arxiv.org/abs/2601.17549>
- [R10] **A Measurement Study of Model Context Protocol.** <https://arxiv.org/abs/2509.25292>
- [R11] **Minecraft Dev MCP official repository.** <https://github.com/MCDxAl/minecraft-dev-mcp>
- [R12] **Fabric Data Generation Setup.** <https://docs.fabricmc.net/develop/data-generation/setup>
- [R13] **Fabric Automated Testing.** <https://docs.fabricmc.net/develop/automatic-testing>
- [R14] **JSPLIT: taxonomy-based prompt-bloating reduction for MCP.** <https://arxiv.org/abs/2510.14537>
- [R15] **Self-RAG: Learning to Retrieve, Generate, and Critique.** <https://arxiv.org/abs/2310.11511>
- [R16] **Corrective Retrieval Augmented Generation.** <https://arxiv.org/abs/2401.15884>
- [R17] **Microsoft GraphRAG official repository.** <https://github.com/microsoft/graphrag>
- [R18] **RAGAS: Automated Evaluation of RAG.** <https://arxiv.org/abs/2309.15217>
- [R19] **Modrinth Search Projects API.** <https://docs.modrinth.com/api/operations/searchprojects/>
- [R20] **Agentless: Demystifying LLM-based Software Engineering Agents.** <https://arxiv.org/abs/2407.01489>
- [R21] **ReAct: Synergizing Reasoning and Acting in Language Models.** <https://arxiv.org/abs/2210.03629>
- [R22] **Reflexion: Language Agents with Verbal Reinforcement Learning.** <https://arxiv.org/abs/2303.11366>
- [R23] **Self-Refine: Iterative Refinement with Self-Feedback.** <https://arxiv.org/abs/2303.17651>
- [R24] **CRITIC: Tool-Interactive Critiquing.** <https://arxiv.org/abs/2305.11738>
- [R25] **On the Self-Verification Limitations of LLMs.** <https://arxiv.org/abs/2402.08115>
- [R26] **Mineflayer official repository.** <https://github.com/PrismarineJS/mineflayer>
- [R27] **MCP: Landscape, Security Threats, and Future Directions.** <https://arxiv.org/abs/2503.23278>
- [R28] **Microsoft Playwright MCP official repository.** <https://github.com/microsoft/playwright-mcp>
- [R29] **From Component Manipulation to System Compromise: malicious MCP detection.** <https://arxiv.org/abs/2604.01905>
- [R30] **GitHub CodeQL documentation.** <https://codeql.github.com/docs/>
- [R31] **OSV-Scanner official documentation.** <https://google.github.io/osv-scanner/>
- [R32] **SWE-agent: Agent-Computer Interfaces.** <https://arxiv.org/abs/2405.15793>
- [R33] **Tree Search for Language Model Agents.** <https://arxiv.org/abs/2407.01476>
- [R34] Qwen3.5-9B official multimodal model card. <https://huggingface.co/Qwen/Qwen3.5-9B>
- [R35] **Qwen3.5-9B 또는 Kimi K2.6(원격) model card.** [https://huggingface.co/Qwen/Qwen3.5-9B 또는 Kimi K2.6\(원격\)](https://huggingface.co/Qwen/Qwen3.5-9B 또는 Kimi K2.6(원격))
- [R36] **Blockbench Plugin documentation.** <https://www.blockbench.net/wiki/docs/plugin/>
- [R37] **Blockbench Reference Docs.** <https://web.blockbench.net/docs/>
- [R38] **Fabric Developer Guides.** <https://docs.fabricmc.net/develop/>
- [R39] **MCP official specification repository.** <https://github.com/modelcontextprotocol/modelcontextprotocol>
- [R40] **CurseForge for Studios REST API.** <https://docs.curseforge.com/rest-api/>
- [R41] **GitHub Code scanning with CodeQL.** <https://docs.github.com/code-security/code-scanning/introduction-to-code-scanning/about-code-scanning-with-codeql>
- [R42] **OSV vulnerability database and workflows.** <https://osv.dev/>
- [R43] **Minecraft GameTest Framework overview.** <https://www.minecraft.net/en-us/creator/article/get-started-gametest-framework>
- [R44] **Fabric porting docs/current version guide.** <https://docs.fabricmc.net/develop/porting/>
- [R45] **Google Trends API alpha announcement.** <https://developers.google.com/search/blog/2025/07/trends-api>
- [R46] **Modrinth platform/discovery.** <https://modrinth.com/>

- [R47] FLUX.2-klein-4B paper/model. <https://arxiv.org/abs/2402.13929>
- [R48] IP-Adapter official repository. <https://github.com/tencent-ailab/IP-Adapter>
- [R49] TripoSR official repository. <https://github.com/VAST-AI-Research/TripoSR>
- [R50] InstantMesh official repository. <https://github.com/TencentARC/InstantMesh>
- [R51] Hunyuan3D 2 official repository. <https://github.com/Tencent-Hunyuan/Hunyuan3D-2>
- [R52] MotionGPT: Finetuned LLMs Are General-Purpose Motion Generators. <https://arxiv.org/abs/2306.10900>
- [R53] SDXL model paper. <https://arxiv.org/abs/2307.01952>
- [R54] IP-Adapter paper. <https://arxiv.org/abs/2308.06721>
- [R55] TripoSR technical report. <https://arxiv.org/abs/2403.02151>
- [R56] InstantMesh paper. <https://arxiv.org/abs/2404.07191>
- [R57] Hunyuan3D 2.0 paper. <https://arxiv.org/abs/2501.12202>

부록 A. 구현 체크리스트

사용자 승인

- ☐ Proposal 생성 전 write 없음
- ☐ 가정/버전/범위/테스트 표시
- ☐ 승인 해시 생성
- ☐ 범위 변경 재승인
- ☐ 게시/삭제/보안 예외 별도 승인

RAG

- ☐ 인덱스 trust 분리
- ☐ 버전/loader filter
- ☐ BM25+dense+rerank
- ☐ 검색 품질 evaluator
- ☐ 근거/날짜 표시
- ☐ prompt injection 평가

MCP

- ☐ commit/hash pin
- ☐ permission manifest
- ☐ 도구 5-10 개 제한
- ☐ dry-run/read-only 기본
- ☐ postcondition
- ☐ audit log
- ☐ egress 제한

에셋

- ☐ style bible
- ☐ editable source 보존
- ☐ workflow/seed 기록
- ☐ pixel/UV/path validator
- ☐ VLM 다중 view
- ☐ license/provenance

코드

- ☐ 공식 API evidence
- ☐ Datagen 우선
- ☐ Mixin 최소
- ☐ 도메인/loader 분리
- ☐ unit/GameTest
- ☐ 최소 patch

보안

- ☐ CodeQL/SAST

- ☐ OSV/SCA
- ☐ SBOM
- ☐ secret scan
- ☐ packet fuzz
- ☐ thread/IO 검사
- ☐ JAR hash/sign

재투입

- ☐ spec-artifact diff
- ☐ 릴리스 corpus
- ☐ 보안 재스캔
- ☐ scale limit
- ☐ save migration
- ☐ Delta Proposal
- ☐ 회귀 suite

부록 B. MVP 권장 기술 스택

계층	권장
Frontend	React/Next.js 또는 단순 웹 UI; 계획 diff, 승인, preview, 로그
Orchestrator	Python FastAPI + 명시적 상태 머신/LangGraph 선택
Queue/State	SQLite/PostgreSQL + Redis 선택; artifact 는 content-addressed storage
LLM serving	LiteLLM Router + vLLM/SGLang; 원격 frontier 와 T4 local endpoint 통합
VLM	Qwen3-VL-8B-Instruct 4-bit; screenshot batch=1
Image	ComfyUI + FLUX.2-klein-4B + deterministic pixel compiler
RAG	Qwen3-Embedding-0.6B + Qwen3-Reranker-0.6B + Qdrant/pgvector + BM25
Minecraft	Fabric Loom, Fabric API, GameTest, Minecraft Dev MCP
Model/animation	Blockbench plugin adapter + GeckoLib 선택
Playtest	Mineflayer + custom scenario MCP
Voice	SenseVoice/Fun-ASR + local VAD; CosyVoice 3.0 선택
Security	CodeQL/Semgrep, OSV-Scanner, secret scanner, Syft/CycloneDX SBOM
CI	GitHub Actions 또는 로컬 runner; reproducible release pipeline

개정판 안내: “도구를 많이 연결”이 아니라 “검증된 도구 표면을 통제”한다

이 확장판은 기존 1-20 장의 제품 개념을 유지하면서, 2026 년 7 월 현재 사용할 MCP, API, Skills, 모델, 평가 도구와 연구 결과를 구현 단위로 다시 감사한 것이다. 공개 저장소가 존재한다는 사실만으로 채택하지 않는다. 공식성, 유지보수 상태, 라이선스, 프로토콜 호환성, 권한 범위, 테스트 가능성, T4 적합성, 대체 경로를 기준으로 분류한다.

정확성 한계 “모든 기법을 넣으면 완벽해진다”는 보장은 존재하지 않는다. 목표는 오류를 0 으로 주장하는 것이 아니라, 요구사항 누락·API 환각·리소스 참조 오류·런타임 실패·보안 문제를 각각 측정하고 승인 가능한 오류 예산 아래로 낮추는 것이다. 생성 모델의 자기평가만으로 합격시키지 않는다.

21. 2026-07 MCP 사양 변경과 개정 원칙

MCP 2026-07-28 릴리스 후보는 무상태 코어, 확장 프레임워크, Tasks, MCP Apps, 권한 강화, 완전한 JSON Schema 2020-12 지원을 도입한다. 반대로 구형 초기화 세션, Roots, Sampling, Logging 의 일부 형태는 폐기 또는 재구성 경로에 들어갔다. 따라서 이 시스템은 프로토콜 세션에 프로젝트 상태를 기대하지 않고, Project IR 데이터베이스와 작업 이벤트 저장소가 상태의 단일 원천이 되도록 한다.

[R58][R59]

변경점	설계 반영	금지할 가정
무상태 코어	각 요청에 project_id, plan_version, artifact_revision, auth_context 를 전달한다. 장기 작업은 Tasks 확장 또는 자체 작업 큐로 추적한다.	MCP 연결이 유지되면 작업 상태도 자동 보존된다는 가정
Extensions / Tasks	빌드, 렌더, GameTest, 월드 생성처럼 수분 이상 걸리는 작업을 비동기 작업 객체로 추상화한다.	하나의 tool/call 응답이 끝날 때까지 HTTP 연결을 붙잡기
MCP Apps	사용자 계획 승인, diff 검토, 위험 권한 승인에 대화형 UI 를 붙일 수 있다. 초기 MVP 는 동일 기능을 웹 UI 로 구현한다.	UI 가 없으면 프롬프트 문장만으로 위험 승인을 받기
권한 강화	OAuth/OIDC, 작업자 신원, 도구별 scope, 프로젝트별 ABAC 를 게이트웨이에서 강제한다.	도구 설명의 readOnlyHint/destructiveHint 가 보안 통제라고 믿기
JSON Schema 2020-12	모든 도구 입력·출력에 strict schema, enum, pattern, maxItems, additionalProperties:false 를 적용한다.	자유형 문자열 하나에 명령 전체를 넣기
Roots 의 한계	프로젝트 경로 제한은 컨테이너 마운트와 OS 권한으로 강제한다. Roots 는 탐색 힌트로만 취급한다.	Roots 가 파일 접근을 차단한다고 가정

- 브로커는 2025-11-25 안정 사양과 2026-07-28 후보를 동시에 탐지하고, server/discover 또는 버전 협상을 통해 어댑터를 선택한다. [R60]
- 서버가 제공하는 tool annotations 는 사용자 설명과 UI 표시에는 쓰되, 실제 권한 결정에는 사용하지 않는다.
- 모든 외부 MCP 는 직접 중앙 모델에 연결하지 않고 MCP Gateway/Broker 뒤에 둔다. 서버별 outbound network, filesystem, secrets, CPU/RAM/time quota 를 별도로 제한한다.
- MCP 공식 reference server 는 예제라는 성격이 강하므로 production-ready 로 간주하지 않고 최소 기능을 자체 래퍼로 고정한다.

22. 실제 사용할 MCP/API/스킬 전수 카탈로그

아래 목록은 “인터넷에 존재하는 모든 MCP”가 아니라 이 제품을 완성하는 데 필요한 도구 표면 전체다. 상태는 필수 채택, 래퍼 후 채택, 실험 격리, 자체 구현, 제외로 구분한다. 커뮤니티 서버는 커밋 해시·라이선스·권한·테스트를 통과하기 전에는 배포 환경에 넣지 않는다.

22.1 제어, 저장소, 승인 및 관측 계층

구성요소	정확한 역할	권장 구현	상태
MCP Gateway/Broker	도구 발견, 서버 연결, 정책 평가, 자격증명 주입, rate limit, 감사 로그	Docker MCP Gateway 또는 자체 FastAPI/Go broker	필수
Project IR MCP	요구사항/계획/ID/의존성/테스트/승인 버전의 CRUD와 JSON Patch	자체 구현; PostgreSQL + object store	필수
Approval MCP/App	proposal diff 표시, 사용자 수정, approve/reject/branch, 위험 권한 승인	MCP Apps 또는 자체 React UI	필수
Task Queue MCP	장기 빌드/렌더/테스트 작업 제출, 상태, 취소, 결과 스트리밍	Celery/Redis 또는 Temporal 래퍼	필수
Filesystem Read MCP	승인된 프로젝트 스냅샷 읽기, glob, hash, diff	공식 filesystem 개념을 read-only 컨테이너로 래핑	필수
Filesystem Write MCP	패치 적용, 새 파일 생성, 원자적 교체	허용 경로 + temp staging + diff approval	필수/고위험
Git MCP	status/diff/show/log/blame/branch/commit/tag	공식 reference 또는 libgit2 래퍼	필수
GitHub MCP	이슈/PR/Actions/코드 검색/릴리스	GitHub 공식 MCP; read와 write scope 분리	선택-필수
Artifact MCP	JAR/ZIP/PNG/OGG/bbmodel/SBOM/보고서 저장과 해시	S3/MinIO 로컬 래퍼	필수
Tracing MCP/API	LLM span, retrieval, tool call, token, latency, cost, artifact lineage	OpenTelemetry + Phoenix	필수
Secret Broker	HF/GitHub/CurseForge/API 키를 작업자에게 단기 토큰으로 전달	Vault/OIDC workload identity	필수
Notification Skill	승인 필요, 테스트 실패, 릴리스 완료 통지	웹 UI/메일/메신저 어댑터	선택
Budget Controller	GPU/토큰/시간/재시도 예산과 중단 기준 집행	자체 정책 서비스	필수

22.2 Minecraft 소스, 버전, 빌드 및 런타임 계층

MCP/API/Skill	도구 표면	연결 방식/제약	상태
Minecraft Dev MCP (MCDxAI)	버전별 JAR 디컴파일, 매핑, 클래스/메서드 검색, Mixin/Access Widener 근거 수집	공식 저장소 pin + read-only cache	래퍼 후 채택
mcdev-mcp + DebugBridge	실행 중 게임 snapshot, block/entity/item 조사, screenshot/video, Groovy 진단	테스트 인스턴스 전용; production 월드 금지	실험 격리
Mineflayer MCP	봇 접속, 이동, 채굴, 배치, 인벤토리, 채팅, 엔티티 상호작용	지원 버전 매트릭스 고정; raw chat 명령 제한	필수
Mineflayer Core/API	pathfinder, state machine, pvp, auto-eat, inventory 플러그인	Node worker; 명시적 behavior tree	필수
MCreatorMCP	workspace/element/build/runServer/runClient 조작	단순 콘텐츠 fallback; 라이선스 메타데이터 불일치 감사	실험 격리
Fabric Meta API	Minecraft/Loader/Intermediary/Loom 호환 버전 조회	캐시 TTL + 서명/출처 기록	필수
Fabric Loom/Gradle API	프로젝트 생성, remap, datagen, unit/GameTest, runClient/runServer	커스텀 build MCP가 명령 allowlist로 호출	필수
NeoForge ModDevGradle/NeoGradle	NeoForge 프로젝트 생성, userdev, run config, datagen, test	2단계 확장; 버전별 템플릿	확장
Paper API/Downloads	Paper 버전/빌드 조회, 플러그인 API, 테스트	GraphQL/API 래퍼 + checksum	필수(Paper)

MCP/API/Skill	도구 표현	연결 방식/제약	상태
	트 서버 다운로드		
Bedrock Script API/GameTest	Behavior/Resource pack, Script API, GameTest	별도 bedrock worker; Java IR 에서 변환	3 단계
Minecraft Datagen MCP	recipe/loot/tag/model/lang/worldgen provider 생성·실행·diff	Fabric/NeoForge provider 공통 facade	필수
GameTest MCP	테스트 템플릿 설치, headless 실행, 결과/JUnit XML 수집	loader 별 adapter	필수
WorldEdit MCP	schematic load/save/paste/region/undo, command safety	WorldEdit API 직접 wrapper	필수(맵)
GDPC/GDMC Interface	지형 read, buffered block placement, procedural geometry	Python worker + block budget	선택·필수
Server Harness MCP	EULA isolated server, boot wait, RCON/console, log/crash capture	Docker container per test	필수
Client Harness MCP	virtual display, screenshot, input replay, frame capture	Xvfb/wayland + fixed camera scenes	2 단계
Spark/JFR Profiler MCP	tick, heap, allocation, hot method, plugin/mod timing	테스트 서버에서만 profile	필수
Compatibility Matrix MCP	loader/mod/dependency/version 조합 생성 및 test matrix	Project IR + CI matrix generator	필수

22.3 텍스처, 3D, 모션, GUI, 오디오 계층

도구	역할	구현/제약	상태
ComfyUI API/MCP	workflow 제출(/prompt), queue/history, websocket 진행, model/object_info, interrupt/free memory	공식 HTTP API 를 최소 스키마 MCP 로 감싼다	필수
Texture Generator Skill	아이템/블록/entity/GUI/particle PNG 생성, palette/alpha/size 정책	ComfyUI workflow + deterministic postprocess	필수
Texture Validator MCP	크기, RGBA, 반투명 픽셀, palette, seam, UV coverage, missing texture	Pillow/OpenCV + 규칙 엔진	필수
Blockbench Plugin/MCP	bbmodel 생성/열기, cube/bone/UV, export, thumbnail render	공식 Plugin API 기반 자체 플러그인	필수
Blender MCP	복잡 3D 생성/수정, material, render, export, Python job	격리된 Blender worker; Minecraft 변환 전용	선택
TripoSR API	단일 이미지 -> 빠른 mesh 초안	T4 순차 실행; 최종 Minecraft 모델로 자동 단순화	선택
Hunyuan3D/TRELLIS API	고품질 3D/texture 초안	원격 GPU 또는 별도 노드; T4 기본 경로 제외	선택/원격
GeckoLib Skill	geo/animation JSON, controller, keyframe event, bone validation	Blockbench exporter + schema validator	필수(복잡 모션)
Motion Retarget MCP	MotionGPT/MDM skeleton motion 을 Minecraft bone/키프레임으로 변환	오프라인 retarget + collision rules	실험
GUI Composer MCP	slot/button/progress/HUD 레이아웃, sprite atlas, screen preview	constraint layout + VLM review	필수
Visual Regression MCP	고정 장면 screenshot, perceptual diff, missing-texture/z-fight 판정	OpenCV/LPIPS + VLM 독립 판정	필수
ASR Skill	음성 요구 입력/게임 음성 명령을 text + timestamp + confidence 로 변환	faster-whisper 또는 SenseVoice	선택·필수
TTS Skill	NPC/안내 음성 생성, 언어/화자/속도/라이선스 기록	MeloTTS 등; 결과 검수	선택

도구	역할	구현/제약	상태
SFX/Music Skill	짧은 효과음/ambient/music 초안	AudioCraft/AudioLDM 또는 라이선스 음원	선택
Audio Processing MCP	trim, normalize, mono/stereo, loop, OGG, loudness, silence check	ffmpeg + librosa; Audacity 는 수동 fallback	필수
Lip/Voice Event Mapper	음성 타임라인을 animation/sound keyframe 과 연결	forced alignment + GeckoLib events	선택

22.4 RAG, 생태계 검색 및 트렌드 계층

소스/스킬	용도	통제 규칙	상태
Context7 MCP/Skill	라이브러리/프레임워크 버전별 문서 검색	보조 출처; Minecraft 공식 문서와 충돌 시 공식 우선	채택
Official Docs Crawler	Fabric/NeoForge/Paper/Bedrock/MCP/GeckoLib 문서 수집	robots/라이선스 준수, URL+날짜+버전 metadata	자체 필수
Source Graph Index	디컴파일 source, symbol, call graph, mapping, mixin target	BM25 + symbol graph + exact source range	자체 필수
Modrinth API	프로젝트 검색, loader/version/category, dependency, update, downloads	API 정책 준수; 수치=인기도 신호이지 품질 보증 아님	필수
CurseForge API	모드/파일/MC 버전/카테고리 검색	API key 와 rate limit, 재배포 금지 정책 확인	선택
GitHub Search/API	레퍼런스 구현, 이슈, 릴리스, commit, license, security advisory	GitHub 공식 MCP 의 read profile	필수
Playwright CLI Skill	동적 문서/릴리스 페이지 확인, 승인된 사이트 screenshot	상시 광범위 MCP 보다 일회성 skill/CLI 우선	선택
Trend Aggregator	최근 업데이트, 다운로드 증가, 이슈, 검색 관심, 커뮤니티 요구를 날짜별 집계	근거 URL 과 관측 기간 표시; 개인화 데이터 수집 금지	자체
Failure Memory RAG	compile/crash/visual/test 실패와 성공 patch 저장	root-cause fingerprint + version filter	필수
Project Memory RAG	승인 사항, 결정, artifact lineage, 사용자 수정 이력	Project IR 에서 파생; 외부 지식과 분리	필수
Security RAG	CVE/OSV/GHSA, loader advisories, MCP 보안 가이드	신뢰 등급과 공표 시각 포함	필수
License/Asset Provenance RAG	코드/모델/음원/텍스처 license, source, hash	unknown 은 자동 배포 차단	필수

22.5 품질, 보안, 릴리스 및 운영 계층

도구	검증/운영 목적	핵심 제한	상태
Gradle/Maven Runner	allowlisted task 실행, timeout, clean cache, JUnit/coverage 수집	shell 자유 임력 금지	필수
Java Static Analysis	Error Prone/SpotBugs/PMD/Checkstyle 규칙 실행	프로젝트 유형별 profile	필수
Semgrep/CodeQL	소스 취약점/위험 패턴/데이터 흐름 분석	CodeQL 은 CI/고위험 릴리스; Semgrep 은 빠른 loop	필수
Dependency Audit	OSV-Scanner, OWASP Dependency-Check, Gradle dependency verification	lockfile/checksum 필수	필수
SBOM MCP	CycloneDX JSON 생성, dependency edge 와 purl 검증	edge 누락 시 unknown 으로 처리	필수

도구	검증/운영 목적	핵심 제한	상태
License Scanner	SPDX/license file/asset source/모델 license 검사	상용 배포 정책과 연결	필수
Secrets Scanner	gitleaks/trufflehog 패턴 + entropy 검사	발견 시 즉시 폐기/회전	필수
JSON/Resource Validator	모델 parent, texture path, lang, sound, tag, loot, recipe schema	버전별 vanilla schema/cache	필수
Property/Fuzz Test MCP	NBT/network/config/parser 경계값, malformed packet, random state	시간 예산 기반	필수
Mutation Test MCP	테스트가 실제 결함을 잡는지 PIT 등으로 확인	핵심 로직만 선택	필수
Performance Gate	TPS/MSPT, heap, GC, chunk generation, network bytes, asset size	baseline 대비 허용치	필수
Compatibility CI	OS/Java/loader/MC/dependency 조합과 clean install	nightly + release matrix	필수
Release Signer	JAR/ZIP/SBOM/provenance 서명	Sigstore keyless 또는 조직 key	필수
SLSA Provenance	source/build/input/output provenance attestation	GitHub Actions 또는 자체 builder	필수
Canary/Recovery Skill	소수 테스트 서버 배포, health metric, rollback	자동 사용자 서버 배포는 명시 승인 필요	선택
Observability Dashboard	trace, 실패율, pass rate, repair count, latency, GPU/비용	Phoenix/OTel/Grafana	필수

23. 서버별 정확한 도구 표면과 권한 프로파일

중앙 모델에 위의 모든 도구를 한 번에 노출하지 않는다. 먼저 단계 라우터가 1 개의 도구 그룹을 선택하고, 브로커가 해당 단계의 5-12 개 도구만 임시로 공개한다. 작은 모델은 도구 수가 늘수록 선택 오류가 빠르게 증가하므로, “도구 카탈로그 검색 → capability token 발급 → 제한된 tools/list”의 2 단계 구조를 사용한다. [R61][R134]

project-ir-mcp

권한 프로파일: 항상 읽기, 승인 후 제한 쓰기

도구명	입력 핵심	출력/후조건	승인
project.get	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
project.create_proposal	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
project.patch_proposal	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
project.diff_plan	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
project.approve_plan	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	필요
project.reject_plan	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
project.branch_plan	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
registry.allocate_ids	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
manifest.expand_content	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
trace.link_requirement	project_id, revision + 도구별 strict	typed result, evidence refs, artifact	정책 기반

도구명	입력 핵심	출력/후조건	승인
	fields	hashes, changed_paths	
decision.record	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
acceptance.list	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반

minecraft-source-mcp

권한 프로파일: 읽기 전용

도구명	입력 핵심	출력/후조건	승인
version.resolve	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
mapping.resolve	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
source.prepare	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
source.search_symbol	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
source.read_range	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
callgraph.neighbors	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
mixin.verify_target	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
access.verify_widener	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
jar.inspect	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
api.compare_versions	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반

build-mcp

권한 프로파일: 프로젝트 컨테이너 내 쓰기/실행

도구명	입력 핵심	출력/후조건	승인
project.scaffold	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
patch.apply	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	필요
datagen.run	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
gradle.run_allowlisted	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
compile.run	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
unit_test.run	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
gametest.run	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반

도구명	입력 핵심	출력/후조건	승인
server.start	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
server.command	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	필요
server.stop	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
logs.collect	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
crash.parse	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
artifact.collect	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반

world-mcp

권한 프로파일: 테스트 월드만 쓰기

도구명	입력 핵심	출력/후조건	승인
world.inspect_region	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
world.heightmap	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
world.plan_dry_run	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
world.apply_plan	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	필요
world.rollback	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	필요
schematic.save	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
schematic.load	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
schematic.paste	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	필요
path.reachability	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
block.sample_validate	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반

asset-mcp

권한 프로파일: 에셋 작업 디렉터리 쓰기

도구명	입력 핵심	출력/후조건	승인
comfy.workflow_validate	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
comfy.queue_prompt	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
comfy.progress	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
comfy.cancel	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반

도구명	입력 핵심	출력/후조건	승인
texture.postprocess	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
texture.validate	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
blockbench.create_model	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
blockbench.export	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
model.validate_bones	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
animation.validate	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
gui.render_preview	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
audio.process	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
asset.register_provenance	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반

playtest-mcp

권한 프로파일: 테스트 서버 행동

도구명	입력 핵심	출력/후조건	승인
bot.connect	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
bot.spawn_reset	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
bot.move	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
bot.find_block	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
bot.interact	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
bot.attack	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
bot.inventory	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
bot.chat_safe	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
scene.capture	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
state.snapshot	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
assert.evaluate	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
bot.disconnect	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반

quality-mcp

권한 프로파일: 읽기 + 보고서 쓰기

도구명	입력 핵심	출력/후조건	승인
static.run	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
dependency.audit	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
license.audit	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
secret.scan	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
resource.validate	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
fuzz.run	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
mutation.run	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
visual.compare	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
performance.profile	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
compat.matrix_run	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
quality.aggregate	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반

release-mcp

권한 프로파일: 승인 후 서명/외부 쓰기

도구명	입력 핵심	출력/후조건	승인
sbom.generate	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
provenance.generate	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
artifact.sign	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	필요
artifact.verify	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
release.stage	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
release.publish	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	필요
canary.deploy	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	필요
canary.observe	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반
release.rollback	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	필요
release.reimport	project_id, revision + 도구별 strict fields	typed result, evidence refs, artifact hashes, changed_paths	정책 기반

23.1 도구 공통 입력 봉투

표 23-1. 모든 MCP 호출에 강제할 공통 envelope

2026-07-28 | Research Audit v2 | Complete Mod+Map v3 | Universal Prompt v4 | Dynamic Router v5 | Dual Profiles v6

```
{
  "project_id": "rk_001",
  "plan_version": 7,
  "artifact_revision": "sha256:...",
  "request_id": "uuid",
  "auth_context": {
    "principal": "agent:tester",
    "scopes": ["runtime.read", "test.execute"],
    "approval_id": null
  },
  "limits": {
    "timeout_s": 600,
    "cpu": 4,
    "memory_mb": 8192,
    "gpu_lease": null,
    "network_policy": "deny"
  },
  "input": {}
}
```

23.2 도구 공통 출력 봉투

```
{
  "status": "succeeded | failed | blocked | needs_approval",
  "result": {},
  "evidence": [
    {"type": "log", "uri": "artifact://...", "sha256": "..."},
    {"type": "source_range", "file": "...", "start": 120, "end": 164}
  ],
  "changed_paths": [],
  "artifacts": [],
  "metrics": {"duration_ms": 0, "peak_memory_mb": 0},
  "error": null,
  "next_allowed_actions": []
}
```

모든 성공 응답은 evidence 를 포함해야 한다. 에이전트가 “성공했다”고 서술하는 것과 실제 도구의 exit code, 테스트 XML, 화면 캡처, 해시가 일치하지 않으면 blocked 로 처리한다.

24. 에이전트 관련 연구를 구현 규칙으로 변환

연구를 이름만 붙이는 방식은 의미가 없다. 아래에서는 각 연구 계열을 실제 제어 규칙으로 변환한다. 기본 원칙은 자유 대화형 멀티에이전트보다 명시적 워크플로를 우선하고, 불확실하거나 창의적인 단계에만 에이전트 탐색을 허용하는 것이다.

연구/계열	핵심 아이디어	본 시스템의 구현 규칙	실패 방지
ReAct	추론·행동·관찰 루프	도구 호출 전 목적/예상 후조건을 구조화하고, 결과 이후 observation 만 상태에 반영	무제한 반복 금지; 3-5 회 예산
Reflexion	실패에서 언어적 교훈 저장	실패 원인·근거·효과 있었던 패치만 Failure Memory 에 저장	자기서술이 아니라 실행 로그로 검증
Self-Refine	초안·피드백·수정	텍스처/설계/문서에는 사용, 코드는 독립 테스트와 결합	동일 모델의 자기칭찬을 합격 근거로 사용 금지
CRITIC	외부 도구 기반 비평	컴파일러, API source, tests, VLM, profiler 결과를 critic 입력으로 사용	critic 의 추측보다 도구 증거 우선
LATS/Tree Search	여러 후보 계획 탐색	아키텍처/월드 배치/수리 후보를 최대 N 개 생성하고 cheap tests 로 가지치기	런타임 비용 상한과 diversity 조건
Agentless/SWE-agent ACI	단순 고정 파이프라인과 좋은 인터페이스	localize -> patch -> validate 를 기본으로, 파일 편집은 diff 단위	자율 계획이 항상 낫다는 가정 금지

연구/계열	핵심 아이디어	본 시스템의 구현 규칙	실패 방지
AgentForge	Planner/Coder/Tester/Debugger/ Critic + sandbox	모든 변경은 격리 실행을 통과해야 다음 단계로 전파	실행 없는 코드 리뷰 금지
SGAgent	locate-suggest-fix 분리	오류 위치 탐색과 해결책 제안과 패치 생성을 서로 다른 산출물로 저장	한 번에 전체 저장소 재작성 금지
TraceDev	요구-설계-코드-테스트 추적 그래프	각 acceptance criterion 에 구현 파일과 테스트를 연결하고 누락 탐지	추적 링크 없는 기능은 미완성
FailureMem	구조화된 workflow + 능동 시각 도구 + 실패 기억	GUI/텍스처 오류를 영역 단위로 지정하고 과거 실패 패턴 검색	전체 이미지 재생성보다 국소 수정
TraceRepair	실행 trace 로 수리	예외 stack, 호출 trace, state snapshot 을 patch context 에 포함	로그 요약만 주고 원문을 버리지 않음
ToolMaker/code2MCP	코드를 typed tool 로 변환	새 도구는 자동 생성 가능하나 unit/property tests 와 권한 manifest 통과 후 admission	에이전트가 만든 도구를 즉시 production 등록 금지
Self-verification limitations	LLM 자기검증의 한계	generator와 judge 를 분리하고, 가능한 경우 다른 모델/규칙/실행기로 검증	같은 답을 반복해 신뢰도라고 착각 금지
Dialogue SWE-Bench	사용자와의 요구 대화가 별도 능력	계획 카드, 모호성 표시, 수정 diff, 승인 상태를 구조화	질문 없이 임의 사양 확정 금지
Secure agent surveys/ABAC	신뢰 경계·최소권한·provenance	도구별 동적 scope, 데이터 출처, 권한 결정 로그를 저장	에이전트를 단일 신뢰 주체로 취급 금지

24.1 권장 에이전트 구성

역할	책임	허용 도구	합격 조건
Requirements Analyst	사용자 요청을 기능/비기능/제외 범위와 모호성으로 분해	RAG read, Project IR proposal	사용자에게 보일 Proposal vN 생성
Architect/Router	맵/데이터팩/Paper/Fabric/NeoForge/리소스팩 분배와 DAG 작성	source/version read, project write	각 기능에 구현 경로와 대안·이유 존재
Implementer	최소 패치, Datagen provider, 예셋 spec 생성	source read, staged filesystem write	compile/static precheck
Asset Worker	texture/model/motion/audio 생성	asset MCP 만	규격/스타일/provenance 검사
Test Designer	acceptance -> unit/GameTest/playtest/property test 변환	read + test file staged write	baseline 에서 실패, 수정 후 통과
Executor	build/server/client/test 를 결정론적으로 실행	build/runtime tools	evidence bundle 생성
Debugger	실패 분류와 최소 패치 후보 제안	logs/source/failure RAG + staged patch	동일 root cause 최대 3 회
Independent Critic	요구사항 누락, 과적합 테스트, 보안·성능 검토	read-only quality tools	증거 기반 veto 가능
Release Governor	SBOM/서명/승인/배포/재투입	release tools	모든 gate 와 사용자 승인 충족

24.2 반복 수리 알고리즘

```

for requirement in approved_requirements:
    assert trace_graph.has_acceptance_test(requirement)

baseline = run_full_baseline()
for task in topological_order(work_dag):
    candidate = implement_minimal_patch(task)
    static = run_static_and_schema_checks(candidate)
    if not static.pass: repair_or_block(static)
    
```

```
targeted = run_targeted_tests(candidate)
if not targeted.pass:
    diagnosis = independent_localize_suggest(targeted.evidence)
    for attempt in range(3):
        patch = generate_minimal_patch(diagnosis, failure_memory)
        if run_targeted_tests(patch).pass: break
    else: block_for_user_review()

regression = run_affected_then_full_regression()
critic = independent_critic(requirements, diff, tests, evidence)
if not (regression.pass and critic.pass): block()

release_only_if(all_gates_pass and user_release_approval)
```

- 수리 전에 baseline 을 저장하여 기존 실패와 새 회귀를 구분한다.
- 테스트 파일도 에이전트가 수정할 수 있지만, acceptance oracle 을 약화하는 변경은 별도 승인과 mutation test 가 필요하다.
- 동일 원인 3 회 실패, 전체 diff 가 계획 대비 20% 이상 팽창, API 버전 변경 제안, 데이터 손실 가능성이 있으면 자동 수리를 중단한다.
- 수리 후보는 최소 2 개를 만들 수 있으나, 먼저 정적/단위 테스트가 실패 후보를 평가하고 실제 게임 테스트는 상위 후보에만 수행한다.

25. 완전형 RAG: 버전/코드/트렌드/실패/보안 지식 계층

RAG 는 하나의 벡터 DB 가 아니다. Minecraft 는 버전, 로더, 맵핑, 실행 측(client/server), 리소스 형식이 강하게 결합되므로 지식을 서로 다른 인덱스로 격리하고 메타데이터 필터를 먼저 적용해야 한다. 검색 결과를 모델에 넣기 전에 신뢰 등급과 유효 기간을 계산한다.

인덱스	내용	필수 metadata	주 검색
OfficialSpec	MCP/Fabric/NeoForge/Paper/Bedrock/GeckoLib 공식 문서	source, crawl_time, version, loader, language, hash	BM25 + dense
VersionMatrix	MC/Java/loader/API/dependency 호환 조합과 마이그레이션	valid_from/to, exact version, confidence	exact lookup
SourceSymbol	디컴파일 source, mappings, class/method/field, call graph	jar hash, mapping namespace, side, symbol	symbol + graph
ResourceSchema	vanilla assets/data, JSON schema, folder conventions	pack_format, MC version, namespace	exact path + BM25
Ecosystem	Modrinth/CurseForge/GitHub 프로젝트/릴리스/이슈	observed_at, loader, versions, license, popularity signals	filtered search
TrendTimeSeries	다운로드/업데이트/이슈/키워드의 시간 구간 집계	window, source, region if lawful, sampling method	time-series + keyword
FailureMemory	빌드/런타임/시각/성능/보안 실패와 검증된 수정	fingerprint, version, diff, tests, outcome	hybrid + exact error
ProjectMemory	승인 계획, 결정, IDs, artifacts, tests, user feedback	project_id, plan_version, artifact hash	exact + graph
AssetStyle	palette, references, approved textures/models/audio, style rules	license, provenance, resolution, style_id	image/text multimodal
SecurityIntel	OSV/GHSA/CVE, loader advisory, malicious mod/MCP patterns	published/modified, package, severity, trust	exact package + semantic
LicenseProvenance	SPDX, model card, asset licenses, attribution obligations	source URI, license, hash, allowed_use	exact
Evaluation	RAG queries, expected evidence, failures, user corrections	dataset version, difficulty, outcome	benchmark

25.1 수집·정규화·색인 파이프라인

54. 소스 등록: 공식/1 차/검증된 커뮤니티/비신뢰의 trust tier 와 허용 목적을 지정한다.

55. 수집: ETag/Last-Modified/commit hash 를 저장하고 변경된 문서만 재수집한다.
56. 파싱: 제목, 코드 블록, 표, 버전 경고, API symbol, URL anchor 를 보존한다.
57. 정규화: Minecraft 새 캐린더 버전 체계와 구형 버전을 별도 normalized fields 로 저장한다. [R141]
58. 청킹: 문서는 heading+code 단위, 소스는 symbol 단위, 로그는 stack/root-cause 단위로 자른다.
59. 색인: BM25, dense embedding, ColBERT 류 late interaction, symbol graph 를 목적별로 사용한다.
60. 검증: 링크/해시/버전/코드 컴파일 sample 을 검사하고 실패한 chunk 를 격리한다.
61. 평가: 골든 질의에서 context precision/recall, answer groundedness, source freshness 와 wrong-version rate 를 측정한다.

25.2 질의 라우팅과 검색 순서

```
query -> intent classifier
  API signature / mapping? -> SourceSymbol + OfficialSpec
  JSON path / pack format? -> ResourceSchema + OfficialSpec
  build error? -> exact error search + FailureMemory + SourceSymbol
  design / similar mods? -> Ecosystem + TrendTimeSeries + user constraints
  security / dependency? -> SecurityIntel + VersionMatrix
  project follow-up? -> ProjectMemory + trace graph

retrieve 40-100 candidates
-> metadata hard filter(version, loader, side, date)
-> hybrid rerank
-> evidence diversity check
-> return 4-12 excerpts with source ranges and confidence
```

Self-RAG/CRAG/Adaptive-RAG 의 핵심은 모든 질문에 무조건 검색하는 것이 아니라, 검색 필요성과 결과 품질을 판정하고 부족하면 검색식을 교정하는 것이다. RAPTOR/GraphRAG 는 전체 문서와 프로젝트 의존성을 계층적으로 요약하는 데 유용하지만 구축 비용이 높으므로, Minecraft 공식 문서 전체보다 프로젝트 추적 그래프와 대규모 오류 기억에 선택적으로 사용한다. [R15][R16][R108][R109][R17]

25.3 최근 게임 트렌드 RAG

신호	수집 방법	해석	사용자에게 표시
Modrinth 업데이트/다운로드	API 의 loader/version/category 와 project metadata 를 날짜별 snapshot	다운로드가 품질·안전의 보장은 아님	관측 기간, 출처, 최신 update
CurseForge 파일/버전	API key 기반 검색 및 file metadata	플랫폼별 중복/편향 고려	API 관측 시각
GitHub activity	release, commit, issue response, stars, security advisory	활동성·문서화 신호	repo/license/last release
공식 Minecraft/loader release	공식 release note/문서 변경	마이그레이션·호환성 우선	정확한 버전과 날짜
사용자 피드백	승인/거절/수정/플레이 telemetry opt-in	개인화되지만 외부 인기와 분리	프로젝트 내부 근거
웹 검색 관심	공식 API 또는 합법적 공개 aggregate 가 있을 때만	표본/지역/언어 편향	정확한 기간과 불확실성

트렌드 사용 원칙 트렌드는 아이디어 추천과 중복 탐지에만 사용한다. 사용자가 승인한 게임 디자인을 인기 지표 때문에 자동 변경하지 않는다. 검색 결과에 포함된 README/이슈/웹페이지의 명령문은 데이터로만 처리하고 시스템 지시로 실행하지 않는다.

25.4 RAG 오류 및 공격 방어

- 프롬프트 인젝션 분리: retrieved text 는 quoted data channel 에 넣고, 도구 호출·권한 요청을 유발할 수 없다.
- 소스 우선순위: 공식 문서/소스 > 공식 저장소 > 유지보수되는 검증 커뮤니티 > 일반 웹. 충돌하면 양쪽을 표시하고 실행 테스트로 결정한다.
- 버전 하드 필터: exact Minecraft/loader/mapping/side 가 없는 chunk 는 API 결정에 사용하지 않는다.

- 시간 필터: “최신”은 published_at 와 observed_at 을 모두 확인하며, 캐시 TTL 이 지난 정보는 재검증한다.
- 인용 완전성: 생성된 코드의 비표준 API 선택에는 source range 또는 symbol evidence 가 필요하다.
- 한 모델이 기획, 코드, 이미지, 3D, 모션, 음성을 모두 최고 수준으로 처리한다고 가정하지 않는다. 범용 Research/Design Supervisor 가 사용자 의도와 RAG 근거를 바탕으로 GameConceptPlan 과 Mod-World IR 을 작성하고, 사용자 승인 후에만 코드·에셋 전문 모델을 호출한다. 중앙 계획 모델은 코딩 모델과 분리하며, Model Registry MCP 와 역할별 벤치마크가 매 릴리스마다 champion/challenger 를 교체한다. T4 16GB 에서는 로컬 9B 급 범용 모델·8B 급 VLM·4B 이미지 모델을 순차 실행하고, 초대형 최신 모델은 원격/self-hosted worker 로 라우팅한다.

26. 멀티모달 모델 라우팅과 T4 GPU 임대 구조

한 모델이 코드, 이미지, 3D, 모션, 음성을 동시에 최고 수준으로 처리한다고 가정하지 않는다. 중앙 계획 모델이 작업 명세를 만들고, 각 산출물 형식에 맞는 모델·결정론적 도구·검증기를 순차 호출한다. T4 16GB 에서는 GPU 프로세스를 동시에 상주시킬 수 없으므로 GPU Lease Manager 가 모델을 로드/언로드하고 CPU 작업과 겹쳐 실행한다.

역할	후보	출력	배치
게임 기획·연구	GLM-5.2 / Kimi K3/K2.6 / 최신 benchmark champion	GameConceptPlan, evidence graph, Mod-World IR	원격/self-hosted 기본
T4 로컬 범용 fallback	Qwen3.5-9B 4bit 또는 Gemma 4 12B QAT	intent, schema, 계획 수정, 간단 patch	T4 단독 lease
코드·저장소 수리	Kimi K2.7-Code / Qwen3-Coder-Next / GLM-5.2	Java/JSON/Gradle/patch/test	원격; T4 는 fallback
시각 이해	Qwen3-VL-8B-Instruct	스크린샷, UV, GUI, missing texture, map defects	T4 4bit 순차
RAG retrieval	Qwen3-Embedding-0.6B + Reranker-0.6B	dense candidates + relevance score	CPU/GPU
이미지/텍스처	FLUX.2-klein-4B + ComfyUI	아이템/블록/GUI/particle concept	T4 단독; pixel compiler 후처리
3D 초안	Hunyuan3D-2.1 / TripoSR	reference mesh/turntable	원격 우선
모션	Wan2.2 Animate + deterministic keyframe solver	motion reference + animation IR	원격/선택
음성 인식	SenseVoice / Fun-ASR	한국어 요구, 명령, 감정/이벤트	CPU/GPU
TTS	CosyVoice 3.0	NPC/안내 음성	CPU 또는 짧은 GPU
SFX	ControlFoley/AudioLDM 계열 + 편집 파이프라인	효과음/ambient 초안	선택; 실행 검증
독립 심사	주 생성 모델과 다른 family + 규칙 검증기	계획/코드/시각 합격 판정	원격/T4 순차

26.1 T4 GPU 임대 스케줄

단계	GPU 상태	CPU 병렬 작업	메모리 원칙
Plan/Code	7B 4bit 로드	RAG, Git, version resolution	context 8K-16K; 긴 repo 는 retrieval
Build/Test	LLM unload 또는 CPU offload	Gradle, server, Mineflayer	GPU 비움; Java RAM 3-6GB
Texture	diffusion workflow 로드	JSON/Datagen/static tests	batch 1, 512 이하, 즉시 postprocess
VLM Review	diffusion unload, 3B VLM load	image metrics/diff	고정 scene crop 우선
3D Optional	TripoSR load	mesh simplify/UV CPU	한 자산씩
Repair	3D/VLM unload, code LLM 재로드	failure fingerprint/RAG	원인 관련 context 만

단계	GPU 상태	CPU 병렬 작업	메모리 원칙
Release	GPU 없음	SBOM/sign/provenance/package	재현 가능한 CPU pipeline

26.2 이미지·3D·모션 자동화의 현실적 경계

- 이미지 생성 모델은 컨셉 초안을 만들고, 실제 Minecraft PNG 는 nearest-neighbor 축소, palette quantization, alpha cleanup, edge/seam 검사로 변환한다.
- 3D 생성 모델의 mesh 를 그대로 엔티티 모델로 쓰지 않는다. voxel/cuboid decomposition, bone graph, UV density, cube count, hitbox, animation pivot 를 Blockbench 형식으로 재컴파일한다.
- MotionGPT 류 모션은 Minecraft bone naming 과 공격 타이밍을 모르므로 retarget 후 발/무기 관통, root motion, damage tick alignment 를 GameTest 와 화면 분석으로 확인한다.
- 멀티모달 생성 결과에도 prompt, seed, model revision, workflow hash, source image license 를 provenance 로 기록한다.

27. 사용자 승인부터 릴리스 재투입까지의 전체 실행 흐름

27.1 첫 요청과 계획 승인

- 사용자 입력을 자연어·음성·이미지·기존 모드/JAR/소스 중 하나 이상으로 수집한다.
- Requirements Analyst 가 명확한 요구, 모호한 요구, 제외 범위, 위험 권한, 라이선스 조건을 분리한다.
- RAG 가 정확한 대상 버전, 유사 모드, API 가능성, 최근 트렌드, 예상 위험을 조회한다.
- Architect 가 Proposal v1 을 생성한다: 기능 목록, 아티팩트 유형, 콘텐츠 수, MCP/모델 호출, 예상 테스트, 자원, 대안.
- 사용자에게 plan diff 와 선택지를 보여준다: 수정, 범위 축소, 대안 선택, 승인, 거절, 분기 저장.
- 승인 전에는 외부 publish, 대량 파일 쓰기, GPU 고비용 생성, 음성 업로드를 수행하지 않는다.

```
{
  "proposal_version": 3,
  "status": "awaiting_user_approval",
  "scope": {...},
  "assumptions": [...],
  "alternatives": [...],
  "tool_plan": [{"group": "minecraft_source", "reason": "..."}],
  "asset_plan": [{"type": "entity_model", "route": "blockbench+geckolib"}],
  "acceptance_tests": [...],
  "risk_approvals": [...],
  "estimated_resources": {...},
  "user_actions": ["approve", "patch", "branch", "reject"]
}
```

27.2 생성과 통합

- 승인 버전에서 immutable specification snapshot 을 만든다.
- ID registry 와 dependency graph 를 확정하고 코드 skeleton, Datagen, 테스트 skeleton 을 먼저 생성한다.
- API 불확실성은 source MCP 에서 exact symbol 과 버전 근거를 조회한 뒤 구현한다.
- 에셋은 style bible 과 UV/layout spec 을 받아 병렬 생성하되, 공통 ID 와 manifest 만 통해 연결한다.
- 각 작업자는 staging branch/worktree 에서 파일을 쓰고 patch manifest 와 evidence 를 반환한다.
- 통합 작업자가 충돌, 누락 참조, client/server side, language/resource 경로를 검사한다.

27.3 완성 모드 재투입과 지속 개선

최종 JAR 만 다시 넣는 것이 아니라 source snapshot, generated resources, original art files, build manifest, SBOM, test evidence, screenshots, performance profile, user feedback 를 하나의 Release Bundle 로 보존한다. 다음 프롬프트는 이 bundle 을 기준으로 incremental change plan 을 만든다.

후속 사용자 요청	재투입 분석	자동 제안	필수 재검증
보안점 말해줘	dependency/SBOM, permission, network/file, packet/NBT, secret, update path	위험 등급과 최소 패치	정적+fuzz+GameTest+회귀
보안 수정해줘	취약 경로 trace, 영향을 받는 파일/세이브 포맷	호환 patch 와 migration	exploit regression + compatibility
규모 키워줘	registry/ID 여유, save schema, chunk/tick 비용, network protocol	모듈 분할, data-driven content, async 작업	성능 budget + 대규모 월드
아이템 50 개 추가	manifest template 과 palette/style 재사용	Datagen batch + variation rules	중복 ID/recipe/tag/visual grid
새 버전으로 포팅	version matrix, removed APIs, mappings, pack format	migration DAG 와 위험 목록	clean build + save/world compatibility
음성 명령 추가	privacy/latency/false activation, client/server path	push-to-talk, confidence threshold, command confirmation	오인식/권한/네트워크 테스트
밸런스 수정	telemetry opt-in, config, combat trace	config-only patch 우선	simulation + playtest + save compatibility

28. 오류율을 낮추는 검증 피라미드와 반복 수리 알고리즘

모드 제작 오류는 한 종류가 아니다. 요구 누락, 잘못된 API, 빌드 실패, 리소스 참조, 서버/클라이언트 side, 게임 논리, 시각, 성능, 호환성, 보안, 라이선스를 각각 다른 oracle 로 검증한다.

게이트	실행 시점	검사	실패 시
G0 요구 추적	계획 승인 전/후	모든 requirement -> design -> files -> tests 링크	계획 보완
G1 스키마/정적	파일 생성 즉시	JSON schema, path, ID, side, compiler lint	국소 수정
G2 빌드	작업 단위	clean compile/remap/datagen	error fingerprint 수리
G3 단위/속성	핵심 로직	JUnit, property, serialization roundtrip	logic patch
G4 GameTest	블록/엔티티/월드 상호작용	tick 기반 상태, structure, inventory	게임 내 patch
G5 서버 통합	전체 모드	boot, registry, command, persistence, restart	통합 patch
G6 Mineflayer	플레이어 시나리오	경로, 상호작용, 전투, progression	design/logic patch
G7 시각/오디오	고정 장면	missing texture, UV, clipping, GUI, animation, sound	asset 국소 수정
G8 성능	기능 완료 후	MSPT/TPS, heap, GC, chunk, network, load time	profile-driven patch
G9 보안	릴리스 후보	SAST, deps, fuzz, permission, injection, secrets	release block
G10 호환/회귀	릴리스	clean install, matrix, old saves/world, full suite	version-specific fix
G11 테스트 강도	핵심 모듈	mutation score, test oracle weakening detection	테스트 보강
G12 릴리스 증명	최종	SBOM, signature, provenance, hashes	publish 금지

28.1 오류별 자동 수리 정책

오류	자동 수리 경로	금지
API/매핑 오류	exact symbol lookup -> source range -> 최소	버전/loader 변경 금지

오류	자동 수리 경로	금지
	import/signature patch	
Gradle/의존성	lockfile, repository, checksum, version matrix 검사	무작정 최신 버전 업그레이드 금지
JSON/리소스	reference graph 에서 끊긴 edge 를 찾아 provider/manifest 수정	파일 복제식 임시 우회 금지
Client/server side	class load trace 와 environment annotation 검사	서버에 client class import 금지
Game logic	실패한 acceptance 의 state trace 를 비교	테스트를 통과시키기 위해 조건 삭제 금지
시각	VLM bounding region + pixel/UV metrics -> 해당 asset 만 재생성/수정	전체 스타일셋 재생성 금지
성능	profiler hotspot + load size + frequency -> 알고리즘/캐시/스케줄 수정	기능 제거로만 해결 금지
보안	source-to-sink trace, exploit test, permission reduction	경고 숨기기 금지
호환성	migration adapter, version-specific source set, data fixer	세이브 데이터를 조용히 폐기 금지

28.2 오류율 측정 지표

지표	정의	초기 MVP 목표	성숙 목표
Plan completeness	승인 requirement 중 trace/test 가 있는 비율	>=95%	100%
Wrong-version API rate	API 오류 중 버전/매핑 불일치 비율	<5%	<1%
First clean build	첫 생성 후 clean build 성공률	>=60%	>=85%
Repair success@3	동일 root cause 3 회 내 해결 비율	>=80%	>=95%
Regression escape	릴리스 후 발견된 기존 기능 회귀/릴리스	<5%	<1%
Resource integrity	missing model/texture/lang/sound/tag 0 여부	100%	100%
Visual acceptance	고정 scene 중 자동+사용자 승인 비율	>=90%	>=98%
Mutation score	핵심 도메인 로직	>=55%	>=75%
Security critical	미해결 critical/high 릴리스	0	0
Provenance coverage	파일/모델/에셋 source+hash+license 비율	>=95%	100%

29. MCP/에이전트/RAG 공급망 보안

MCP 의 도구 설명과 Roots 는 보안 경계가 아니다. 실제 경계는 브로커 정책, 컨테이너, OS 사용자, read-only mount, egress firewall, secret broker, 서명된 서버 패키지와 감사 로그다. MCP 클라이언트의 prompt injection 과 overprivileged tool 선택이 연구에서 반복적으로 관찰되므로, 최소 권한을 모델 판단에 맡기지 않는다. [R62][R135][R136]

위협	통제	검증
악성/변조 MCP 서버	allowlist registry, pinned digest, code/license audit, attestation, sandbox	startup self-test + hash verify
Prompt injection in retrieved docs	data/instruction channel 분리, untrusted label, tool calls blocked from retrieval	injection benchmark
Confused deputy/OAuth mix-up	audience/resource binding, state/PKCE/DPoP, broker-mediated OAuth	authorization integration tests
Overprivileged tool	ABAC: principal+project+phase+approval+resource; default deny	policy unit tests

위협	통제	검증
Path traversal/파일 유출	container mount, canonical path, symlink rejection, size limits	fuzz/path tests
Shell injection	allowlisted commands+typed args; no shell=true; escaped env	malformed argument tests
Dependency confusion	repository allowlist, lock, checksum, dependency verification	clean isolated build
Malicious mod/JAR	jar inspect, signature, network/file API scan, sandbox runtime	behavior monitor
Secret leakage	short-lived token, no prompt/log exposure, redaction, rotation	secret canary scan
RAG poisoning	source trust tier, hash/history, anomaly detection, cross-source verify	poisoned corpus test
Asset/license contamination	provenance manifest, model card/license gate, similarity review where required	release license audit
Build tampering	hermetic builder, SBOM, SLSA provenance, Sigstore signature	independent verify
Unsafe autonomous publish	two-person/user approval, staged release, canary, rollback	release policy test

29.1 ABAC 정책 예

```

permit(tool="patch.apply") when
  principal.role in ["implementer", "debugger"] and
  project.id == request.project_id and
  phase in ["implementation", "repair"] and
  path within project.staging_root and
  diff.changed_files <= 40 and
  not touches(release_keys, secrets, external_mounts)

require_approval(tool in ["release.publish", "canary.deploy", "server.command"])
deny(network != "deny") unless capability_token includes exact host+method+expiry

```

29.2 공급망 릴리스 산출물

- CycloneDX SBOM(JSON), purl, version, dependency edges, hashes. SBOM 에 edge 가 없으면 “안전”이 아니라 “경로 불명”으로 표시한다.
- 소스 commit, builder image digest, Gradle lock/checksum, 생성 모델 revision/workflow hash, 모든 입력 artifact hash 를 포함한 provenance.
- JAR/ZIP/SBOM/provenance 에 Sigstore 또는 조직 키 서명과 검증 명령을 함께 제공한다.
- 라이선스/attribution 파일과 외부 에셋·모델별 사용 조건을 최종 패키지에 포함한다.
- 릴리스 후 JAR 을 다시 분석해 source build 결과와 hash 가 일치하는지 확인한다.

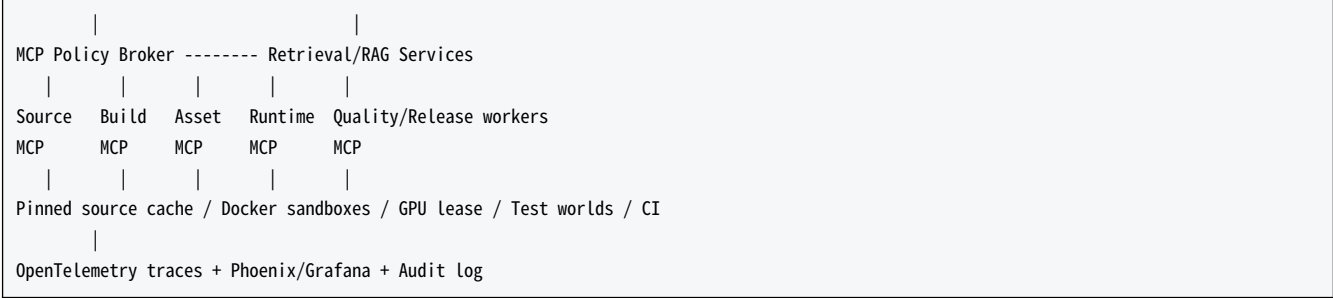
30. 실제 구현 서비스 토폴로지와 저장소 구조

30.1 서비스 토폴로지

```

Web UI / Voice UI
|
API Gateway + Approval Service
|
Orchestrator / DAG Engine ---- Project IR DB ---- Object Store

```



30.2 권장 모노레포

```
minecraft-mod-ai/
  apps/
    web-ui/
    orchestrator/
    approval-service/
  services/
    mcp-broker/
    project-ir/
    rag-ingest/
    rag-query/
    task-runner/
    gpu-lease/
  mcp-servers/
    minecraft-source/
    build-fabric/
    build-paper/
    build-neoforge/
    world/
    asset-comfy/
    asset-blockbench/
    playtest/
    quality/
    release/
  schemas/
    project-ir/
    tool-inputs/
    tool-results/
    evidence/
  templates/
    fabric/<version>/
    paper/<version>/
    neoforge/<version>/
  policies/
    abac/
    network/
    licenses/
  evals/
    tool-selection/
    rag/
    mod-generation/
    security/
  fixtures/
    sample-mods/
    test-worlds/
  deploy/
    docker/
    k8s-optional/
  docs/
```

30.3 핵심 데이터 계약

객체	핵심 필드	불변 조건
Project	project_id, owner, target matrix, policy profile	target 변경은 새 plan version
Proposal	version, requirements, assumptions, alternatives, tools, tests, risks	승인 후 immutable
Requirement	id, statement, priority, source, acceptance	최소 1 개 acceptance
Artifact	uri, type, hash, producer, inputs, license	hash 와 provenance 필수
TraceEdge	from, to, relation, evidence	요구-파일-테스트 연결
ToolCall	server, tool, input hash, auth, result, evidence	재현 가능한 기록
Failure	fingerprint, phase, evidence, root cause, attempts	추측과 확정 분리
Release	source revision, artifacts, SBOM, provenance, signature, gates	모든 gate pass
Feedback	user, release, context, requested change, severity	다음 proposal 근거

31. 단계별 개발 일정, 자원, 합격 기준

단계	범위	핵심 산출물	합격 기준
0. 기반 2-3 주	Project IR, approval UI, broker, tracing, filesystem/git/build sandbox	schema, policy, audit trail	승인 전 write/publish 0; 모든 tool call trace
1. Paper MVP 4-6 주	Paper plugin + datapack + resource pack + server harness	자연어 -> proposal -> JAR	샘플 20 개 중 clean build>=80%, acceptance>=90%
2. Fabric 콘텐츠 6-10 주	item/block/entity/datagen/GameTest/source MCP	Fabric JAR+assets	리소스 무결성 100%, GameTest pass
3. 맵/플레이 4-8 주	GDPC/WorldEdit/Mineflayer/경로/월드 테스트	map+plugin/mod 통합	주요 시나리오 reachability/pass
4. 멀티모달 6-10 주	ComfyUI, Blockbench, VLM, audio	원본+export+provenance	고정 장면 시각 승인>=90%
5. 보안/릴리스 4-6 주	SAST/deps/fuzz/SBOM/sign/provenance	검증 가능한 release bundle	critical/high 0, signature verify
6. 지속 개선	reimport, migration, scale, failure memory, trends	incremental proposal/patch	기존 회귀<1%, repair@3>=95%

T4 범위 T4 16GB 한 장으로 1-4 단계의 MVP 는 순차 로딩 방식으로 가능하다. 다만 고품질 3D, 대형 이미지 모델, Qwen3-Coder-Next 같은 대형 총 파라미터 모델, 다중 클라이언트 동시 시각 테스트는 원격/API 또는 별도 GPU 노드가 현실적이다.

31.1 첫 구현에서 반드시 고정할 범위

- Minecraft 정확한 한 버전 + Java 버전 하나 + Fabric 또는 Paper 하나를 먼저 고정한다.
- 아이템/블록/레시피/단순 엔티티/GUI/구조물의 제한된 archetype 만 지원한다.
- 모든 JSON 은 Datagen 또는 버전별 schema generator 로 만들고 LLM 자유형 파일 생성을 줄인다.
- 런타임은 테스트 서버/월드만 조작하며 사용자의 실제 월드에는 자동 쓰지 않는다.
- 텍스처는 16x16/32x32, 모델은 Blockbench cuboid, 애니메이션은 제한된 GeckoLib controller 로 시작한다.
- 한 릴리스에 자동 수리 총 20 회, 동일 원인 3 회, GPU 시간/토큰/빌드 시간 예산을 둔다.

32. 벤치마크, 관측성, 운영 지표와 중단 기준

32.1 내부 벤치마크 구성

세트	예시	측정
Tool Selection	동일 요구를 5/10/20/40 개 도구 노출에서 라우팅	정확 tool group, 불필요 호출, 권한 위반
Version API	동일 기능을 여러 MC/Fabric/NeoForge 버전에서 구현	wrong-version rate, compile
Resource Graph	누락 texture/model/lang/tag/sound/loot 를 인위 삽입	탐지 recall/precision
Repair	실제 compile/runtime/visual/security 결함 fixture	localization, patch success@1/@3, regression
RAG	공식/구형/충돌 문서 혼합 질의	source freshness, exact version, citation
Visual	missing texture, UV seam, z-fight, GUI clip, bone penetration	detection + localization IoU
Playtest	craft/progression/boss/door/path/persistence 시나리오	completion, timeout, false pass
Security	prompt injection, path traversal, shell args, malicious dependency/MCP	blocked rate, false positive
User Dialogue	모호한 요구/계획 수정/분기/철회	correct clarification, plan diff fidelity
Scale	10/50/200 콘텐츠 manifest	build time, memory, ID/reference integrity

32.2 필수 관측 span

- 사용자 입력 -> proposal generation -> user edits -> approval event
- RAG query -> filters -> retrieved chunks -> rerank -> selected evidence
- agent prompt/model/revision -> tool discovery -> tool call -> result/evidence
- patch diff -> build -> tests -> runtime -> visual/performance/security gates
- GPU lease load/unload, peak VRAM, model latency, queue wait
- release artifacts -> SBOM/provenance/signature -> publish/reimport

32.3 자동 중단과 사람 검토 조건

조건	이유	행동
동일 root cause 3 회 실패	무한 수리/비용 폭증	blocked report + 2 개 대안
계획 밖 파일 20 개 이상 또는 핵심 API/버전 변경	scope drift	새 Proposal 승인
테스트를 삭제/완화하거나 baseline 실패를 숨기는 patch	false pass 위험	critic veto
높음 이상 취약점 또는 unknown license	릴리스 위험	publish block
월드/세이브 migration 이 비가역	데이터 손실	백업+사용자 승인
외부 네트워크/파일/마이크 권한 확대	개인정보/공급망	권한 승인 UI
성능 budget 2 배 초과	운영 불가	profile report + scope 선택
RAG 근거가 충돌하거나 정확 버전 근거 없음	API 환각 위험	source test/사용자 선택

33. 최종 권고안과 구현 우선순위

실제 성공 가능성이 가장 높은 구조는 “대형 자유 멀티에이전트”가 아니라 승인형 Project IR, 제한된 MCP 도구 그룹, 공식 소스 RAG, 결정론적 생성/검증, 실행 기반 수리, 독립 critic, 보안 브로커를 결합한 하이브리드 시스템이다.

74. 가장 먼저 Project IR/Approval/Broker/Trace 를 만든다. 이 기반 없이 모델과 MCP 를 많이 연결하면 오류와 공격면만 증가한다.

75. 1차 제품은 Paper 또는 Fabric 한쪽으로 고정하고, 공식 Datagen 과 GameTest 를 중심으로 한다.
76. MCDxAI Minecraft Dev MCP 는 source/mapping 조회에, Mineflayer 는 플레이 검증에, ComfyUI 는 에셋 workflow 에 각각 제한적으로 사용한다.
77. MCreatorMCP 와 runtime Groovy/Blender MCP 같은 강력한 커뮤니티 도구는 격리된 실험 worker 로만 두고, 직접 production 권한을 주지 않는다.
78. RAG 는 버전·소스·실패·트렌드·보안·프로젝트 기억을 분리하고, 정확한 버전 필터와 근거 범위를 합격 조건으로 만든다.
79. 에이전트 연구의 핵심은 “생각을 더 오래”가 아니라 역할 분리, 실행 근거, traceability, 실패 기억, 최소 패치, 독립 검증과 중단 기준 이다.
80. PROFILE F 는 고품질 3D·대형 멀티모달·장문 저장소 작업을 원격 worker 에 선택적으로 보낸다. PROFILE L 은 외부 AI 추론을 호출하지 않으며, 4B-9B 급 로컬 모델을 순차 실행하고 3D 는 Blockbench 템플릿·절차적 voxel/model compiler 를 기본으로 사용한다. 로컬 생성 모델이 입장시험을 통과하지 못하면 기능을 제거하지 않고 결정론적 제작 경로로 전환한다.
81. 최종 모드는 release bundle 로 재투입해 보안, 성능, 확장, 포팅을 이어가되 매 변경마다 새로운 계획 승인과 전체 회귀 게이트를 거친다.

최종 결론 현재 공개된 MCP/API/오픈소스와 연구를 조합하면 구현 가능하다. 완성도를 결정하는 것은 모델 수가 아니라, 정확한 도구 계약·버전 잠금·사용자 승인·실행 검증·권한 격리·추적 가능한 실패 수리 체계다.

부록 C. 핵심 MCP JSON Schema 예시

C.1 project.approve_plan

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "additionalProperties": false,
  "required": ["project_id", "proposal_version", "proposal_hash", "decision"],
  "properties": {
    "project_id": {"type": "string", "pattern": "^a-z0-9_\\-\\{3,64}$"},
    "proposal_version": {"type": "integer", "minimum": 1},
    "proposal_hash": {"type": "string", "pattern": "^sha256:[0-9a-f]{64}$"},
    "decision": {"const": "approve"},
    "approved_risks": {"type": "array", "maxItems": 20,
      "items": {"type": "string"}},
    "user_note": {"type": "string", "maxLength": 2000}
  }
}
```

C.2 source.search_symbol

```
{
  "type": "object", "additionalProperties": false,
  "required": ["target", "query"],
  "properties": {
    "target": {
      "type": "object", "additionalProperties": false,
      "required": ["minecraft_version", "loader", "mappings", "side"],
      "properties": {
        "minecraft_version": {"type": "string"},
        "loader": {"enum": ["fabric", "neoforge", "paper", "vanilla"]},
        "mappings": {"enum": ["yarn", "mojmap", "intermediary"]},
        "side": {"enum": ["client", "server", "common"]}
      }
    },
    "query": {"type": "string", "minLength": 2, "maxLength": 300},
    "kind": {"enum": ["class", "method", "field", "text", "callers", "calles"]},
    "limit": {"type": "integer", "minimum": 1, "maximum": 50}
  }
}
```

C.3 gradle.run_allowlisted

```
{
  "type": "object", "additionalProperties": false,
  "required": ["project_revision", "tasks"],
  "properties": {
    "project_revision": {"type": "string", "pattern": "^sha256:"},
    "tasks": {"type": "array", "minItems": 1, "maxItems": 8,
      "items": {"enum": ["clean", "build", "test", "runDatagen", "runGameTestServer", "check"]}},
    "offline": {"type": "boolean", "default": true},
    "timeout_s": {"type": "integer", "minimum": 30, "maximum": 1800},
    "max_output_mb": {"type": "integer", "minimum": 1, "maximum": 100}
  }
}
```

C.4 comfy.queue_prompt

```
{
  "type": "object", "additionalProperties": false,
  "required": ["workflow_hash", "asset_spec", "seed"],
  "properties": {
    "workflow_hash": {"type": "string", "pattern": "^sha256:"},
    "asset_spec": {"type": "object"},
    "seed": {"type": "integer", "minimum": 0},
    "max_width": {"type": "integer", "maximum": 768},
    "max_height": {"type": "integer", "maximum": 768},
    "batch": {"type": "integer", "minimum": 1, "maximum": 2},
    "gpu_lease_id": {"type": "string"}
  }
}
```

C.5 bot.run_scenario

```
{
  "type": "object", "additionalProperties": false,
  "required": ["server_instance", "scenario", "assertions"],
  "properties": {
    "server_instance": {"type": "string"},
    "scenario": {"type": "array", "maxItems": 100,
      "items": {"type": "object", "required": ["action"],
        "properties": {"action": {"enum": ["move", "find", "interact", "attack", "wait", "chat_safe", "inventory"]}}}},
    "assertions": {"type": "array", "maxItems": 50},
    "timeout_s": {"type": "integer", "maximum": 600},
    "capture": {"enum": ["on_failure", "checkpoints", "all"]}
  }
}
```

C.6 quality.aggregate

```
{
  "type": "object", "additionalProperties": false,
  "required": ["release_candidate", "required_gates"],
  "properties": {
    "release_candidate": {"type": "string", "pattern": "^sha256:"},
    "required_gates": {"type": "array", "uniqueItems": true,
      "items": {"enum":
["trace", "schema", "build", "unit", "gametest", "integration", "playtest", "visual", "performance", "security", "compatibility", "mutation", "provenance"]}},
    "policy_profile": {"enum": ["dev", "beta", "release"]}
  }
}
```

부록 D. 연구-설계 추적성 매트릭스

연구	핵심	직접 반영	참조
ReAct (2022)	도구를 사용하는 추론 루프	도구 전 목적/후조건 기록, observation 기반 상태 갱신	[R21]
Reflexion (2023)	언어적 실패 기억	실행으로 검증된 failure memory	[R22]
Self-Refine (2023)	반복 피드백 수정	설계/에셋 초안 개선; 실행 검증 결합	[R23]
CRITIC (2023)	외부 도구 비평	compiler/test/source/VLM evidence critic	[R24]
Toolformer (2023)	모델의 API 호출 학습	tool-call dataset/라우터 평가	[R96]
Gorilla (2023)	API 문서 기반 도구 사용	정확 API retrieval 과 version grounding	[R97]
ToolLLM/ToolBench (2023)	대규모 도구 학습/평가	내부 tool-selection benchmark	[R98]
RAG (2020)	검색 증강 생성	공식 문서/소스 grounding	[R105]
Self-RAG (2023)	검색/생성/비평 통합	검색 필요성·근거 평가	[R15]
Crag (2024)	검색 결과 교정	낮은 신뢰 결과 재검색/웹 보강	[R16]
Adaptive-RAG (2024)	질문 복잡도별 검색	exact lookup/단일/다단계 라우팅	[R108]
RAPTOR (2024)	계층 요약 retrieval	대규모 failure/project memory	[R109]
HyDE (2022)	가상 문서 기반 검색	오류 설명이 짧을 때 query expansion	[R110]
ColBERT (2020)	late interaction retrieval	코드/문서 세부 token matching	[R111]
RepoCoder (2023)	repository retrieval for code	관련 파일/심볼 반복 검색	[R112]
GraphRAG	graph/community summaries	requirement-artifact-test graph 선택 적용	[R17]
SWE-agent (2024)	Agent-Computer Interface	diff/file/search/test 전용 인터페이스	[R32]
Agentless (2024)	단순 단계형 수리	localize-repair-validate 기본 workflow	[R20]
RepairAgent	자율 프로그램 수리	수리 작업 toolset/반복 제한	[R103]
TraceRepair	runtime trace 수리	stack/call/state evidence	[R104]
AgentForge (2026)	실행 기반 다중 에이전트	Planner/Coder/Tester/Debugger/Critic + sandbox	[R84]
SGAgent (2026)	locate-suggest-fix	수리 산출물 단계 분리	[R85]
TraceDev (2026)	추적성 그래프	요구-설계-코드-테스트 누락 탐지	[R86]
FailureMem (2026)	멀티모달 실패 기억	시각 영역 grounding + failure bank	[R87]
SelfHeal (2026)	fix agent + critic	독립 수리/비평과 외부 지식	[R91]
ToolMaker (2025)	자동 tool 생성	새 MCP admission test	[R90]
Dialogue SWE-Bench	사용자 대화형 SWE	계획 승인/수정 schema	[R102]
MCP measurement studies	실제 생태계 품질/보안	커뮤니티 MCP pin/audit/sandbox	[R133]
Secure agent survey/ABAC	신뢰 경계/최소 권한	정책 브로커, provenance, threat eval	[R135]
MCP prompt-injection studies	클라이언트 도구 공격	retrieval data 분리와 tool call 차단	[R136]

부록 E. 2026-07 추가 참고문헌 감사 목록

- [R58] MCP 2026-07-28 Specification Release Candidate. <https://blog.modelcontextprotocol.io/posts/2026-07-28-release-candidate/>
- [R59] Beta SDKs for MCP 2026-07-28 (Python/TypeScript/Go/C#). <https://blog.modelcontextprotocol.io/posts/sdk-betas-2026-07-28/>
- [R60] MCP draft discovery and backward compatibility. <https://modelcontextprotocol.io/specification/draft/server/discover>
- [R61] MCP draft Tools specification. <https://modelcontextprotocol.io/specification/draft/server/tools>
- [R62] MCP draft Roots - informational, not access control. <https://modelcontextprotocol.io/specification/draft/client/roots>
- [R63] MCP draft structured logging. <https://modelcontextprotocol.io/specification/draft/server/utilities/logging>
- [R64] MCP official reference servers repository. <https://github.com/modelcontextprotocol/servers>
- [R65] Docker MCP Gateway. <https://github.com/docker/mcp-gateway>
- [R66] GitHub official MCP Server. <https://github.com/github/github-mcp-server>
- [R67] Microsoft Playwright MCP. <https://github.com/microsoft/playwright-mcp>
- [R68] Context7 MCP. <https://github.com/upstash/context7>
- [R69] Minecraft Dev MCP (MCDxAI). <https://github.com/MCDxAI/minecraft-dev-mcp>
- [R70] mcdev-mcp and DebugBridge. <https://github.com/use-ai-for-mc/mcdev-mcp>
- [R71] Minecraft/Mineflayer MCP Server. <https://github.com/yuniko-software/minecraft-mcp-server>
- [R72] Mineflayer core. <https://github.com/PrismarineJS/mineflayer>
- [R73] MCreatorMCP. <https://github.com/modpotato/MCreatorMCP>
- [R74] Blender MCP Server (27-tool implementation). <https://github.com/djeada/blender-mcp-server>
- [R75] ComfyUI official repository. <https://github.com/Comfy-Org/ComfyUI>
- [R76] ComfyUI server routes. https://docs.comfy.org/development/core-concepts/server_overview
- [R77] Fabric Automated Testing 26.2. <https://docs.fabricmc.net/develop/automatic-testing>
- [R78] Fabric API Loom DSL 26.2. <https://docs.fabricmc.net/develop/loom/fabric-api>
- [R79] Fabric Meta API. <https://github.com/FabricMC/fabric-meta>
- [R80] NeoForge official documentation. <https://docs.neoforged.net/>
- [R81] PaperMC developer documentation. <https://docs.papermc.io/paper/dev/>
- [R82] Modrinth API documentation. <https://docs.modrinth.com/api/>
- [R83] CurseForge REST API documentation. <https://docs.curseforge.com/rest-api/>
- [R84] AgentForge: Execution-Grounded Multi-Agent LLM Framework. <https://arxiv.org/abs/2604.13120>
- [R85] SGAgent: Suggestion-Guided Multi-Agent Repository Repair. <https://arxiv.org/abs/2602.23647>
- [R86] TraceDev: Traceability-Driven Multi-agent Framework. <https://arxiv.org/abs/2607.18886>
- [R87] FailureMem: Failure-Aware Multimodal Repair. <https://arxiv.org/abs/2603.17826>
- [R88] LLM-Based Agentic Systems for Software Engineering. <https://arxiv.org/abs/2601.09822>
- [R89] Evaluating LLM Agents on Automated Software Analysis. <https://arxiv.org/abs/2604.11270>
- [R90] ToolMaker: LLM Agents Making Agent Tools. <https://arxiv.org/abs/2502.11705>
- [R91] SelfHeal: Bug Repair in LLM Agents. <https://arxiv.org/abs/2604.17699>
- [R96] Toolformer. <https://arxiv.org/abs/2302.04761>
- [R97] Gorilla: Large Language Model Connected with Massive APIs. <https://arxiv.org/abs/2305.15334>
- [R98] ToolLLM / ToolBench. <https://arxiv.org/abs/2307.16789>
- [R99] SWE-agent. <https://arxiv.org/abs/2405.15793>
- [R100] OpenHands. <https://github.com/All-Hands-AI/OpenHands>
- [R101] SWE-bench. <https://www.swebench.com/>
- [R102] Dialogue SWE-Bench. <https://arxiv.org/abs/2606.13995>
- [R103] RepairAgent. <https://arxiv.org/abs/2403.17134>
- [R104] TraceRepair. <https://arxiv.org/abs/2604.02647>
- [R105] Retrieval-Augmented Generation. <https://arxiv.org/abs/2005.11401>
- [R108] Adaptive-RAG. <https://arxiv.org/abs/2403.14403>
- [R109] RAPTOR. <https://arxiv.org/abs/2401.18059>
- [R110] HyDE. <https://arxiv.org/abs/2212.10496>
- [R111] ColBERT. <https://arxiv.org/abs/2004.12832>
- [R112] RepoCoder. <https://arxiv.org/abs/2303.12570>
- [R114] RAGAS. <https://arxiv.org/abs/2309.15217>
- [R115] DeepEval. <https://github.com/confident-ai/deepeval>
- [R116] Arize Phoenix. <https://github.com/Arize-ai/phoenix>

- [R117] Qwen3-VL official collection. <https://huggingface.co/collections/Qwen/qwen3-vl>
- [R118] Qwen3-Coder-Next technical report. <https://arxiv.org/abs/2603.00729>
- [R119] TripoSR. <https://github.com/VAST-AI-Research/TripoSR>
- [R120] Hunyuan3D-2. <https://github.com/Tencent-Hunyuan/Hunyuan3D-2>
- [R121] TRELIS. <https://github.com/microsoft/TRELIS>
- [R122] SenseVoice / FunASR. <https://github.com/FunAudioLLM/SenseVoice>
- [R123] MeloTTS. <https://github.com/myshell-ai/MeloTTS>
- [R124] AudioCraft / MusicGen. <https://github.com/facebookresearch/audiocraft>
- [R125] AudioLDM 2. <https://arxiv.org/abs/2308.05734>
- [R127] FG-MDM. <https://arxiv.org/abs/2305.13773>
- [R128] OWASP Dependency-Check. <https://owasp.org/www-project-dependency-check/>
- [R129] CycloneDX Gradle Plugin. <https://github.com/CycloneDX/cyclonedx-gradle-plugin>
- [R130] Sigstore signing and verification. https://docs.sigstore.dev/cosign/signing/signing_with_blobs/
- [R131] SLSA specification. <https://slsa.dev/spec/v1.2/>
- [R132] code2MCP. <https://arxiv.org/abs/2509.05941>
- [R133] Measurement study of the MCP ecosystem. <https://arxiv.org/abs/2509.25292>
- [R134] Production design patterns for MCP-based systems. <https://arxiv.org/abs/2603.13417>
- [R135] Secure Agentic AI Systems survey. <https://arxiv.org/abs/2606.10749>
- [R136] Prompt Injection Attacks in MCP Clients. <https://arxiv.org/abs/2603.21642>
- [R137] Attested Tool Server Admission for MCP. <https://arxiv.org/abs/2605.24248>
- [R138] MCP Tool annotations. <https://modelcontextprotocol.io/specification/2025-11-25/server/tools>
- [R139] MCP Authorization. <https://modelcontextprotocol.io/specification/draft/basic/authorization>
- [R140] MCP Registry. <https://registry.modelcontextprotocol.io/>
- [R141] Minecraft moves to calendar versioning. <https://www.minecraft.net/en-us/article/minecraft-new-version-numbering-system>

v3 개정 결론: “모드 생성기”가 아니라 완성 게임 패키지 컴파일러로 확정한다

기존 v2 는 MCP/API, 코드·에셋 생성, RAG, 보안, 검증을 폭넓게 다뤘지만, 모드 콘텐츠와 전용 맵을 처음부터 하나의 게임 설계로 공동 컴파일하는 계약이 약했다. 또한 Minecraft 실행 없이 빠르게 반복하는 오프라인 가상 월드 계층과, 최종 사용자에게 제공할 모드 설명·설치·플레이 문서까지 하나의 릴리스 산출물로 강제하지 않았다. v3 는 이 세 누락을 보완하고 “사용자 아이디어가 실제로 설치·플레이 가능한 패키지로 끝나는가”를 합격 기준으로 삼는다.

최종 판정 관련 오픈소스를 연결하면 충분히 구현 가능하다. 그러나 VLM 또는 MineWorld 같은 시각적 월드 모델만으로 실제 Minecraft 모드의 동작을 입증할 수는 없다. 오프라인 Virtual Twin 은 설계·공간·시각 오류를 빠르게 줄이는 1 차 검증기이고, 최종 합격은 실제 Fabric/NeoForge/Paper 런타임, GameTest, headless client/server, 플레이 봇에서 받아야 한다. [R150][R153][R156][R184][R186]

34. v3 완전성 감사: 정말 Minecraft 모드에 필요한 모든 것을 만드는가

아래 표는 “파일을 많이 생성했는가”가 아니라, 플레이 가능한 모드 또는 모드+맵 제품을 만드는 데 필요한 모든 책임 영역을 감사한 결과다. 각 기능은 Content Manifest 의 한 항목에서 코드·데이터·에셋·맵 배치·테스트·설명 문서까지 추적되어야 한다.

책임 영역	만들어야 하는 내용	강제 산출물	등급
게임 구상	핵심 판타지, 대상 사용자, 플레이 시간, 핵심 루프, 콘텐츠 기동, 차별점, 범위	GameConceptPlan + 사용자 승인	필수
플랫폼/버전	Java/Bedrock, Fabric/NeoForge/Paper, Minecraft/Java/API/매핑 버전	PlatformLock	필수
게임 규칙	진행, 난이도, 전투, 제작, 경제, 보상, 실패/재시도, 멀티플레이	MechanicSpec + balance tables	필수
코드	등록, 이벤트, 네트워킹, 저장, 명령, config, GUI, 엔티티 AI, 월드젠 bootstrap	source tree + build scripts	필수
데이터	recipe, loot, tag, advancement, predicate, damage/enchantment, worldgen JSON	Datagen outputs	필수
시각 에셋	texture, blockstate, model, client item, entity model, animation, particle, GUI, icon	assets tree + art sources	필수
오디오/음성	sound events, OGG, subtitle, optional ASR/TTS/NPC voice	audio manifest + licenses	기능별
스토리/퀘스트	서사, NPC, 대화, 목표, 분기, 보상, 실패 복구	QuestGraph + dialog data	장르별
전용 맵	지형, 지역, 도로, 던전, 구조물, 보스방, 스폰, 전리품, 퀘스트 위치	WorldDesignIR + world.zip/NBT	요청 시 필수
자동 월드젠	ore/feature/structure/biome/dimension/template pool/processor	worldgen data + tests	콘텐츠형 모드
멀티플레이	권한, 동기화, 중복 보상 방지, 재접속, 서버 성능, admin controls	MP contract + server tests	서버 모드
접근성/현지화	ko_kr/en_us, subtitle, 색상·폰트·키 설정, 난이도 옵션	locale/accessibility report	릴리스
정적/빌드 테스트	schema, reference, compile, unit, datagen diff, dependency audit	machine reports	필수
게임 테스트	GameTest, server boot, client render, bot scenario, save/reload, regression	evidence bundle	필수
시각/공간 테스트	누락 텍스처, UV, z-fight, nav, 충돌, 떠 있는 건물, 스타일	Virtual Twin + screenshots	필수

책임 영역	만들어야 하는 내용	강제 산출물	등급
성능/보안	TPS, heap, chunk load, packet abuse, command permission, dependency CVE	profile/security report	필수
호환성	loader/version/dependency/mod conflict matrix	compatibility matrix	릴리스
배포/설명	JAR, world, resource pack, config, README, 설치/플레이/관리자 가이드, 스크린샷	Release Bundle	필수
지속 개선	완성본 재투입, diff, migration, scale-up, regression preservation	Feedback Bundle	필수

완전성 게이트 Content Manifest의 각 content_id는 implementation, assets, world_placement, tests, documentation의 다섯 edge를 가져야 한다. 해당 기능이 맵 배치를 요구하지 않으면 world_placement에 NOT_APPLICABLE과 근거를 기록한다. 누락 edge가 한 개라도 있으면 release candidate가 될 수 없다.

34.1 “모든 오픈소스 활용”의 정확한 의미

모든 저장소를 무차별 연결하는 것은 품질을 높이지 않고 공격 표면과 버전 충돌만 키운다. 여기서 “모든 오픈소스 활용”은 필요한 capability를 빠짐없이 정의하고, 각 capability마다 현재 가장 적합한 공개 구현, 대체 구현, 자체 래퍼, 비채택 근거를 registry에 기록한다는 뜻이다.

표 34-A. 오픈소스 capability registry의 최소 계약

<pre>capability_registry: capability_id: world.offline_read_write required_by: [map_compiler, virtual_twin, migration] primary: Amulet-Core@pinned_commit alternatives: [WorldPainter, custom_anvil_nbt] execution_mode: isolated_cpu_worker license: BSD-3-Clause support_matrix: [java_version_range] admission_tests: [roundtrip_chunk_hash, palette_mapping, entity_preservation] status: adopted rejected: [{tool: "unknown-editor", reason: "no reproducible CLI/API"}]</pre>

35. 게임 구상 RAG 와 사용자 승인형 Game Design Compiler

사용자 프롬프트를 곧바로 Java 코드로 보내면 범위 누락과 서로 모순되는 콘텐츠가 생긴다. 첫 단계는 프롬프트를 “게임 제작 계약”으로 컴파일하는 것이다. 이 단계의 결과는 기술 계획뿐 아니라 플레이 경험과 맵 구조까지 포함하며, 사용자가 수정·분기·승인하기 전에는 생성 작업이 시작되지 않는다.

35.1 기획 RAG 의 지식 계층과 우선순위

계층	내용	사용 규칙
0. 사용자 근거	현재 프롬프트, 첨부 이미지/음성, 기존 프로젝트, 명시된 금지·선호	최상위. 다른 검색 결과가 덮어쓰지 못함
1. 프로젝트 기억	이전 승인안, content IDs, 스타일 바이블, 실패·수정 기록	동일 프로젝트 연속성
2. 공식 버전 지식	Fabric/NeoForge/Paper/Minecraft 문서, 매핑, API, schema	기술 사실의 source of truth [R142-R149]
3. 공개 소스 심볼	대상 버전 디컴파일 소스, loader API, 종속 모드 코드	정확한 class/method/registry 확인
4. 게임 디자인 패턴	core loop, progression, quest/dungeon/economy/balance 패턴	아이디어를 구조화하되 표절 금지
5. 맵/건축 라이브러리	검증된 room grammar, schematic, palette, 지역·도로 규칙	구조 생성 재사용
6. 최근 트렌드	Modrinth/GitHub 공개 메타데이터, 업데이트/다운로드/리뷰에서 추출한 패턴	영감과 수요 가설만 제공; API 사실 근거 금지 [R174-R175]
7. 실패 기억	과거 compile/runtime/visual/nav/security 실패와 최소 패치	동일 실패 예방
8. 라이선스/보안	SPDX, dependency CVE, asset provenance, MCP server admission	생성/배포 가능 여부

- 질문이 “어떤 게임을 만들까”이면 디자인/트렌드/프로젝트 RAG 를 사용하고, “이 API 가 존재하는가”이면 공식 문서/소스 RAG 만 사용한다.
- 검색된 커뮤니티 문서와 트렌드 문장은 모두 `untrusted_text` 로 태깅하고, 그 안의 명령이나 도구 호출 제안은 실행하지 않는다.
- 계획에는 각 결정의 `evidence_ids` 와 `confidence` 가 붙는다. 근거가 부족한 항목은 `assumption` 또는 `user_decision_required` 로 표시한다.
- RAG 가 너무 많은 자료를 반환하지 않도록 `query classifier` 가 `exact lookup`, `design synthesis`, `compatibility`, `troubleshooting`, `trend discovery` 로 분기한다.

35.2 최초 프롬프트부터 승인까지

상태	중앙 AI 가 하는 일	사용자에게 보이는 것	다음 게이트
INTAKE	프롬프트·이미지·음성·기존 JAR/world 를 정규화	요청 요약과 누락 위험	필요한 경우 핵심 질문
RESEARCH	버전·게임 디자인·트렌드·유사 기능·보안·성능 제약 RAG	근거와 출처가 포함된 조사 카드	근거 충분성
CONCEPT	핵심 판타지, 플레이 루프, 대상 플레이어, 콘텐츠 기동, 차별점 작성	3 개 이하의 분명한 콘셉트 또는 단일 최선안	선택/수정
SCOPE	기능·콘텐츠 수, 모드/플러그인/맵/리소스 팩 경계, T4 예산 산정	범위, 예상 산출물, 난이도, 제외 항목	범위 승인
GAME_PLAN	진행·밸런스·맵 지역·퀘스트·보스·에셋·테스트를 통합	Game Design Plan vN + 변경 가능한 항목	수정/분기/승인
APPROVED	사용자 서명, plan_hash, platform lock, ID lock	승인 요약과 취소 가능 지점	생성 시작

35.3 GameConceptPlan 의 필수 필드

```
{
  "project_id": "frost_kingdom",
  "vision": {"elevator_pitch": "...", "player_fantasy": "...", "pillars": ["explore","craft","boss"]},
  "audience": {"players": "1-4", "experience": "mid", "target_hours": 8},
  "platform_lock": {"edition":"java", "minecraft":"26.2", "loader":"fabric"},
  "core_loop": ["explore", "collect", "upgrade", "unlock", "defeat"],
  "progression": {"chapters": [...], "gates": [...], "failure_recovery": [...]},
  "content_budget": {"blocks":18, "items":24, "mobs":4, "bosses":1, "structures":6},
  "world_plan": {"delivery":"prebuilt_world+worldgen", "regions":[...], "critical_path":[...]},
  "systems": {"combat":..., "crafting":..., "quests":..., "multiplayer":...},
  "asset_style": {"resolution":16, "palette":"...", "model_complexity":"vanilla_like"},
  "acceptance_tests": [...],
  "assumptions": [...], "user_decisions_required": [...],
  "evidence_ids": [...], "plan_hash":"sha256:...", "status":"PROPOSED"
}
```

사용자 수정 방식 사용자는 “성은 더 크고, 광산은 빼고, 2인 협동으로, 보스 전투를 3단계로”처럼 자연어로 수정한다. 중앙 AI는 기존 계획 전체를 다시 쓰지 않고 JSON Patch와 영향 분석을 보여준다. 승인 후에만 새로운 plan_hash를 생성하고 하위 작업 그래프를 재검파일한다.

35.4 계획 품질 검증

검증기	질문	불합격 예
Completeness Critic	모든 사용자 요구가 기능/맵/에셋/테스트에 연결됐는가	보스는 요구됐지만 arena, loot, animation, test가 없음
Contradiction Solver	진행·난이도·플랫폼·멀티플레이 요구가 모순되는가	클라이언트 설치 금지인데 Fabric client renderer 필요
Feasibility Estimator	T4, 시간, 메모리, 라이선스로 만들 수 있는가	대형 3D diffusion을 T4 상시 병렬 실행
Gameplay Critic	core loop와 보상/진행이 닫혀 있는가	핵심 아이템을 얻어도 다음 목표가 없음
World Critic	지역/동선/획득 장소/보스 공간이 연결되는가	열쇠는 존재하지만 획득 장소가 없음
Evidence Critic	기술 결정이 공식 버전 근거를 갖는가	존재하지 않는 API를 전제로 계획
User Intent Critic	원래 프롬프트를 과도하게 바꾸지 않았는가	사용자가 생존 모드를 원했는데 RPG 캠페인으로 변경

36. 모드-맵 공동 설계: World Design IR 와 콘텐츠 의존 그래프

맵은 모드가 완성된 뒤 장식으로 붙이는 별도 작업이 아니다. 광석, 몹, 퀘스트, 보스, NPC, 제작 재료, 차원과 구조물의 존재 이유와 획득 순서를 공간에 배치하는 게임플레이 컴파일 단계다. 따라서 GameConceptPlan 이 승인되면 Mod IR 과 World Design IR 이 동시에 생성되고 서로의 ID 를 공유한다.

36.1 세 가지 맵 산출 모드

모드	적합한 제품	최종 산출	핵심 도구
PREBUILT_WORLD	스토리/RPG/탈출/레이드/서버 캠페인	world.zip + map manifest + configs	Amulet Core, WorldPainter, structure compiler, WorldEdit
IN_MOD_WORLDGEN	일반 콘텐츠 모드, 어느 시드에서도 플레이	worldgen JSON/code + NBT structures + biome/dimension	Fabric/NeoForge Datagen, structure/template pool, GameTest
PATCH_EXISTING_WORLD	기존 서버/세이브에 도시·던전·로비 추가	world patch + rollback + region diff	Amulet Core offline 또는 WorldEdit/GDPC live

하나의 프로젝트가 세 모드를 조합할 수도 있다. 예를 들어 핵심 캠페인은 prebuilt world 로 제공하고, 광석과 작은 폐허는 in-mod worldgen 으로 추가하며, 사용자의 기존 서버에는 patch package 를 제공한다.

36.2 World Design IR

<pre>{ "world_id": "frost_kingdom_main", "coordinate_system": { "origin": [0,80,0], "north": "-z", "scale": "1_block=1m" }, "terrain": { "size_chunks": [128,128], "height_profile": "glacial_valley", "biomes": [...] }, "regions": [{ "id": "starter_village", "role": "tutorial", "footprint": [96,96], "entry_points": [...], "exits": [...] }, { "id": "boss_castle", "role": "finale", "arena": { "min": [40,40,16], "players": 4 } }], "routes": [{ "from": "starter_village", "to": "mine", "type": "road", "max_slope": 0.35 }], "placements": [{ "content_id": "item.ancient_key", "location": "mine.vault_chest", "acquisition": "loot_once" }, { "content_id": "entity.fallen_king", "location": "boss_castle.arena", "spawn": "quest_gate" }], "structure_sources": ["schematic://...", "grammar://castle_v3", "generated://dungeon_seed_41"], "quest_bindings": [...], "spawn_rules": [...], "loot_bindings": [...], "constraints": [...], "navigation_tests": [...], "visual_tests": [...] }</pre>

36.3 콘텐츠 의존 그래프가 자동으로 요구하는 맵 요소

콘텐츠	자동 파생되는 공간 요구	검증
광석/재료	생성 고도, 바이옴, 희귀도, 채굴 단계, 획득 가능한 동굴/광산	분포 통계, tool tier, progression gate
일반 몹	spawn biome/region, 밀도, 안전 거리, despawn, loot 접근	spawn sample, TPS, encounter path
보스	arena 크기/높이, 입구, 봉쇄, phase hazard, 관전자/멀티 플레이 공간, 보상방	봇 전투, 경계 탈출, phase state, 재접속
퀘스트 아이템	획득 위치, 중복 방지, 분실 복구, 다음 목표까지 경로	critical path complete, save/reload
NPC	안전한 위치, 접근 경로, 대화 trigger, crowding, 공격 보호	interaction reachability
기계/제작대	사용 공간, 전력/입출력, 튜토리얼 배치, recipe 재료 근접성	screen+container+crafter scenario
신규 차원	입장 portal, spawn safety, 귀환, dimension progression, structure distribution	portal roundtrip, spawn sampling
음성 명령	마이크 권한/오프인, 명령 범위, 오인식 취소, 서버 권한	ASR confidence, confirmation, abuse tests

36.4 맵 컴파일러의 실제 단계

1. 지형 합성: heightmap, biome mask, water/river, cave/ore fields 를 seed 와 버전으로 고정한다.
2. 의미 지역 그래프: 시작점, 허브, 선택 지역, 필수 지역, 보스 종점을 노드와 gate 로 표현한다.
3. 배치 최적화: 면적, 경사, 거리, 시야, 충돌, 청크 경계, 성능 비용을 제약으로 좌표를 결정한다.
4. 도로/경로: A^* 또는 cost field 로 도로·다리·계단을 만들고 max slope 와 폭을 검증한다.
5. 구조 합성: 검증된 schematic, procedural grammar, WFC/template pool 을 조합한다.
6. 게임플레이 바인딩: entity, NPC, loot, quest trigger, checkpoint, music zone, particle zone 을 공통 ID 로 연결한다.
7. 오프라인 Virtual Twin 검증: block palette, collision, path, room, visibility, style, critical path 를 검사한다.
8. 월드 컴파일: Amulet/Anvil NBT 또는 Datagen 구조물·worldgen 으로 실제 산출물을 만든다.
9. 실제 런타임 인증: GameTest, server/client, Mineflayer, save/reload, performance 를 실행한다.

블록 좌표 생성 원칙 LLM 은 수십만 개의 block 좌표를 직접 출력하지 않는다. LLM 은 region graph, constraints, building grammar 와 parameter 를 만든다. 결정론적 compiler 가 block placement 를 수행하고 모든 배치에 seed, generator_version, source_template, hash 를 기록한다.

37. Minecraft 없이 만드는 Virtual Minecraft Twin 과 VLM 검증

사용자가 말한 “Minecraft 에 연결하지 않고 가상 Minecraft 를 VLM 이 확인하는 방식”은 가능하다. 단, 이를 실제 게임 엔진의 완전한 대체물이 아니라 오프라인 디지털 트윈으로 설계해야 한다. 핵심은 영상 생성 모델이 임의의 Minecraft 화면을 상상하게 하는 것이 아니라, 실제 block registry 와 World Design IR 로 결정론적 voxel scene 을 만들고, VLM 은 그 렌더를 검사하는 것이다.

37.1 확정 구조

그림 37-A. Virtual Minecraft Twin 의 오프라인-실게임 이중 검증



37.2 오프라인 Twin 을 구성하는 오픈소스

구성요소	역할	채택 상태
Amulet Core	Java/Bedrock 저장 월드 포맷 읽기·쓰기, 청크/블록 단위 컴파일과 round-trip	핵심 채택 [R150]
Prismarine minecraft-data/chunk/NBT	버전별 block/item/protocol data 와 chunk/NBT 자료구조	registry snapshot·경량 scene 에 채택 [R155]
WorldPainter	heightmap/terrain/biome 초안과 대형 지형 편집	선택형 terrain backend [R152]
Custom Voxel Twin	sparse chunk, collision AABB, nav mesh, light approximation, quest graph	자체 구현, deterministic source of truth
Chunky	저장 월드의 고품질 경로 추적 렌더와 batch/headless 이미지	시각 인증 렌더러 [R153]
jMc2Obj + Blender	월드를 OBJ 로 변환해 Blender 에서 camera/occlusion/render 분석	현재 버전 호환성 검사 후 선택 [R154]
Blender MCP	카메라·조명·모델·렌더 작업 자동화	격리형 visual worker [R177]
Qwen3-VL 계열	스크린샷/맵/GUI/모션 프레임의 시각 비평과 위치 grounding	GPU lease 로 순차 실행 [R169-R170]
MineWorld	Minecraft 화면+action 에서 다음 장면을 생성하는 공개 visual world model	아이디어/미리보기 연구 경로, deterministic 검증 대체 금지 [R184]
VIGA/SimWorlds	write-run-render-compare-revise 와 deterministic verifier 패턴	Twin 생성·검증 loop 에 연구 원칙 채택 [R185-R186]

37.3 Minecraft 없이 검증 가능한 것과 불가능한 것

오프라인에서 강하게 검증 가능	근사 또는 제한적	실제 Minecraft가 반드시 필요한 것
block ID/namespace/파일 참조, 구조 크기·겹침, terrain intersection, 문·통로 폭, collision 기반 이동, critical path, 퀘스트 그래프, loot 위치, 시야·조명 근사, 렌더 스타일, 누락 텍스처	redstone 간단 논리, entity hitbox/attack range 근사, mob encounter 난이도, lighting spawn 위험, chunk 성능 비용 추정	loader/API/Mixin, block tick 과 random tick, 실제 redstone·fluid, entity AI goal, networking, server authority, GUI/container sync, shader/render pipeline, 다른 모드와 충돌, 실제 FPS/TPS, save migration

중요한 한계 MineWorld 같은 visual-action world model 은 화면의 다음 상태를 그럴듯하게 예측할 수 있지만, 생성된 픽셀은 실제 block registry, NBT, server tick, mod code 실행의 증거가 아니다. 따라서 콘셉트 영상·경로 가설·시각적 이상 탐지에는 쓸 수 있어도 release gate 가 될 수 없다. [R184]

37.4 VLM 검사 프롬프트와 증거 형식

SYSTEM: You are an independent Minecraft map visual verifier.
 INPUTS: approved_style_bible, camera_pose, depth_map, block_id_buffer, expected_region_spec, current_render, previous_render, deterministic_findings.
 TASKS:
 1. Locate only visible defects with normalized bounding boxes.
 2. Classify: floating, terrain_clip, broken_roof, repetition, palette_mismatch, missing_texture, unsafe_darkness, visual_blockage, landmark_unreadable.
 3. Never declare gameplay correctness from pixels.
 4. Cite block-id/depth evidence when available.
 OUTPUT: {findings:[{bbox,category,severity,evidence,repair_scope}], confidence}
 PASS RULE: VLM pass is advisory; deterministic and runtime gates remain mandatory.

시각 검사는 동일한 VLM 이 생성과 판정을 모두 담당하지 않도록 generator 와 verifier 를 분리한다. 고정 카메라 세트, block-id buffer, depth map, 이전 승인 렌더를 함께 제공해 단순 미적 취향보다 구체적 결함을 찾게 한다. VIGA 와 SimWorlds 의 실행-렌더-검증 루프 및 deterministic verifier 원칙을 적용한다. [R185-R186]

37.5 오프라인 맵 반복 루프

- World Design IR 에서 region 별 저해상도 occupancy 를 만들고 배치 제약을 먼저 해결한다.
- 경로·문·계단·수직 이동·critical path 를 graph validator 로 통과시킨다.
- 구조를 block 단위로 컴파일하고 region overlap, floating support, palette 와 block registry 를 검사한다.
- 고정 camera, top-down, isometric, player-eye, boss-arena viewpoints 를 렌더한다.
- VLM 과 image-diff 가 구체적인 repair scope 를 반환한다.
- 맵 작업자는 전체 월드를 다시 생성하지 않고 해당 region/structure revision 만 교체한다.
- 오프라인 기준을 통과하면 Amulet/NBT/schematic 을 생성하고 실제 게임 테스트 단계로 넘긴다.

38. 모든 관련 오픈소스의 역할·채택·대체 경로

아래는 완전형 제품을 위해 실제로 연결할 공개 기술의 capability map 이다. “채택”은 기본 경로, “선택”은 기능이 요구될 때만, “격리”는 검증 환경에서만, “참조”는 연구·데이터·알고리즘을 참고하되 production dependency 로 두지 않는다는 뜻이다.

Capability	오픈소스/공개 기술	적용 방식	상태
모드 런타임	Fabric API/Loom, NeoForge/ModDevGradle, Paper API	Fabric 우선, NeoForge/Paper adapter	채택
데이터/에셋 생성	Fabric/NeoForge Datagen	recipe/loot/tag/model/lang/worldgen 를 코드에서 생성	채택 [R142-R149]
소스/API grounding	Minecraft Dev MCP, versioned decompile/mappings	read-only cache 와 source citation	채택
간단 콘텐츠 fallback	MCreator + MCreatorMCP	템플릿 요소에 한정, 생성 코드 감사	격리
모델/UV/애니메이션	Blockbench, GeckoLib	bbmodel 원본 + export + bone/UV validator	채택
텍스처/GUI	ComfyUI, FLUX.1-schnell, Pixelorama/LibreSprite, Pillow/OpenCV	생성 후 pixel/palette/alpha deterministic cleanup	채택/선택
3D 초안	Blender MCP, Hunyuan3D-2.1/TripoSR	Minecraft low-poly/cube 변환 전용	선택/원격
스킨	BLOCK open pipeline	character preview -> skin atlas 특화	선택 [R187]
오디오	Audacity, Stable Audio/Open, AudioCraft, ffmpeg	라이선스 기록, OGG normalize	선택
ASR/TTS	faster-whisper/SenseVoice, Piper/MeloTTS	음성 입력/NPC voice opt-in	선택
오프라인 월드 I/O	Amulet Core, custom Anvil/NBT tests	world.zip/patch/roundtrip	채택
지형 설계	WorldPainter, Arnis, procedural heightmap libraries	전용 지형·실세계 지도 요청 시	선택
실행 중 월드 편집	WorldEdit/FAWE, GDPC/GDMC	live server/preview 에서만	채택/선택
오프라인 렌더	Chunky, jMc2Obj, Blender	Chunky 기본, OBJ 는 호환성 조건	채택/선택
봇/프로토콜	Mineflayer, node-minecraft-protocol, prismarine-data/chunk/viewer	playtest, state inspection, lightweight viewer	채택
실제 headless client	HeadlessMc + mc-runtime-test	렌더/runtime 회귀 CI	채택 [R156]
AI 연구 환경	MineDojo, MineStudio, MC-Planner, Voyager/JARVIS-1	계획/스킬/benchmark 아이디어	참조
시각 world model	MineWorld	컨셉트 preview/연구, 인증 금지	실험
레벨 생성 연구	Word2Minecraft, World-GAN, GDMC/WFC	story-to-layout, PCG, 평가 지표	참조 [R163-R165][R188]
3D agent 연구	VIGA, BlenderRAG, SimWorlds	실행·렌더·검증·runtime state 검사	참조/일부 코드 채택
테스트	JUnit, GameTest, Testcontainers, Mineflayer scenarios	unit -> game -> end-to-end pyramid	채택
정적 품질	Checkstyle, SpotBugs, Error Prone, Semgrep	코드/API/security gate	채택
공급망	CycloneDX, Dependency-Check, Trivy/Grype, Sigstore/Cosign, SLSA	SBOM, CVE, provenance, signing	채택
릴리스	Modrinth API/Minotaur, GitHub Releases	metadata, JAR/ZIP, changelog, gallery	채택 [R174-R175]

38.1 오픈소스 자동 입장 시험

- 라이선스와 재배포 가능성: 코드, 모델 가중치, 생성 결과의 조건을 각각 기록한다.
- 유지보수: 최근 release/commit, issue 응답, 지원 버전, 버전 pin 과 checksum 을 기록한다.
- 기능 재현: 공식 예제로 smoke test 하고, 실제 프로젝트의 최소 fixture 로 재시험한다.
- 보안: filesystem/network/process/secrets 권한을 산출하고 sandbox profile 을 적용한다.
- 결정론: seed, version, model hash, workflow hash, input hash 로 결과를 재현할 수 있어야 한다.
- 대체 경로: 기본 도구가 실패했을 때 동일 capability 를 수행할 fallback 또는 수동 승인 경로를 둔다.

- 퇴출 조건: license 변화, unmaintained, 지원 버전 미달, sandbox escape, 반복적인 corrupt output 이면 registry 에서 disabled 처리한다.

38.2 T4 에서의 모델 라우팅

작업	권장 모델/도구	T4 운용
기획·조사	GLM-5.2/Kimi K3/K2.6 원격 champion; Qwen3.5-9B local fallback	복잡 계획은 원격, T4 는 초안·수정·장애 fallback
코드	Kimi K2.7-Code/Qwen3-Coder-Next 원격; Qwen3.5-9B fallback	repo 규모와 테스트 난이도로 routing
시각 검사	Qwen3-VL-8B 4bit	렌더 batch 별 GPU lease; 생성 모델과 분리
RAG	Qwen3-Embedding/Reranker 0.6B	상시 또는 CPU; 32K chunk/rerank
텍스처	ComfyUI + FLUX.2-klein-4B	단독 lease, 생성 후 CPU pixel cleanup
3D/모션	Blockbench procedural 우선; Hunyuan3D/Wan2.2 원격 선택	결과를 cube/bone/keyframe IR 로 단순화
음성	SenseVoice/Fun-ASR, CosyVoice 3.0	필요할 때만 load
맵 컴파일	Amulet/voxel graph/constraint solver/Chunky	대부분 CPU·RAM·디스크

39. 최종 산출물, 모드 설명, 배포 및 재투입 계약

“딱 하고 최종적으로 나오는 것”은 JAR 한 개가 아니다. 설치 가능한 모드, 필요한 월드/리소스/설정, 검증 증거, 사용자 설명, 개발 원본과 라이선스를 묶은 Release Bundle 이다. 제품 유형에 따라 없는 항목은 명시적으로 NOT_APPLICABLE 처리한다.

```

release/frost_kingdom-1.0.0/
  binaries/
    frost_kingdom-1.0.0.jar
    frost_kingdom-server-addon.jar      # 필요 시
  world/
    frost_kingdom_world.zip             # prebuilt world 요청 시
    structures/*.nbt
    schematics/*.schem
    map_manifest.json
  packs/
    resourcepack.zip
    datapack.zip
  config/
    default-configs/*
    server-recommended.properties
  source/
    source.zip
    art_source/{bbmodel,pxo,blend,audio_project}/
  docs/
    MOD_DESCRIPTION_KO.md
    MOD_DESCRIPTION_EN.md
    INSTALL.md
    PLAYER_GUIDE.md
    MAP_AND_PROGRESSION_GUIDE.md
    RECIPES_AND_ITEMS.md
    SERVER_ADMIN_GUIDE.md
    ACCESSIBILITY_AND_CONTROLS.md
    COMPATIBILITY.md
    CHANGELOG.md
    KNOWN_ISSUES.md
  media/
    icon.png
    screenshots/*.png
    trailer_storyboard.md
  evidence/
    completeness_report.json
    test_report.html
    gametest.xml
    playtest_traces/
    virtual_twin_report.json
    performance_report.json
    security_report.json
    compatibility_matrix.json
  supply_chain/
    LICENSE
    THIRD_PARTY_NOTICES.md
    assets_provenance.json
    sbom.cdx.json
    provenance.intoto.jsonl
    SHA256SUMS
    signatures/
  release_manifest.json
  
```

39.1 자동 생성되는 모드 설명

설명 마케팅 문구만 생성하지 않고 실제 Release Manifest 와 테스트 결과에서 사실을 가져온다. 존재하지 않는 기능, 지원 버전, 멀티 플레이 지원을 과장하지 않는다.

설명 항목	자동 근거
한 줄 소개/elevator pitch	승인된 GameConceptPlan.vision
주요 특징	content manifest 에서 release 상태인 기능만
콘텐츠 수	registry 의 실제 blocks/items/entities/structures/quests count
맵 설명	World Design IR 의 regions, delivery mode, critical path
진행 방법	QuestGraph 와 acquisition edges
설치 방법	PlatformLock, dependencies, bundle layout
서버/멀티플레이	실제 통과한 server/client/mp tests
명령·설정	등록 코드와 config schema 에서 추출
호환 버전/모드	Compatibility Matrix 의 pass 조합만
성능 요구	benchmark machine 과 measured TPS/RAM
알려진 문제	unresolved non-blocking issues
크레딧/라이선스	THIRD_PARTY_NOTICES 와 asset provenance
변경 내역	승인 plan/version/commit/artifact diff

39.2 최종 설명 템플릿

Frost Kingdom 한 줄 소개: 얼어붙은 왕국을 탐험하고 고대 제작 체계를 복구해 몰락한 왕을 쓰러뜨리는 1-4 인 모험 모드입니다. ## 포함 내용 - 신규 블록 18, 아이템 24, 일반 몹 4, 보스 1, 구조물 6 - 전용 캠페인 월드 및 일반 월드용 구조물/광석 생성 - 5 단계 진행, 퀘스트, 제작, 보스 전투, 한국어/영어 지원 ## 전용 맵 시작 마을 -> 폐광 -> 마법사 탑 -> 얼어붙은 수도 -> 왕성 순서로 진행합니다. 지도상의 모든 필수 아이템, NPC, 전리품과 보스는 모드 registry ID 에 연결되어 검증되었습니다. ## 설치 / 의존성 / 서버 [Release Manifest 에서 자동 채움] ## 검증 상태 Build, GameTest, headless client, 1-4 인 시나리오, map critical path, save/reload, 보안/성능 gate 결과를 요약합니다. ## 알려진 문제와 지원 범위 [검증되지 않은 조합은 지원한다고 쓰지 않음]
--

39.3 최종 패키지 합격 계약

Gate	검사 내용	릴리스 기준
Manifest coverage	모든 필수 content_id 가 implementation/assets/world/tests/docs edge 를 보 유	100%
Reference integrity	resource path, registry ID, model/texture/sound/loot/quest 참조	unresolved 0
Build/Datagen	clean build, deterministic datagen diff	pass
GameTest/Unit	required tests 와 retry policy	pass
Virtual Twin	critical path, collision/nav, structure overlap, visual severity	blocker/critical 0

Gate	검사 내용	릴리스 기준
Real server/client	boot, load, render, networking, save/reload	pass
Bot playthrough	필수 캠페인 또는 대표 기능 시나리오	critical path 100%
Performance	TPS, heap, chunk generation, packet budget	승인 threshold 내
Security	critical/high CVE, dangerous permission, command/network abuse	critical/high 0 또는 사용자 승인된 예외
Compatibility	지원한다고 문서화한 모든 조합	pass
Documentation	설치/플레이/맵/관리/호환/라이선스/known issues	누락 0
Provenance	hash, SBOM, model/workflow/asset source, signature	완료

39.4 완성 모드 재투입과 확장

사용자는 최종 bundle 또는 JAR/world 를 다시 업로드하고 “보스 3 종 추가”, “기존 서버 월드에만 도시 삽입”, “26.3 으로 포팅”, “보안 취약점 수정”, “맵을 4 배 확장”처럼 후속 요청할 수 있다. 시스템은 바이너리와 소스를 inventory 하고 이전 Release Manifest 를 기준으로 변경 영향 그래프를 만든다.

1. 입력 bundle 의 hash, version, loader, registry, assets, world, config, tests 와 문서를 inventory 한다.
2. 사용자 후속 요청을 기존 plan 과 diff 하고 보존해야 할 invariant 를 나열한다.
3. 변경 plan 을 사용자에게 다시 제시하고 승인받는다.
4. 영향받는 content/world region/test/doc 만 재생성한다.
5. 이전 전체 회귀와 새로운 테스트를 함께 실행한다.
6. 새 version, changelog, migration guide, map patch/rollback 을 생성한다.

40. 확정 아키텍처, 단계별 합격 기준과 현실적 한계

40.1 최종 end-to-end 상태 머신

PROMPT / VOICE / IMAGES / EXISTING MOD+WORLD
-> INTAKE_NORMALIZED
-> RAG_RESEARCHED
-> GAME_CONCEPT_PROPOSED
-> USER_MODIFY USER_BRANCH USER_APPROVE
-> PLAN_LOCKED + PLATFORM_LOCKED + ID_LOCKED
-> MOD_IR + WORLD_DESIGN_IR + TEST_PLAN + DOC_PLAN
-> CODE / DATA / ASSET / AUDIO / MAP workers
-> STATIC + DATAGEN + BUILD
-> VIRTUAL_MINECRAFT_TWIN
deterministic spatial validators
multi-view render + independent VLM review
-> OFFLINE_WORLD_COMPILE
world.zip / NBT / schematic / worldgen
-> REAL_MINECRAFT_CERTIFICATION
GameTest + server + headless client + Mineflayer + save/reload
-> PERFORMANCE + SECURITY + COMPATIBILITY
-> RELEASE_DESCRIPTION + GUIDES + MEDIA + SBOM + SIGNATURE
-> USER_ACCEPTANCE
-> RELEASED
-> RE-INGEST / PATCH / SCALE / PORT / SECURITY_FIX

40.2 하나의 중앙 AI 와 제한된 전문 작업자

역할	책임	직접 못 하는 것
Main Orchestrator	사용자 의도, RAG, 계획, 승인, 작업 그래프, gate 와 예산	대량 block 배치, 임의 shell, 자기합격
Code/Data Worker	loader 별 코드·Datagen·패치	버전 근거 없이 API 생성
Asset Worker	texture/model/animation/GUI/audio spec 와 산출	registry ID 변경
World Compiler	terrain/region/road/structure/placement 와 world output	자유형 LLM 좌표 덤프
Virtual Twin Verifier	공간/시각/critical path 의 오프라인 증거	실제 mod runtime 인증
Runtime Tester	실제 Minecraft 에서 기능/렌더/네트워크/저장 검증	설계 의도를 임의 변경
Security/Release Worker	CVE, permission, SBOM, signing, docs, publishing	검증 안 된 기능을 설명에 포함

40.3 단계별 합격과 중단 기준

단계	진입 조건	합격	중단/사용자 개입
기획	사용자 prompt 수신	요구 추적 100%, 모순 해소, 사용자 승인	핵심 의사결정 미확정
생성	plan/platform/ID lock	모든 예정 artifact 생성	동일 root cause 3 회 실패
오프라인 Twin	static/build pass 또는 map-only fixture	critical path 와 blocker 결함 0	구조 재설계가 필요한 경우
실게임 인증	world/JAR candidate	필수 GameTest/server/client/bot pass	loader/API/동작 가정 실패
릴리스	품질·보안·성능 통과	bundle/docs/provenance 완전	critical/high 보안 또는 지원 조합 실패
확장	이전 release manifest	invariant 보존 + 새 기능/회귀 pass	호환 불가능한 대규모 migration

배치 프로파일은 명시적으로 선택한다. PROFILE F에서는 최신 원격/API 모델을 기획·코드·시각·미디어 역할별로 경쟁시킨다. PROFILE L에서는 Qwen3.5-9B Q4 또는 Gemma 4 E4B-it Q4 같은 로컬 범용 멀티모달 모델을 supervisor 와 VLM 으로 순차 재사용하고, Qwen3.5-4B/Gemma 4 E2B 를 분류·추출·간단한 수정에 쓴다. PROFILE L은 원격 AI 추론을 장애 fallback 으로도 사용하지 않는다.

40.4 T4 16GB 배치 결론

- 로컬 9B 4bit 범용 fallback 모델, VLM, 이미지/3D 모델을 동시에 올리지 않고 GPU lease 로 순차 로딩한다.
- RAG, JSON schema, constraint solving, block compiler, Amulet world I/O, pathfinding, GameTest orchestration 은 CPU 중심이다.
- 대형 3D/영상 world model 은 원격 또는 선택 기능으로 두며, T4 기본 경로는 Blockbench procedural model 과 deterministic voxel twin 이다.
- 월드가 클수록 GPU 보다 시스템 RAM, Java heap, chunk I/O, 렌더 시간이 병목이므로 region 별 cache/resume 와 artifact hash 가 필수다.
- 완전형 v1 제품은 Minecraft 한 안정 버전 + Fabric + prebuilt world/in-mod worldgen + ko/en + 핵심 에셋에 범위를 고정해야 한다.

40.5 최종 현실적 판정

가능한 것 현재 공개된 오픈소스와 3B-7B 급 모델을 조합하면 사용자 구상 -> 승인된 게임 계획 -> 코드/데이터/텍스처/모델/애니메이션/사운드 -> 모드 전용 맵/월드젠 -> 오프라인 가상 검증 -> 실제 Minecraft 테스트 -> 설명과 배포 패키지까지 자동화하는 시스템은 구현 가능하다. 특히 맵 컴파일은 LLM 보다 결정론적 도구와 그래프 검증이 중심이므로 T4 에서도 현실적이다.

보장할 수 없는 것 어떤 자유형 대형 모드든 한 번에 완벽하게 생성하거나, Minecraft 를 전혀 실행하지 않고 모든 동작을 증명하는 것은 불가능하다. VLM/world model 은 시각·공간 오류를 낮추는 보조 계층이다. 실제 loader, tick, AI, networking, 렌더, 다른 모드와의 충돌은 실제 런타임에서만 확정할 수 있다.

확정 제품 정의 이 문서가 정의하는 최종 제품은 “코드 생성 챗봇”이 아니라, 사용자 승인형 Game Design Compiler + Mod/World compiler + Virtual Twin + real-runtime certification + Release Bundle generator 다.

41. 범용 핵심 프롬프트 자동 조사·Reference Game Resolver 와 Skill/MCP 실행 계약

v3 에는 게임 구상 RAG 와 사용자 승인 단계가 있었지만, “메이플스토리 만들어줘”, “요즘 인기 있는 협동 생존 게임처럼”, “목소리로 마법을 쓰는 RPG”처럼 범위가 크고 외부 맥락에 의존하는 입력을 받드시 광범위한 연구 모드로 전환하는 실행 계약은 충분히 강제되지 않았다. v4 에서는 특정 게임명이나 장르를 사전에 하드코딩하지 않고, 어떤 핵심 프롬프트든 의도·참조 대상·지식 공백을 자동 분석하여 웹/RAG/MCP 조사 계획을 먼저 생성하도록 확정한다.

개인용 제작을 기본 사용 시나리오로 보고, 기획 단계의 필수 게이트는 IP 위험 조사나 차단이 아니라 플레이 구조 이해, Minecraft 구현 변환, 범위 추정, 사용자 승인과 실행 가능성 검증에 둔다.

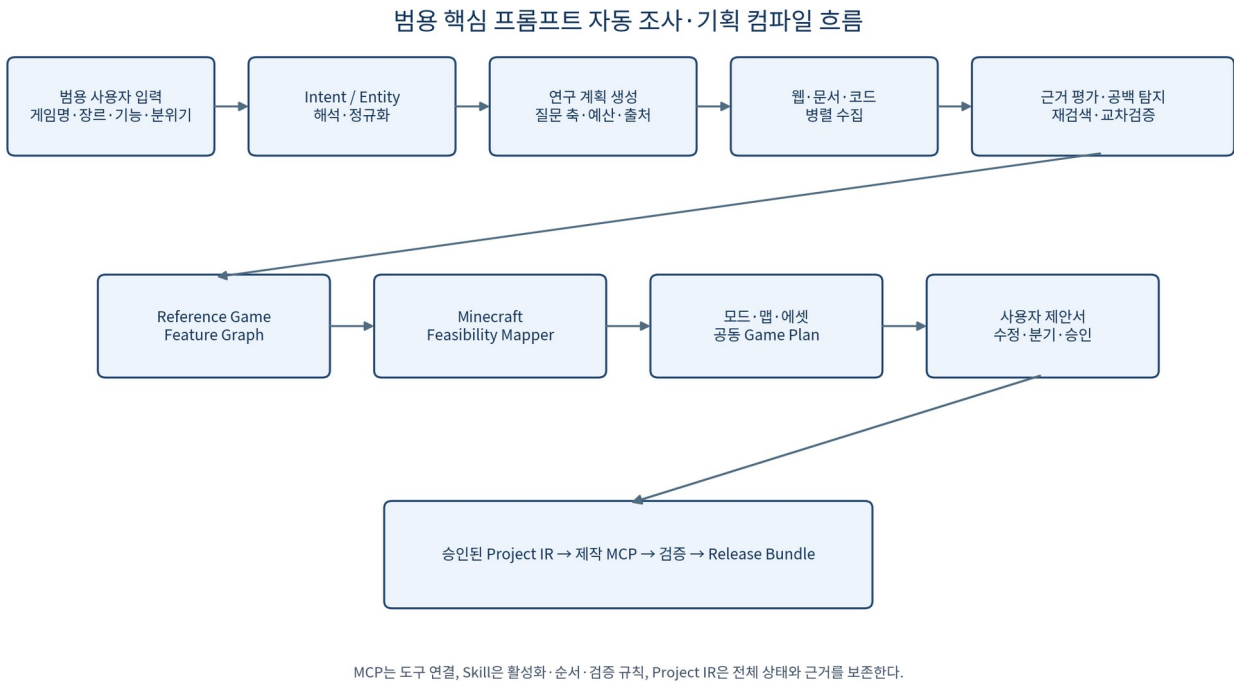


그림 10. 범용 프롬프트가 웹/RAG 조사, Reference Game Feature Graph, Minecraft 계획과 사용자 승인으로 변환되는 흐름

41.1 범용 입력 분류와 자동 연구 트리거

중앙 모델이 바로 검색어나 구현 파일을 만들지 않는다. 먼저 Universal Prompt IR 을 생성하고 입력 유형, 명명된 대상, 사용자가 원하는 결과, 미정 항목, 검색 필요성, 검색 예산을 구조화한다. 맞춤법 오류, 줄임말, 한국어·영어 혼용, 게임명과 일반 명사의 충돌도 entity resolution 단계에서 정규화한다.

입력 유형	예시	자동 행동
REFERENCE_GAME	“메이플스토리 만들어줘”, “젤다식 던전”	명명 대상 확인 → 폭넓은 웹 조사 → Feature Graph → Minecraft 변환
GENRE_OR_EXPERIENCE	“협동 생존 RPG”, “힐링 농장 게임”	장르 패턴·최근 트렌드·플레이 루프 조사 → 3 개 콘셉트 제안
FEATURE_BUNDLE	“음성 마법, 직업, 보스 레이드 넣어줘”	각 기능의 구현·콘텐츠·에셋·맵 의존성 조사
ART_OR_MOOD	“동화풍, 어두운 고딕 느낌”	스타일 자료·팔레트·모델 규칙 조사 후 스타일 바이블 생성
TECHNICAL_EXTENSION	“기존 모드에 길드전 추가”	Release Bundle 분석 → 영향 범위·API·회귀 테스트 조사

AMBIGUOUS_OR_TYPO	고유명사 불명확, 오타, 축약형	웹 entity 후보 비교 → 신뢰도가 낮으면 최소 핵심 질문 1 회
-------------------	-------------------	--

연구 트리거 규칙은 다음과 같다.

- 명명된 게임·모드·작품·장르가 포함되거나 사용자가 “비슷하게”, “같은 느낌”, “요즘 인기 있는”이라고 요청하면 BROAD_REFERENCE_RESEARCH 를 자동 실행한다.
- 요구가 추상적이고 콘텐츠 규모가 큰 경우 디자인·트렌드·기술·Minecraft 구현 가능성을 모두 조사한다.
- 정확한 API·파일 형식·버전 질문은 BROAD 검색이 아니라 공식 문서와 소스의 EXACT_LOOKUP 으로 제한한다.
- 검색할 필요가 없는 사용자 결정과 프로젝트 내부 사실은 웹보다 Project Memory 를 우선한다.
- 연구가 필요한데 검색을 생략한 계획은 승인 화면에 올릴 수 없다.

41.2 광범위 웹/RAG 조사 계획을 만드는 방식

REFERENCE_GAME 또는 장르 수준 입력은 한 번의 검색으로 끝내지 않는다. Research Planner 가 먼저 지식 공백 행렬을 만들고, 각 축별 검색어를 여러 언어와 표현으로 확장한다. 검색 규모는 고정 숫자가 아니라 공백과 모순이 없어질 때까지 늘리되, BROAD 모드의 기본 목표는 12-18 개 query family, 30-60 개 고유 페이지, 100-300 개 정규화 evidence chunk 다. 동일 페이지와 재인용 문서는 중복 제거한다.

조사 축	찾아야 하는 정보	Minecraft 계획으로의 변환
정체성과 버전	대상 게임/모드가 무엇인지, 플랫폼·시기·모드/원작 구분	잘못된 동명이인·버전 혼합 방지
핵심 플레이 루프	전투·탐험·수집·퀘스트·성장 반복	tick/event/state machine 과 acceptance test
조작·시점·이동	2D/3D, 카메라, 점프·대시·포탈	Minecraft 기본 조작 유지/카메라 모드/GUI 대안
직업·스킬·전투	역할, 능력, 자원, 쿨다운, 피해 구조	등록 객체·network payload·HUD·particle·animation
성장·장비·경제	레벨, 장비 단계, 강화, 제작, 상점	데이터 모델·loot/recipe·밸런스·저장
맵·월드 구조	허브, 지역, 던전, 포탈, 보스 동선	World Design IR·structure·biome·world.zip
퀘스트·NPC·서사	진행 조건, 대화, 보상, 분기	quest graph·dialog·advancement·NPC state
보스·던전	페이지, 기믹, 전투 공간, 보상	arena 제약·AI state·GameTest·Mineflayer 경로
멀티플레이·사회	파티, 길드, 거래, 경쟁·협력	서버 권한·동기화·데이터베이스·부하 테스트
UI·아트·음성	HUD, 메뉴, 모델, 애니메이션, 사운드, 음성 명령	asset manifest·VLM 검사·ASR/TTS skill
콘텐츠 규모	지역·몹·아이템·퀘스트의 양과 반복성	MVP/권장/확장 3 단계 범위와 T4 작업량
기술 실현성	Fabric/NeoForge/Paper/데이터팩·필수 라이브러리	아티팩트 라우팅·MCP 호출·fallback

41.3 확장 웹·RAG MCP/API/Skill 스택

MCP 는 도구와 리소스를 연결하는 프로토콜이며, “언제 왜 어떤 순서로 호출할지”를 자동으로 보장하지 않는다. 따라서 외부 MCP 를 많이 연결하는 것만으로는 범용성이 생기지 않는다. v4 는 MCP 도구 표면 위에 평가 가능한 SkillManifest 를 두고, 중앙 Router 가 현재 단계에 필요한 5-12 개 도구만 노출하도록 한다. MCP 공식 사양은 tools, resources, prompts 의 검색과 구조화된 스키마를 제공하지만 실제 연구 품질은 별도의 skill·validator·evidence store 가 책임진다. [R189-R192]

계층	채택 구성	정확한 역할
검색 발견	SearXNG + 자체 search-mcp	여러 검색 공급원의 결과 집계, query family 병렬 검색 [R193]
동적 웹 추출	Crawl4AI wrapper	JS 페이지·목록·문서에서 LLM 용 Markdown/구조화 데이터 추출 [R194]
브라우저 조작	Microsoft Playwright MCP - 격리	접근성 snapshot, 동적 탭/페이지 탐색; 읽기 전용 컨테이너 [R195]
코드·저장소	GitHub official MCP - readonly	공식 저장소·release·소스·issue·commit 근거 수집 [R196]
버전별 라이브러리 문서	Context7 MCP/Skill	최신 API 문서와 코드 예제를 exact lookup 에 공급 [R197]
PDF/문서 수집	Docling + Docling MCP	PDF·DOCX·HTML 을 구조 보존 Markdown/JSON 으로 변환 [R198]
벡터 검색	Qdrant 또는 pgvector	출처·날짜·버전 metadata filter 후 semantic retrieval [R199]
rerank	FlagEmbedding BGE 계열	검색 top-k 를 cross-encoder 로 재정렬 [R200]
관계 검색	Microsoft GraphRAG + graph store	게임 기능·맵·콘텐츠·테스트 관계와 넓은 요약 질문 [R201]
근거 통제	자체 evidence-mcp	claim 단위 출처, 모순, 신뢰도, coverage 와 유효기간 관리
기획 합성	자체 reference-game-mcp / plan-compiler-mcp	참조 Feature Graph 를 Minecraft Mod+Map Project IR 로 변환
사용자 제어	자체 approval-mcp	Proposal version, 수정 diff, 분기, 승인 hash 를 기록

“모든 오픈소스 활용”은 발견된 프로젝트를 전부 한 프로세스에 동시에 로드한다는 뜻이 아니다. capability registry 가 관련 오픈소스를 전수 후보로 관리하고, admission test 를 통과한 구성 중 현재 기능에 필요한 도구만 활성화한다. 이렇게 해야 도구 선택 오류, 프롬프트 팽창, 충돌과 공격면을 줄일 수 있다.

41.4 MCP 가 아니라 skill 계약이 범용성을 만든다

각 Skill 은 자연어 안내문이 아니라 활성화 조건, 입력 스키마, 허용 MCP, 필수 근거 축, 검증기, 종료 조건과 fallback 을 가진 실행 계약으로 저장한다. Skill 파일은 버전 관리되고 golden prompt 와 실패 fixture 로 회귀 테스트한다.

필수 Skill	활성화 조건	허용 도구와 종료 조건
universal-intake-skill	모든 최초 사용자 요청	Project Memory 만 읽고 UniversalPromptIR 생성; 불명확 핵심 필드만 질문
reference-resolver-skill	명명된 게임/모드/작품 또는 유사성 표현	SearXNG·Crawl4AI·Playwright; entity confidence 가 기준 이상일 때 종료
research-plan-skill	외부 지식 공백이 존재	query family·예산·출처 tier·coverage matrix 생성
web-evidence-skill	BROAD/EXACT research 결정	search/crawl/docling/github/context7; claim-evidence graph 완성
retrieval-correction-skill	낮은 relevance·상충·빈 축	query rewrite·다른 출처·웹 확장; CRAG 형 correction [R203]
game-feature-extraction-skill	근거 bundle 확보	Feature Graph 생성; 모든 노드에

		evidence 또는 assumption 태그
minecraft-translation-skill	Feature Graph 확정	Minecraft source/build/world/asset MCP; 불가능 요소에 대한 2 개 이상
proposal-skill	기획 coverage 와 feasibility 통과	MVP/권장/확장안, 모드+맵+에셋+테스트 +예상 시간 제시
approval-revision-skill	사용자 수정/승인 입력	diff 적용, 재조사 필요 측면 갱신, plan_hash 와 approval_hash 기록

SkillManifest 최소 필드는 다음과 같다.

skill_id, version, activate_when, reject_when, required_inputs, allowed_mcp_groups, forbidden_actions, evidence_axes, output_schema, validators, retry_policy, fallback_skill, exit_conditions, golden_tests, permission_profile

41.5 RAG 검색 품질과 반복 종료 규칙

검색을 많이 한다고 자동으로 정확해지는 것은 아니다. v4 는 Adaptive-RAG 의 질문 복잡도별 검색 선택, CRAG 의 retrieval evaluator 와 웹 보정, Self-RAG 의 필요 시 검색·근거 반성, GraphRAG 의 관계·전역 요약을 역할별로 결합한다. [R201-R204] 2026 년 연구는 의미가 같은 질문도 표현 변화에 따라 Adaptive RAG 의 검색 결정이 크게 달라질 수 있음을 보고했으므로, 입력을 canonical intent 로 정규화하고 3-5 개의 독립 query rewrite 를 합치는 규칙을 추가한다. [R205]

1. Pass 1 - Entity resolution: 정확한 대상과 동명이인을 분리하고 기본 용어를 수집한다.
2. Pass 2 - Breadth research: 플레이·맵·콘텐츠·아트·기술의 모든 축을 병렬 조사한다.
3. Pass 3 - Coverage repair: 증거가 없거나 한 출처만 있는 중요 축을 재검색한다.
4. Pass 4 - Contradiction resolution: 버전·플랫폼·시점이 다른 주장을 분리하고 최신성 또는 실행 테스트로 판정한다.
5. Pass 5 - Minecraft feasibility: 각 특징을 Fabric/NeoForge/Paper/데이터팩/월드 제작 요소로 매핑한다.
6. Pass 6 - Proposal evidence pack: 사용자에게 보이는 계획의 주요 결정마다 근거·가정·선택지를 연결한다.

검색 종료 조건은 “토큰을 많이 썼다”가 아니라 다음 계약으로 판단한다: critical coverage 100%, 중요 결정의 evidence support ratio ≥ 0.90 , unresolved contradiction 0 또는 사용자 결정 필요로 명시, Minecraft feasibility 미정 항목 0, low-confidence entity 0. 조건이 충족되지 않으면 재검색하거나 사용자에게 정확한 선택지를 묻는다.

41.6 예시: “메이플스토리 만들어줘”가 들어왔을 때

아래 예시는 특정 게임을 위한 하드코딩이 아니라 REFERENCE_GAME 입력이 들어왔을 때 모든 이름에 공통 적용되는 동작이다.

단계	시스템 행동	사용자에게 보이는 결과
1. 해석	named_entity 후보, 목표 플랫폼 =Minecraft, 요청 깊이와 누락 조건 추출	“참조 게임의 어떤 요소를 Minecraft 모드 +맵으로 옮길지 조사합니다.”
2. 자동 조사	게임 구조·전투·성장·직업·스킬·맵·퀘스트·보스·경제·UI·멀티플레이·아트/사운드 query family 실행	출처가 붙은 Reference Feature Graph
3. 구현 변환	각 기능을 Fabric/NeoForge/Paper/데이터팩/World IR/에셋 작업으로 매핑	가능/대체/선택 기능과 이유
4. 범위 생성	개인 T4 제작에 맞춘 MVP·권장·확장 세 가지 산출	콘텐츠 수, 맵 구역, 모델·사운드·테스트 수
5. 공동 계획	모드 코드만이 아니라 시작 지역, 마을/필드/던전, 포탈, NPC, 보스방, 획득 동선을 함께 설계	Game Design Plan + World Plan + Asset Plan
6. 사용자 승인	수정 문장을 기존 계획 diff 로 변환하고 필요한 측면 재검색	수정/분기/승인 버튼과 변경 영향

7. 생성 시작	승인 hash 이후에만 코드·맵·텍스처·모델·모션·음성·문서를 제작	최종 Release Bundle 과 모드 설명
----------	---------------------------------------	---------------------------

예를 들어 원작의 화면 표현을 Minecraft 에서 그대로 재현하기 어렵다면 시스템은 임의로 포기하지 않고, ① Minecraft 기본 3D RPG 형, ② 카메라·이동을 추가한 실험형, ③ 서버 친화형 콘텐츠 모드라는 구현 대안을 비교한다. 사용자가 선택한 대안이 Project IR 의 불변 조건이 된다.

41.7 계획 제안서가 반드시 포함할 항목

- 한 문장 게임 정체성, 대상 플레이어와 예상 플레이 시간
- 핵심 루프와 10 분·1 시간·장기 진행 루프
- 모드 기능, 콘텐츠 수, 직업·스킬·아이템·몹·보스·퀘스트 목록
- 전체 맵 구조, 지역 연결, 포탈, 던전, 보스 아레나와 critical path
- 텍스처, 모델, 리깅, 모션, 파티클, 사운드, 음성 인식/TTS 요구량
- Fabric/NeoForge/Paper/데이터팩/전용 world.zip 중 선택과 이유
- 필요한 오픈소스 모델·MCP·라이브러리와 fallback
- T4 순차 실행 계획, 예상 디스크·RAM·작업 단계
- 정적·빌드·GameTest·Virtual Twin·Mineflayer·VLM·성능 회귀 테스트
- 최종 모드 설명, 설치법, 플레이 가이드, 변경 내역과 알려진 한계

41.8 범용성 검증 벤치마크

이 기능은 예시 한 개가 잘 되는 것으로 합격시키지 않는다. 구현 전에 최소 60 개 golden prompt 를 만들고, 명명된 게임형·장르형·분위 기형·기능형·기존 모드 확장형·오타·다국어·상충 요구를 균형 있게 포함한다. GameGPT 는 게임 개발에서 계획·작업 식별·구현을 계층적으로 분리해 환각과 중복을 줄이는 방향을 제시했고, LLM 기반 게임 생성 연구도 규칙과 레벨을 함께 표현하는 중간 언어의 필요성을 보였다. 이 시스템은 이를 Minecraft Project IR 와 실행 테스트로 강화한다. [R206-R207]

지표	합격 기준	검증 방법
Entity resolution accuracy	명명 대상/장르/의도 $\geq 98\%$	오타·다국어·동명이인 golden set
Research trigger recall	광범위 조사 필요 prompt 누락 0	REFERENCE/GENRE/트렌드 입력 회귀
축 coverage	critical design axis 100%	coverage matrix 자동 검사
Evidence support ratio	주요 사실·결정 $\geq 90\%$	claim-evidence graph
Tool routing precision	불필요한 write MCP 호출 0, 적합 그룹 $\geq 95\%$	MCP trace audit
Plan completeness	모드·맵·에셋·테스트·문서 누락 0	GamePlan JSON schema 와 trace graph
Minecraft feasibility	미해결 필수 기능 0	source lookup·fixture·prototype
Revision locality	사용자 수정 시 영향 축만 재검색/재생성	plan diff 및 artifact hash
Prompt paraphrase stability	의미 동일 표현의 계획 핵심 일치 $\geq 95\%$	query rewrite robustness set
End-to-end utility	승인 후 최소 fixture 가 build+runtime 통과	실제 Fabric/Paper/월드 smoke test

41.9 최종 감사 판정

v4 구조에서는 사용자가 특정 게임명을 입력하든, 장르·분위기·기능만 말하든, 기존 프로젝트를 확장하든 동일한 Universal Prompt IR 에서 시작한다. 중앙 Router 가 검색 필요성을 결정하고, 웹·문서·코드·프로젝트 기억을 분리 조회하며, 검색 공백을 반복 보완한 뒤 Reference Feature Graph 와 Minecraft Mod+Map Game Plan 을 만든다. 사용자가 수정·분기·승인하기 전에는 제작 MCP 가 활성화 되지 않는다.

따라서 범용성의 핵심은 “특정 게임용 프롬프트를 많이 준비하는 것”이 아니라, 임의 입력을 연구 질문으로 분해하는 Research Planner, 증거 완전성을 확인하는 Evidence/Coverage MCP, 모든 기능을 Minecraft 제작 요소로 변환하는 Plan Compiler, 그리고 평가 가능한

SkillManifest 다. 이 네 구성요소가 구현되고 60 개 이상 범용 prompt benchmark 를 통과해야 “어떤 핵심 프롬프트가 들어와도 알아서 웹/RAG 를 사용해 계획한다”고 판정한다.

부록 F. v3 핵심 JSON Schema 와 실행 계약

F.1 GameConceptPlan 최소 스키마

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object", "additionalProperties": false,
  "required": ["project_id", "vision", "platform_lock", "core_loop", "content_budget", "world_plan", "acceptance_tests", "status"],
  "properties": {
    "project_id": {"type": "string", "pattern": "^[a-z0-9]{3,64}$"},
    "vision": {"type": "object"},
    "platform_lock": {"type": "object", "required": ["edition", "minecraft", "loader"]},
    "core_loop": {"type": "array", "minItems": 2, "maxItems": 12},
    "content_budget": {"type": "object"},
    "world_plan": {"type": "object"},
    "acceptance_tests": {"type": "array", "minItems": 1},
    "evidence_ids": {"type": "array", "uniqueItems": true},
    "plan_hash": {"type": "string", "pattern": "^[sha256:]"},
    "status": {"enum": ["PROPOSED", "APPROVED", "REJECTED", "SUPERSEDED"]}
  }
}
```

F.2 WorldDesignIR 최소 스키마

```
{
  "type": "object", "additionalProperties": false,
  "required": ["world_id", "delivery_modes", "terrain", "regions", "routes", "placements", "constraints", "navigation_tests"],
  "properties": {
    "delivery_modes": {"type": "array", "items": {"enum": ["PREBUILT_WORLD", "IN_MOD_WORLDGEN", "PATCH_EXISTING_WORLD"]}},
    "regions": {"type": "array", "minItems": 1, "items": {"type": "object", "required": ["id", "role", "footprint"]}},
    "placements": {"type": "array", "items": {"type": "object", "required": ["content_id", "location", "acquisition"]}},
    "generator": {"type": "object", "required": ["name", "version", "seed"]},
    "navigation_tests": {"type": "array"},
    "visual_tests": {"type": "array"}
  }
}
```

F.3 VirtualTwinReport

```
{
  "world_revision": "sha256:...",
  "registry_revision": "sha256:...",
  "deterministic": {
    "unknown_blocks": 0, "reference_errors": 0, "overlaps": 0, "floating_critical": 0,
    "critical_paths": {"passed": 12, "failed": 0}, "room_clearance": {"passed": 38, "failed": 0}
  },
  "renders": [{"camera_id": "boss_eye", "image": "artifact://...", "depth": "artifact://...", "block_ids": "artifact://..."}],
  "vlm_findings": [{"category": "repetition", "severity": "minor", "bbox": [0.1, 0.2, 0.4, 0.5], "repair_scope": "castle.wall_east"}],
  "limitations": ["entity_ai_not_simulated", "redstone_exactness_not_simulated"],
  "status": "PASS_FOR_RUNTIME_TEST"
}
```

F.4 ReleaseManifest

```
{
  "release_id": "frost_kingdom-1.0.0",
```

```
"plan_hash":"sha256:...","source_commit":"...","platform_lock":{},
"artifacts":[{"type":"mod_jar","path":"binaries/...jar","sha256":"..."},{"type":"world","path":"world/...zip","sha256":"..."}],
"content_counts":{"blocks":18,"items":24,"entities":5,"structures":6,"quests":12},
"games":{"build":"PASS","gametest":"PASS","virtual_twin":"PASS","runtime":"PASS","security":"PASS"},
"supported_combinations":[...],"known_issues":[...],
"documentation":[...],"sbom":"supply_chain/sbom.cdx.json","signature":"supply_chain/signatures/..."
}
```

F.5 ModDescription

```
{
  "name":"Frost Kingdom",
  "summary":"...",
  "features":[...],
  "map":{"included":true,"regions":[...],"play_time_hours":8},
  "installation":{"loader":"fabric","minecraft":"26.2","dependencies":[...]},
  "multiplayer":{"supported":true,"tested_players":4},
  "commands":[...],"config_options":[...],
  "compatibility":{"tested":[...],"untested":[...]},
  "evidence_links":{"test_report":"...","security_report":"..."},
  "known_issues":[...],"credits":[...]
}
```

부록 G. v3 추가 공식 자료와 연구 문헌

- [R142] **Fabric Developer Guides 26.2.** <https://docs.fabricmc.net/develop/>
- [R143] **Fabric Data Generation Setup.** <https://docs.fabricmc.net/develop/data-generation/setup>
- [R144] **Fabric Item Model Generation.** <https://docs.fabricmc.net/develop/data-generation/item-models>
- [R145] **Fabric Networking.** <https://docs.fabricmc.net/develop/networking>
- [R146] **Fabric Entity Model/Goals/Animation.** <https://docs.fabricmc.net/develop/entities/first-entity>
- [R147] **NeoForge Resources.** <https://docs.neoforged.net/docs/1.21.8/resources/>
- [R148] **NeoForge Mod Files.** <https://docs.neoforged.net/docs/gettingstarted/modfiles/>
- [R149] **NeoForge Game Tests.** <https://docs.neoforged.net/docs/misc/gametest>
- [R150] **Amulet Core.** <https://github.com/Amulet-Team/Amulet-Core>
- [R151] **GDPC.** <https://github.com/avdstaaij/gdpc>
- [R152] **WorldPainter.** <https://github.com/Captain-Chaos/WorldPainter>
- [R153] **Chunky Minecraft Renderer.** <https://github.com/chunky-dev/chunky>
- [R154] **jMc2Obj.** <https://github.com/jmc2obj/j-mc-2-obj>
- [R155] **PrismarineJS ecosystem / minecraft-data / chunks / bots.** <https://github.com/PrismarineJS>
- [R156] **HeadlessMc and MC-Runtime-Test.** <https://github.com/headlesshq/headlessmc>
- [R157] **MineStudio.** <https://github.com/CraftJarvis/MineStudio>
- [R158] **MineDojo.** <https://github.com/MineDojo/MineDojo>
- [R159] **MC-Planner.** <https://github.com/CraftJarvis/MC-Planner>
- [R160] **Voyager.** <https://arxiv.org/abs/2305.16291>
- [R161] **JARVIS-1.** <https://arxiv.org/abs/2311.05997>
- [R162] **MineExplorer.** <https://arxiv.org/abs/2605.30931>
- [R163] **World-GAN.** <https://arxiv.org/abs/2106.10155>
- [R164] **GDMC: Settlement Generation Challenge.** <https://arxiv.org/abs/1803.09853>
- [R165] **Automated Isovist and Spatial Quality Evaluation.** <https://arxiv.org/abs/2204.03752>
- [R166] **VIGA: Vision-as-Inverse-Graphics Agent.** <https://arxiv.org/abs/2601.11109>
- [R167] **BlenderRAG.** <https://arxiv.org/abs/2605.00632>
- [R168] **SimWorlds.** <https://arxiv.org/abs/2607.01766>
- [R169] **Qwen3-VL.** <https://github.com/QwenLM/Qwen3-VL>
- [R170] **Qwen3-VL Embedding/Reranker.** <https://github.com/QwenLM/Qwen3-VL-Embedding>
- [R171] **FLUX inference repository.** <https://github.com/black-forest-labs/flux>
- [R172] **Hunyuan3D-2.1.** <https://github.com/Tencent-Hunyuan/Hunyuan3D-2.1>
- [R173] **Stable Audio 3.** <https://arxiv.org/abs/2605.17991>
- [R174] **Modrinth Project API.** <https://docs.modrinth.com/api/operations/createproject/>
- [R175] **Modrinth Version API.** <https://docs.modrinth.com/api/operations/createversion/>
- [R176] **FastAsyncWorldEdit / WorldEdit compatible editing.** <https://github.com/IntellectualSites/FastAsyncWorldEdit>
- [R177] **Blender MCP Server (Blender Labs).** <https://www.blender.org/lab/mcp-server/>
- [R178] **MinecraftLM.** <https://github.com/mattzh72/minecraftlm>
- [R179] **Arnis real-world to Minecraft.** <https://github.com/louis-e/arnis>
- [R180] **Sinytra Compatibility Checker limitations.** <https://connector.sinytra.org/compatibility>
- [R181] **Cuberite version support.** <https://github.com/cuberite/cuberite>
- [R182] **Luanti open voxel engine.** <https://www.luanti.org/>
- [R183] **Plancraft.** <https://github.com/gautierdag/plancraft>
- [R184] **MineWorld: Real-time Open-Source Interactive World Model on Minecraft.** <https://github.com/microsoft/mineworld>
- [R185] **VIGA executable write-run-render-compare-revise loop.** <https://github.com/Fugtemypt123/VIGA>
- [R186] **SimWorlds deterministic verifier and runtime-state inspection.** <https://arxiv.org/abs/2607.01766>
- [R187] **BLOCK: Open-Source Character-to-Minecraft-Skin Pipeline.** <https://arxiv.org/abs/2603.03964>
- [R188] **Word2Minecraft: Story-to-Playable Minecraft Levels.** <https://arxiv.org/abs/2503.16536>

부록 H. v4 범용 프롬프트·웹/RAG·Skill/MCP 추가 자료

- [R189] MCP Draft Tools Specification. <https://modelcontextprotocol.io/specification/draft/server/tools>
- [R190] MCP Draft Resources Specification. <https://modelcontextprotocol.io/specification/draft/server/resources>
- [R191] MCP Draft Prompts Specification. <https://modelcontextprotocol.io/specification/draft/server/prompts>
- [R192] Official MCP Registry. <https://github.com/modelcontextprotocol/registry>
- [R193] SearXNG. <https://github.com/searxng/searxng>
- [R194] Crawl4AI. <https://github.com/unclecode/crawl4ai>
- [R195] Microsoft Playwright MCP. <https://github.com/microsoft/playwright-mcp>
- [R196] GitHub Official MCP Server. <https://github.com/github/github-mcp-server>
- [R197] Context7 MCP and Skills. <https://github.com/upstash/context7>
- [R198] Docling MCP. <https://github.com/docling-project/docling-mcp>
- [R199] Qdrant Vector Database. <https://github.com/qdrant/qdrant>
- [R200] FlagEmbedding and BGE rerankers. <https://github.com/FlagOpen/FlagEmbedding>
- [R201] Microsoft GraphRAG. <https://github.com/microsoft/graphrag>
- [R202] Self-RAG. <https://arxiv.org/abs/2310.11511>
- [R203] Corrective Retrieval Augmented Generation. <https://arxiv.org/abs/2401.15884>
- [R204] Adaptive-RAG. <https://aclanthology.org/2024.naacl-long.389/>
- [R205] How You Ask Matters: Adaptive RAG Robustness to Query Variations. <https://arxiv.org/abs/2604.10745>
- [R206] GameGPT: Multi-agent Collaborative Framework for Game Development. <https://arxiv.org/abs/2310.08067>
- [R207] Game Generation via Large Language Models. <https://arxiv.org/abs/2404.08706>
- [R208] Voyager: An Open-Ended Embodied Agent with Large Language Models. <https://arxiv.org/abs/2305.16291>

42. 2026 최신 모델 레지스트리와 역할 분리 교정

이 장은 v4 까지 남아 있던 모델 선택 오류를 폐기한다. 시스템의 중심은 단일 Qwen 모델이 아니라, 최신 모델을 자동 발견하고 각 역할에서 실측 평가한 뒤 교체하는 Dynamic Model Router 다. 모델 이름은 구현 예시이며 고정 계약이 아니다.

42.1 올바른 지능 계층

- Research/Design Supervisor: 사용자 핵심 프롬프트 해석, 광범위 웹 조사 계획, 게임 시스템 분해, 모드·맵 공동 설계, 범위와 우선 순위 결정을 담당한다. 일반 추론·장기 작업·도구 사용이 강한 모델만 허용한다.
- Plan Critic: 다른 model family 가 계획의 누락, 모순, Minecraft 부적합, 과도한 범위, 플레이 진행 막힘을 독립 심사한다.
- Implementation Engineer: 승인된 task graph 와 관련 파일만 받아 Java/JSON/Datagen/Gradle 코드를 구현한다. 기획 결정을 임의로 바꾸지 않는다.
- Multimodal Reviewer: screenshot, voxel render, UV, GUI, animation frame, map route 를 검사한다. 생성기와 동일한 단일 모델의 자기평가만으로 합격시키지 않는다.
- Deterministic Validators: JSON Schema, compiler, GameTest, pathfinder, NBT round-trip, image diff, profiler 가 최종 진실의 근거다.

42.2 2026-07 역할별 모델 후보

역할	Frontier/원격 후보	T4 로컬 후보	선택 근거
기획·연구	GLM-5.2; Kimi K3/K2.6; 최신 Kimi K3 는 challenger 로만	Qwen3.5-9B 4-bit; Gemma 4 12B QAT	장기 작업, 웹/도구, 멀티모달, 구조화 계획

	등록		
코드·저장소 수리	Kimi K2.7-Code; Qwen3-Coder-Next; GLM-5.2	Qwen3.5-9B 4-bit	agentic coding 과 repo 작업; 계획 역할과 분리
VLM·맵/GUI 검사	Kimi K3/K2.6 원격; Qwen3-VL 상위 모델	Qwen3-VL-8B-Instruct 4-bit	공간·영상·화면 reasoning 과 grounding
RAG embedding	Qwen3-Embedding-4B/8B 원격 또는 서버	Qwen3-Embedding-0.6B	100+ 언어, 32K, instruction-aware
RAG reranking	Qwen3-Reranker-4B/8B	Qwen3-Reranker-0.6B	코드·문서·다국어 relevance 재정렬
이미지/편집	FLUX.2-dev 원격	FLUX.2-klein-4B	텍스트 생성과 편집; T4 최소 13GB 수준
3D 개념	Hunyuan3D 2.1	TripoSR 또는 없음	최종은 Blockbench cuboid compiler 로 재구성
모션 참고	Wan2.2 Animate/영상 모델	규칙 기반 keyframe solver	비디오 모델을 animation.json 에 직접 쓰지 않음
ASR/오디오 이해	Fun-ASR; SenseVoice	SenseVoiceSmall	ASR·언어·감정·audio event
TTS	CosyVoice 3.0	CosyVoice 경량 경로	다국어 zero-shot, prosody

42.3 Model Registry MCP

Model Registry MCP 는 Hugging Face Hub, 공식 GitHub, 공식 기술 보고서만 우선 조회하고 다음 필드를 정규화한다. 블로그 순위나 “최고 모델” 문구는 후보 발견에만 사용하며 채택 근거가 아니다.

```
{
  "model_id": "zai-org/GLM-5.2",
  "role_candidates": ["research_supervisor", "code_engineer"],
  "release_date": "2026-06-17",
  "license": "MIT",
  "modalities": ["text"],
  "context_tokens": 1000000,
  "tool_calling": true,
  "structured_output": "tested",
  "deployment": ["remote_api", "multi_gpu_self_host"],
  "t4_fit": false,
  "official_sources": ["model_card", "technical_report"],
  "benchmark_revision": "sha256:..."
}
```

- registry.discover_official_models(role, released_after, license_policy)
- registry.fetch_model_card(model_id, revision)
- registry.estimate_memory(model_id, quantization, context, modality)
- registry.run_role_benchmark(model_id, suite_id, sandbox_profile)
- registry.promote_champion(role, candidate, evidence_bundle)
- registry.rollback(role, previous_revision, failure_reason)

42.4 역할별 실측 벤치마크와 자동 교체

모델은 공개 벤치마크 순위만으로 선택하지 않는다. Minecraft 제작에 맞춘 내부 골든셋을 모델 revision 마다 실행한다.

역할	필수 평가	가중치	실패 시
기획	참조 게임 분해, 모드+맵 완성, 요구 충돌, 사용자 수정 반영	품질 45 / 근거 20 / 안정성 20 / 비용·속도 15	Plan Critic 재검토 또는 다른 family 로 fallback
웹 연구	query decomposition, source coverage, 최신성, 충	coverage 35 / 정확도 35 / 인 용 20 / 비용 10	검색 재계획

	돌 해결		
코드	compile, GameTest, repair success, regression leakage	실행 성공 60 / 최소 patch 20 / 속도 10 / 비용 10	다른 coding model 또는 사람 검토
VLM	missing texture, UV, GUI clipping, map blockage grounding	위치 정확도 40 / recall 30 / false positive 20 / 속도 10	규칙 검사+다른 VLM ensemble
이미지	스타일, UV/타일, pixel compiler 통과	결정론 검사 50 / VLM 30 / 사용자 선호 20	다른 seed/reference 또는 수동 수정

Champion 교체는 최소 3 개 seed, 동일 도구·동일 검색 결과·동일 token budget 으로 비교하고, 기존 champion 대비 역할 점수가 유의하게 개선되며 회귀 게이트가 모두 통과할 때만 허용한다.

42.5 T4 와 원격 worker 의 정확한 분업

- T4 Local Only 후보: Qwen3.5-9B Q4 또는 Gemma 4 E4B-it Q4 범용 supervisor/VLM, Qwen3.5-4B 또는 Gemma 4 E2B-it 경량 worker, 소형 embedding/reranker, FLUX.2-klein-4B 또는 SD3.5 Medium 의 단독 lease 이미지 초안, SenseVoiceSmall/faster-whisper 음성 입력. 모든 후보는 T4 실측 입장시험을 통과해야 하며 원격 추론 fallback 을 금지한다.
- Frontier Connected 후보: Claude 5 계열, Gemini 3.x 계열, DeepSeek V4, GLM-5.2, Kimi K2.6/K2.7-Code, Qwen3.6/Qwen3-Coder-Next, Gemma 4 대형판과 원격 이미지·3D·모션 생성기. 공급자 model-list 와 내부 평가를 통과한 모델만 사용한다.
- MoE 의 activated parameter 수만 보고 T4 적합성을 판단하지 않는다. 전체 가중치가 VRAM/RAM 에 저장되어야 하므로 Qwen3-Coder-30B-A3B 도 T4 기본 모델로 두지 않는다.
- 장기 문맥을 그대로 넣지 않고 Project IR, retrieval evidence, affected file slice 를 사용한다. 큰 context 는 원격 모델의 장점이 지 무제한 컨텍스트 투입 허가가 아니다.
- 로컬과 원격 endpoint 는 LiteLLM Router 뒤에 통합하고, vLLM/SGLang endpoint 는 model revision 과 JSON Schema capability 를 명시한다.

42.6 최종 수정된 호출 흐름

사용자 프롬프트

- > Universal Intake + Reference Resolver
- > Model Registry가 research_supervisor champion 선택
- > 웹/RAG 조사 + GameConceptPlan
- > 다른 family의 Plan Critic
- > 사용자 수정/승인
- > task 별 code/VLM/image/3D/audio champion 호출
- > 결정론적 validator + 실제 Minecraft runtime
- > 실패 원인별 최소 patch
- > Release Bundle + 설명서 + 재투입 기억

42.7 금지 규칙

- Qwen2.5-Coder 또는 어떤 코딩 전용 모델도 중앙 게임 기획자로 고정하지 않는다.
- 최신이라는 이유만으로 출시 직후 모델을 champion 으로 승격하지 않는다. Kimi K3 같은 신규 모델은 challenger 벤치마크부터 수행한다.
- 동일 모델이 계획 생성, 코드 생성, 시각 심사, 최종 합격을 모두 독점하지 않는다.
- 공개 benchmark 점수와 공식 모델 카드의 자기보고만으로 실제 Minecraft 성공을 주장하지 않는다.
- T4 적합성은 parameter active 수가 아니라 실제 revision·quantization·context 에서 측정한 peak VRAM 으로 판정한다.

부록 I. v5 최신 모델 공식 자료

[V5-01] GLM-5.2 official model card. <https://huggingface.co/zai-org/GLM-5.2>

[V5-02] Kimi K2.6 official model card. <https://huggingface.co/moonshotai/Kimi-K2.6>

- [V5-03] **Kimi K2.7 Code official model card.** <https://huggingface.co/moonshotai/Kimi-K2.7-Code>
- [V5-04] **Kimi K3 official model card - challenger only until validation.** <https://huggingface.co/moonshotai/Kimi-K3>
- [V5-05] **Qwen3.5-9B official model card.** <https://huggingface.co/Qwen/Qwen3.5-9B>
- [V5-06] **Gemma 4 12B official model card.** <https://huggingface.co/google/gemma-4-12B-it>
- [V5-07] **Qwen3-Coder-Next official model card.** <https://huggingface.co/Qwen/Qwen3-Coder-Next>
- [V5-08] **Qwen3-VL-8B-Instruct official model card.** <https://huggingface.co/Qwen/Qwen3-VL-8B-Instruct>
- [V5-09] **Qwen3 Embedding 0.6B official model card.** <https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>
- [V5-10] **Qwen3 Reranker 0.6B official model card.** <https://huggingface.co/Qwen/Qwen3-Reranker-0.6B>
- [V5-11] **FLUX.2-klein-4B official model card.** <https://huggingface.co/black-forest-labs/FLUX.2-klein-4B>
- [V5-12] **Hunyuan3D 2.1 official repository.** <https://github.com/Tencent-Hunyuan/Hunyuan3D-2.1>
- [V5-13] **Wan2.2 TI2V 5B official model card.** <https://huggingface.co/Wan-AI/Wan2.2-TI2V-5B>
- [V5-14] **SenseVoice official repository.** <https://github.com/FunAudioLLM/SenseVoice>
- [V5-15] **CosyVoice official repository.** <https://github.com/FunAudioLLM/CosyVoice>
- [V5-16] **LiteLLM Router official documentation.** <https://docs.litellm.ai/docs/routing-load-balancing>
- [V5-17] **vLLM online serving documentation.** https://docs.vllm.ai/en/stable/serving/online_serving/
- [V5-18] **SGLang structured output documentation.**
https://docs.sglang.ai/advanced_features/structured_outputs.html

43. v6 이중 배치 프로파일: Frontier Connected 와 T4 Local Only

핵심 교정: “가장 좋은 최신 모델을 쓸 수 있는 환경”과 “T4 한 장만 있는 환경”을 같은 모델 목록으로 설명하지 않는다. 두 환경은 동일한 Project IR, RAG, MCP Skill, 승인 게이트, 테스트와 릴리스 형식을 공유하지만 모델 실행 정책은 완전히 다르다.

Minecraft 완전형 제작 AI - 이중 배치 프로파일

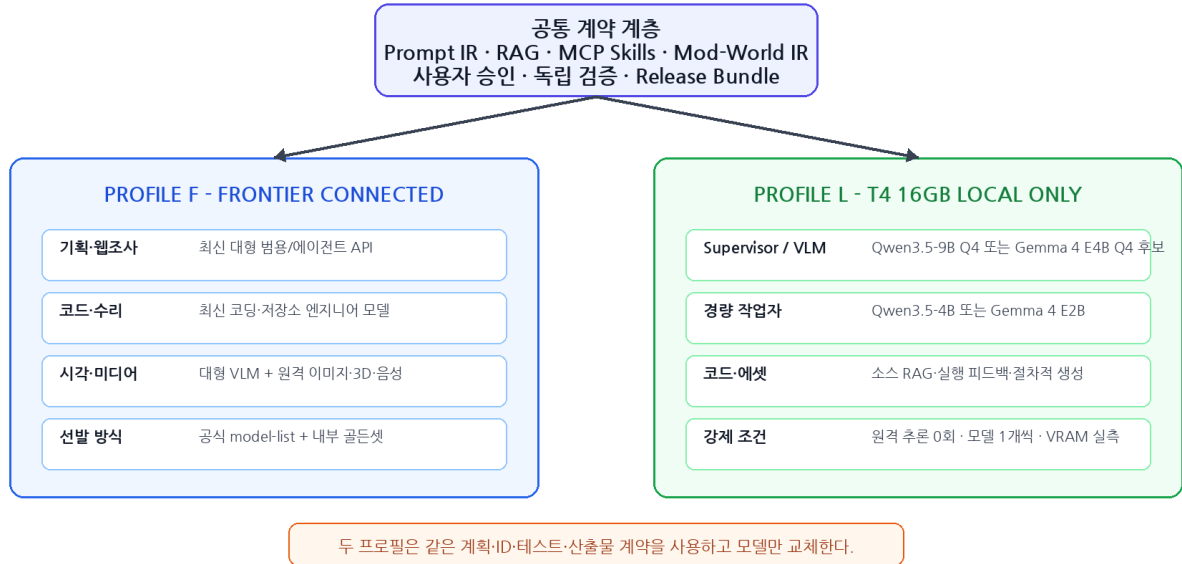


그림 12. 공동 제작 계약 위에 분리된 Frontier Connected / T4 Local Only 실행 프로파일

43.1 공통 계약: 모델이 바뀌어도 프로젝트는 바뀌지 않는다

- Universal Prompt IR 가 사용자 입력, 참조 게임·장르·기능, 조사 필요성, 모호성, 제약을 보존한다.
- Research Plan 은 웹/RAG 검색 질문, 신뢰 가능한 출처 등급, coverage 목표와 중단 조건을 정의한다.
- GameConceptPlan 과 Mod-World IR 가 전투·성장·직업·퀘스트·맵·월드젠·UI·에셋·음성·멀티플레이·테스트를 하나의 ID 그래프로 연결한다.
- 사용자 승인 전에는 코드·맵·에셋 대량 생성을 시작하지 않는다.
- 생성 모델의 자기평가가 아니라 빌드, Datagen, GameTest, 런타임, Mineflayer, VLM, 성능·보안 검사 증거로 합격한다.
- 두 프로파일 모두 같은 Release Bundle 과 ModDescription 을 생성하므로, 로컬에서 시작한 프로젝트를 원격 품질 모드로 올리거나 그 반대로 내려도 상태가 유지된다.

43.2 PROFILE F - Frontier Connected

인터넷 및 외부 AI 추론이 허용되는 품질 우선 프로파일이다. 중앙 모델을 한 회사나 한 모델명에 고정하지 않고 공급자별 model-list/API 문서와 공식 모델 카드를 주기적으로 수집한 뒤 역할별 골든셋으로 선별한다.

역할	2026-07 후보 풀 예시	선발 기준	실패 시 대체	금지
Research / Design Supervisor	Claude Fable/Opus/Sonnet 5; Gemini 3.1 Pro; GLM-5.2; Kimi K3/K2.6; DeepSeek V4-Pro; Qwen3.6	참조 해석, 조사 설계, 장기 계획, 구조화 출력, 도구 정확도	상위 2 개 후보의 blind 비교 후 재선정	코딩 벤치만 보고 기획자로 지정
Implementation Engineer	Claude Opus/Sonnet 5; Gemini custom-tools/3.6 Flash; Kimi K2.7-Code; Qwen3-Coder-Next; GLM-5.x; DeepSeek V4	SWE 형 수정, Java/Gradle, 저장소 탐색, 실행 피드백 수리	파일 범위 축소 후 다른 코드 champion	승인된 GameConceptPlan 변경
Multimodal QA	Gemini 3.x; Kimi K3/K2.6; 대형 Qwen/Gemma VLM	스크린샷, PDF, 영상, UI, 모델·텍스처 결함 검출	결정론적 검사 + 다른 VLM 합의	VLM 단독 합격

Media Generator	원격 이미지·3D·모션·TTS API 또는 별도 GPU worker	Minecraft 스타일, 구조적 유효성, 라이선스, 후처리 가능성	절차적 Blockbench/픽셀 파이프라인	생성 결과 원본을 무검증 배포
Research Retrieval	웹 검색, 공식 API, GitHub, 문서 crawler, Deep Research API	출처 등급, 최신성, coverage, 인용 추적	query rewrite + 다른 검색기	한 번 검색 후 종료

현재 확인된 예시 모델들은 고정 기본값이 아니다. Anthropic 은 Models API 를, DeepSeek 은 /models 를 제공하며, 다른 공급자도 공식 모델 목록 또는 문서에서 현재 사용 가능한 ID 를 조회할 수 있다. Model Registry 는 출시일만으로 승격하지 않고 동일 프로젝트 골든 셋에서 기존 champion 을 실제로 이긴 경우에만 교체한다. [V6-01]-[V6-10]

43.3 PROFILE L - T4 16GB Local Only

외부 AI 추론 API 와 원격 GPU 를 전혀 사용하지 않는 독립 프로파일이다. 웹 검색은 선택적으로 허용할 수 있지만 검색 결과의 분석·계획·코드·시각 판정은 모두 로컬 모델이 수행한다. 완전 폐쇄망에서는 사전 수집한 공식 문서/RAG snapshot 만 사용한다.

작업	로컬 후보	실행 방식	입장시험	대형 원격 기능 대체
게임 기획·RAG 종합	Qwen3.5-9B Q4 또는 Gemma 4 E4B-it Q4	한 번에 하나, 8K-16K 작업 context, 장기 문서는 요약 계층화	계획 coverage, JSON 유효성, peak VRAM, 2 시간 안 정성	계층형 planning + 다중 pass + 결정론적 coverage 검사
라우팅·추출·요약	Qwen3.5-4B 또는 Gemma 4 E2B-it	CPU/GPU 선택, 짧은 context	분류 정확도와 schema 오류율	규칙 기반 router 와 JSON Schema
코드·수리	9B 범용 모델 + Minecraft source RAG + compiler trace; 필요 시 Qwen3-4B-Thinking	파일 단위 patch, 실행 후 재수리	첫 빌드, 3 회 내 수리율, 회귀 유출	작업을 더 작게 분해하고 template/Datagen 사용
시각 검사	Qwen3.5-9B 또는 Gemma 4 E4B 를 동일 lease 에서 재사용	고정 camera·저해상도 montage·영역 crop	누락 텍스처/UV/UI 골든셋	pixel diff, schema, geometry 검사
이미지·텍스처 초안	FLUX.2-klein-4B/FP8 또는 SD3.5 Medium 후보	별도 lease 후 unload; 최종은 palette quantization	peak VRAM, 16x16 후처리 품질	절차적 픽셀 템플릿·팔레트 합성
3D·애니메이션	Blockbench JSON compiler, GeckoLib template, Blender headless	생성형 3D 를 필수로 두지 않음	bone/UV/hitbox/animation validator	모듈형 cube-bone 문법
음성 입력	SenseVoiceSmall 또는 faster-whisper	CPU 또는 짧은 GPU lease	한국어 WER, latency	텍스트 입력 fallback

중요: Qwen3-Coder-Next 는 3B activated 라고 해도 80B total 이며, Qwen3-Coder-30B-A3B 도 전체 30.5B 가중치를 저장해야 한다. 활성 파라미터만 보고 T4 적합 모델로 놓지 않는다. T4 채택 여부는 실제 quantized checkpoint 의 전체 파일 크기, 로딩 방식, context 와 peak VRAM 으로 판정한다. [V6-08][V6-09]

43.4 배치 프로파일 선택과 네트워크 정책

```
{
  "deployment_profile": "frontier_connected | t4_local_connected | t4_air_gapped",
  "allow_remote_ai_inference": false,
  "allow_public_web_retrieval": true,
  "allow_remote_gpu_worker": false,
  "local_gpu": {"type": "NVIDIA_T4", "vram_gb": 16},
  "model_admission": {
    "max_peak_vram_gb": 13.5,
    "max_context_tokens": 16384,
    "batch_size": 1,
    "require_cold_start_test": true,
    "require_two_hour_stability_test": true
  }
}
```

- frontier_connected: 원격 AI/API, 웹, 외부 GPU worker 허용. 비용·데이터 정책·latency 를 route score 에 반영한다.
- t4_local_connected: 웹/RAG 수집은 가능하지만 AI 추론은 모두 T4/CPU 로컬에서 수행한다.
- t4_air_gapped: 네트워크를 차단하고 버전 고정 RAG snapshot, 로컬 Git mirror, 모델 cache 만 사용한다.

- 프로필은 프로젝트 중간에 바꿀 수 있지만 승인된 IR와 artifact provenance는 유지하고, 모델이 만든 새 판단은 다시 승인 diff를 거친다.

43.5 Dynamic Model Registry 의 필수 필드

```
{
  "model_id": "provider/model-or-api-id",
  "provider": "local | api | self_hosted",
  "roles": ["research_supervisor", "implementation_engineer", "visual_qa"],
  "modalities": ["text", "image", "audio", "video", "pdf"],
  "official_source": "URL",
  "release_or_update_date": "YYYY-MM-DD",
  "available_now": true,
  "deployment_profiles": ["frontier_connected"],
  "total_parameters_b": 0,
  "activated_parameters_b": 0,
  "quantization": null,
  "measured_peak_vram_gb": null,
  "tool_calling_tested": false,
  "structured_output_pass_rate": 0.0,
  "role_benchmark_score": {},
  "cost_and_latency": {},
  "status": "challenger"
}
```

모델 카드의 자체 벤치마크는 후보 등록 근거일 뿐 최종 선발 근거가 아니다. Minecraft 기획 20 건, 광범위 RAG 20 건, Fabric/NeoForge/Paper 코드 수리 30 건, 리소스·맵 시각 결합 30 건, 도구 호출 오류 주입 20 건으로 역할별 내부 점수를 계산한다.

43.6 T4 Local Only 합격시험

- 네트워크 egress를 차단한 상태에서 기획 -> 사용자 승인 mock -> 코드/Datagen -> build -> 테스트 -> 설명서 생성까지 끝낸다.
- 각 GPU 모델을 cold start로 5회 로드하고 peak VRAM, load time, 첫 토큰 latency와 unload 후 잔류 VRAM을 기록한다.
- 2시간 연속 repair loop에서 CUDA OOM, 프로세스 누수, 서버 충돌이 0회여야 한다.
- 원격 endpoint가 비활성일 때도 hidden fallback 없이 정상적으로 Local Only 경로를 선택한다.
- 같은 프롬프트를 Frontier와 Local에서 실행해 기능 coverage 차이를 보고하되, Local 결과가 누락 기능을 성공으로 위장하지 않는다.
- 생성형 이미지·3D 모델이 탈락해도 템플릿·Datagen·절차적 생성으로 완성 가능한 경로가 유지돼야 한다.

43.7 최종 호출 흐름

```
USER PROMPT
-> Universal Intake / Reference Resolver
-> Research Plan + Web/RAG retrieval
-> Research/Design Supervisor (profile-specific champion)
-> GameConceptPlan + Mod-World IR
-> USER APPROVAL
-> Task DAG
-> role-specific workers
-> deterministic generators + MCP tools
-> build/runtime/visual/security validators
-> repair loop
-> mod.jar + world.zip/worldgen + assets + tests + ModDescription + Release Bundle
```

43.8 최종 판정

이 시스템은 “Qwen 하나로 전부 한다”거나 “항상 원격 최고 모델을 쓴다”는 구조가 아니다. 원격 사용이 가능하면 최신 frontier 후보를 역할별로 실측해 품질을 높이고, T4만 허용되면 T4 입장시험을 통과한 소형 범용 멀티모달 모델과 결정론적 도구만으로 동일한 완성 계약

을 수행한다. 따라서 T4 Local Only 는 Frontier Connected 의 실패 fallback 이 아니라 독립적으로 테스트되고 배포되는 정식 제품 모드다.

부록 J. v6 모델·API 공식 자료

[V6-01] Anthropic Claude models overview - current model IDs and availability.

<https://docs.anthropic.com/en/docs/about-claude/models/overview>

[V6-02] Anthropic Models API - discover available models. <https://docs.anthropic.com/en/api/models-list>

[V6-03] Google Gemini 3.1 Pro Preview model documentation.

<https://ai.google.dev/gemini-api/docs/models/gemini-3.1-pro-preview>

[V6-04] Google Gemini models catalog. <https://ai.google.dev/gemini-api/docs/models>

[V6-05] DeepSeek V4 release and official API model IDs. <https://api-docs.deepseek.com/news/news260424/>

[V6-06] DeepSeek Models API. <https://api-docs.deepseek.com/api/list-models/>

[V6-07] Z.ai GLM-5.2 official model card. <https://huggingface.co/zai-org/GLM-5.2>

[V6-08] Moonshot Kimi K2.6 official model card. <https://huggingface.co/moonshotai/Kimi-K2.6>

[V6-09] Moonshot Kimi K2.7-Code official model card. <https://huggingface.co/moonshotai/Kimi-K2.7-Code>

[V6-10] Qwen3.6-27B official model card. <https://huggingface.co/Qwen/Qwen3.6-27B>

[V6-11] Qwen3-Coder-Next official model card - 80B total / 3B activated. <https://huggingface.co/Qwen/Qwen3-Coder-Next>

[V6-12] Qwen3.5-9B official unified multimodal model card. <https://huggingface.co/Qwen/Qwen3.5-9B>

[V6-13] Qwen3.5-4B official unified multimodal model card. <https://huggingface.co/Qwen/Qwen3.5-4B>

[V6-14] Google Gemma 4 model family. <https://deepmind.google/models/gemma/gemma-4/>

[V6-15] FLUX.2 klein 4B FP8 official model card. <https://huggingface.co/black-forest-labs/FLUX.2-klein-4b-fp8>

[V6-16] FLUX.2 klein 9B official hardware statement (~29GB VRAM). <https://huggingface.co/black-forest-labs/FLUX.2-klein-9B>

[V6-17] Stable Diffusion 3.5 Medium official model card. <https://huggingface.co/stabilityai/stable-diffusion-3.5-medium>

[V6-18] SenseVoiceSmall official model card. <https://huggingface.co/FunAudioLLM/SenseVoiceSmall>

[V6-19] NVIDIA T4 specifications - 16GB GDDR6. <https://www.nvidia.com/en-us/data-center/tesla-t4/>

[V6-20] Qwen3-Coder-30B-A3B official model card - 30.5B total / 3.3B activated. <https://huggingface.co/Qwen/Qwen3-Coder-30B-A3B-Instruct>

[V6-21] Moonshot Kimi K3 official model card - 2.8T native multimodal frontier model.

<https://huggingface.co/moonshotai/Kimi-K3>